# Deployment strategies

03

# Down for everyone or just me?



# Is the application available?



Is the application available?



Is the application available?



Is the application available?




Is the application available?

# Push vs Pull

## Push vs Pull

# Startup probe

# Liveness probe

# Readiness probe

## Push vs Pull

Startup probe

**Liveness probe**

Readiness probe

## Push vs Pull

Startup probe

Liveness probe

**Readiness probe**

# 3 probes



# /start

ENDPOINT



/start

ENDPOINT

/health

ENDPOINT



**/start**

ENDPOINT

**/health**

ENDPOINT

**/ready**

ENDPOINT





**/start**
ENDPOINT

**Startup**
PROBE

**/health**
ENDPOINT

**/ready**
ENDPOINT





/start
ENDPOINT

Startup
PROBE

/health
ENDPOINT

Liveness
PROBE

/ready
ENDPOINT





| | |
|---|---|
| **/start**<br>ENDPOINT | **Startup**<br>PROBE |
| **/health**<br>ENDPOINT | **Liveness**<br>PROBE |
| **/ready**<br>ENDPOINT | **Readiness**<br>PROBE |

# Liveness probe

## Liveness probe

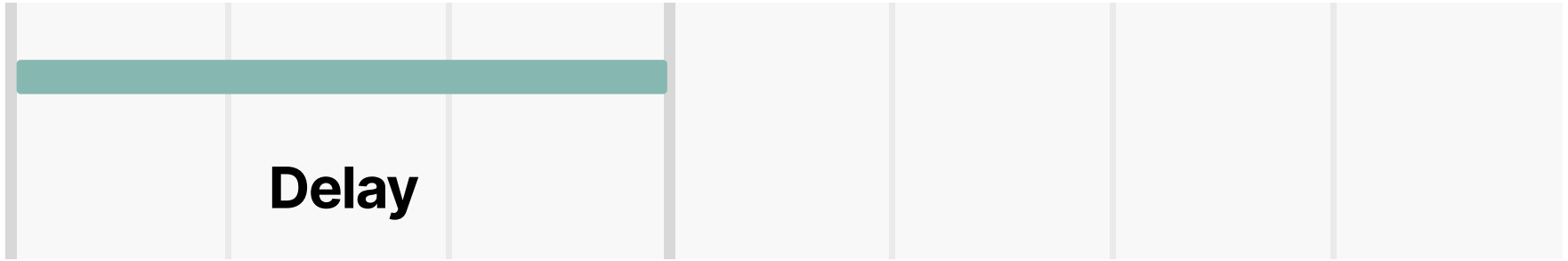
# Process started

# Process healthy

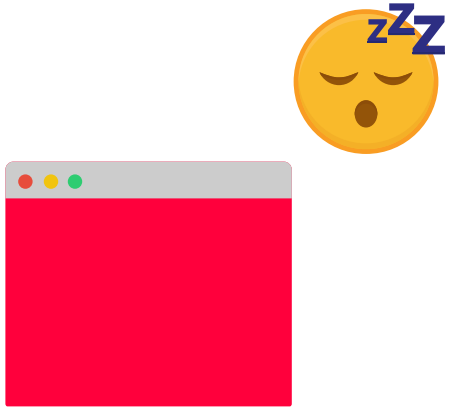
## Liveness probe

# Process started
# Process healthy

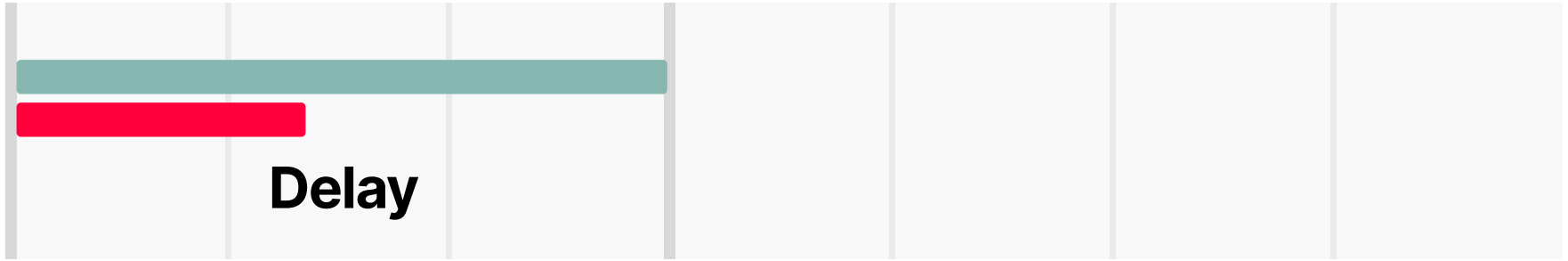
# Liveness probe



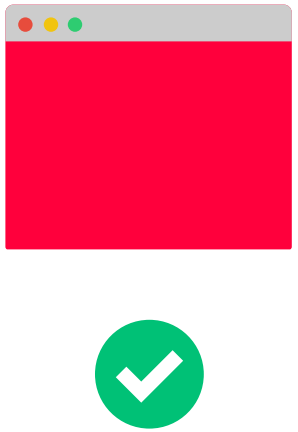

```yaml
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 2
  successThreshold: 1
  periodSeconds: 10
  initialDelaySeconds: 30
```



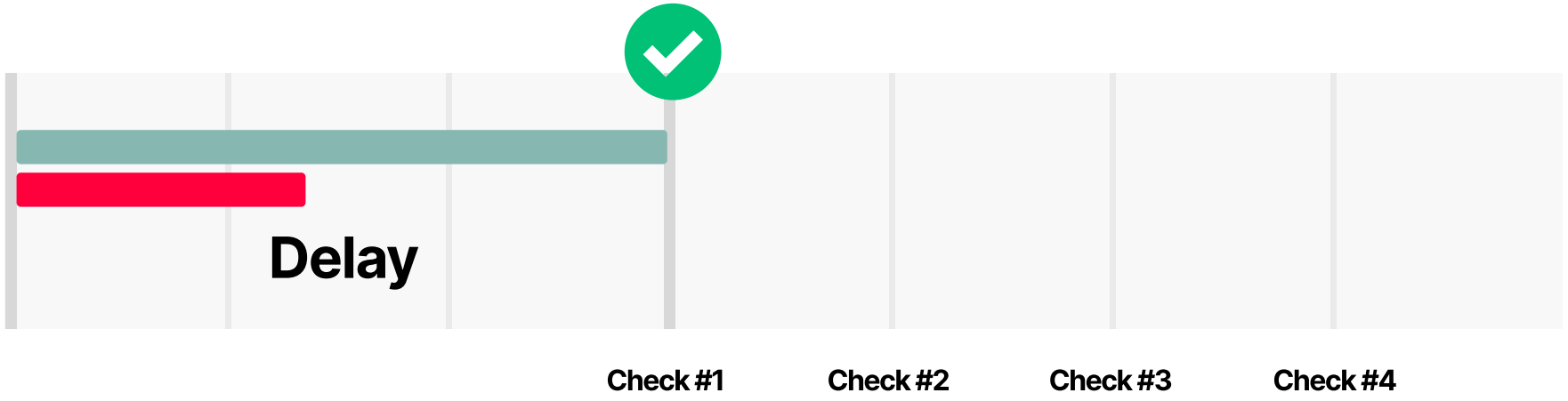
# Liveness probe




```yaml
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 2
  successThreshold: 1
  periodSeconds: 10
  initialDelaySeconds: 30
```



# Liveness probe

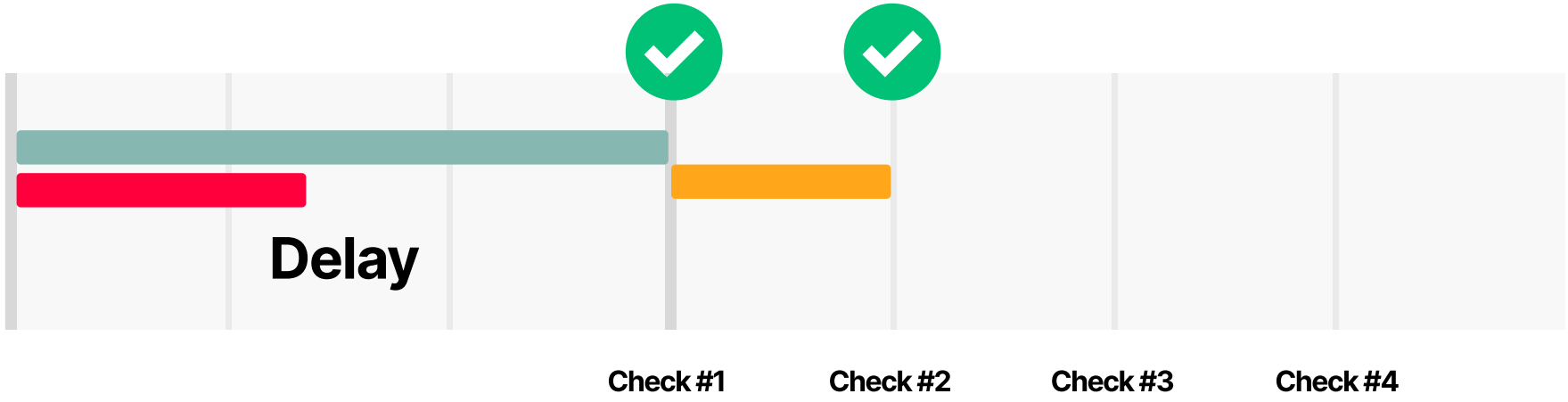



```yaml
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 2
  successThreshold: 1
  periodSeconds: 10
  initialDelaySeconds: 30
```

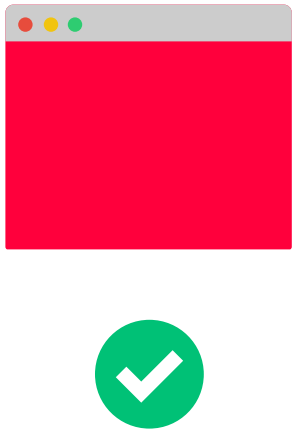




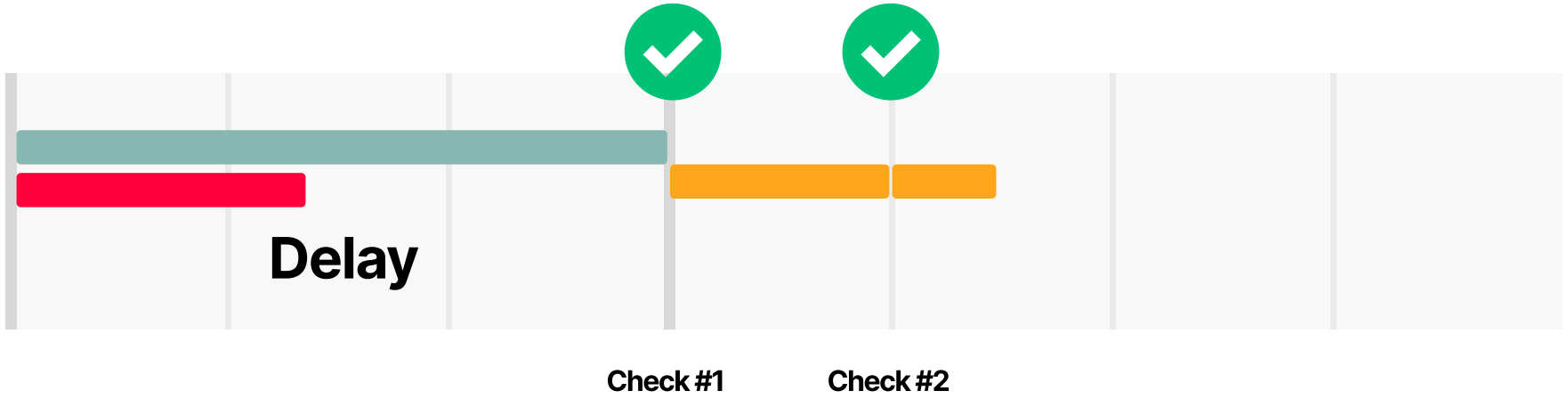
# Liveness probe



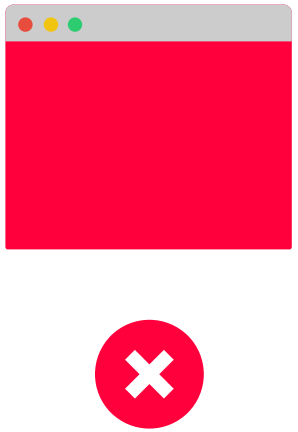

```yaml
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 2
  successThreshold: 1
  periodSeconds: 10
  initialDelaySeconds: 30
```



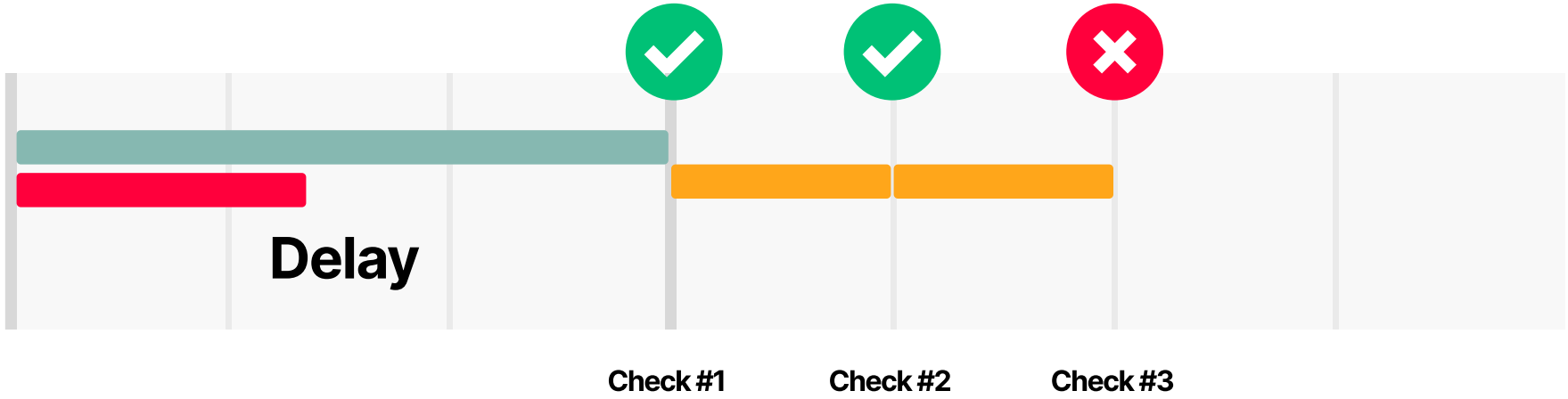
# Liveness probe



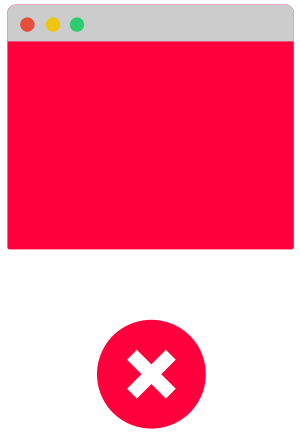

```yaml
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 2
  successThreshold: 1
  periodSeconds: 10
  initialDelaySeconds: 30
```



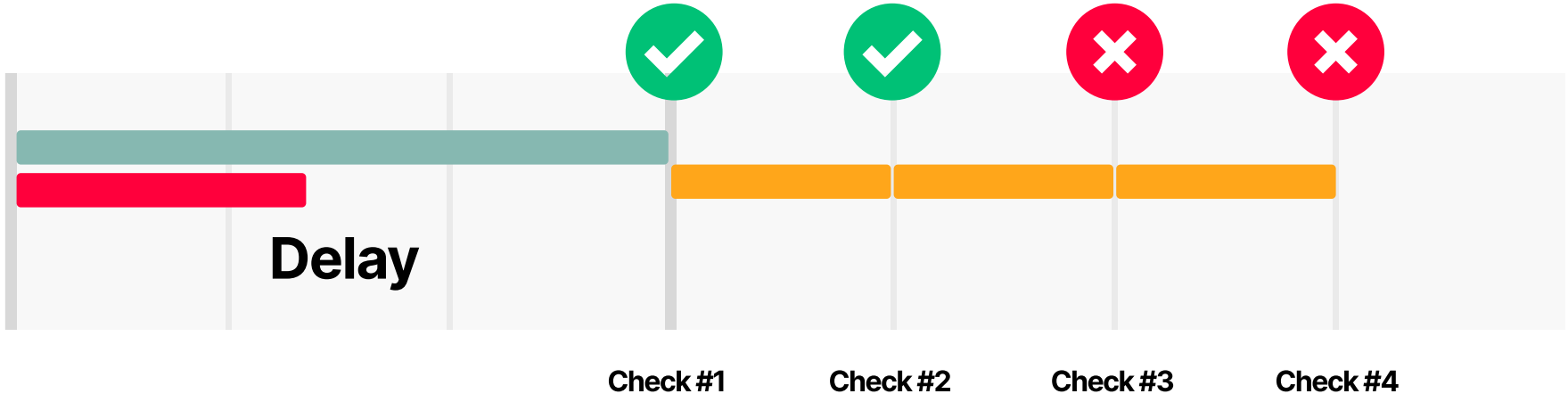
# Liveness probe



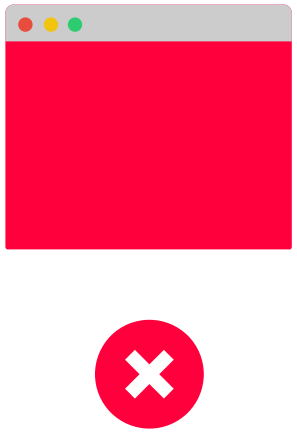

```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 2
  successThreshold: 1
  periodSeconds: 10
  initialDelaySeconds: 30
```



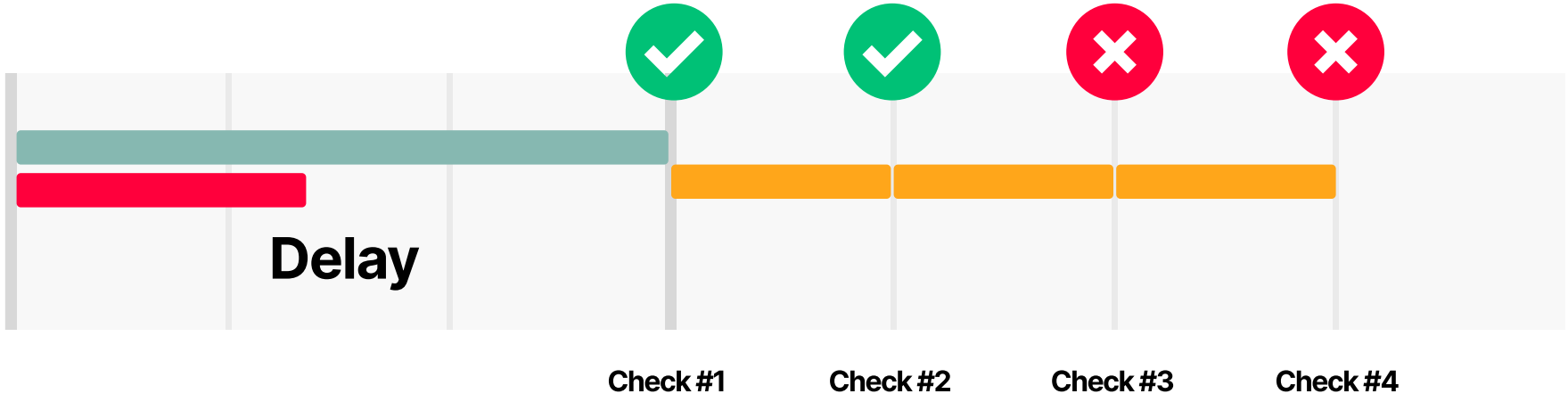
# Liveness probe



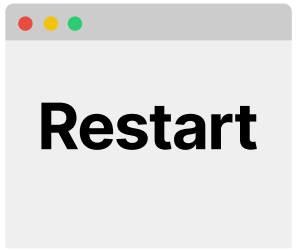

```yaml
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 2
  successThreshold: 1
  periodSeconds: 10
  initialDelaySeconds: 30
```



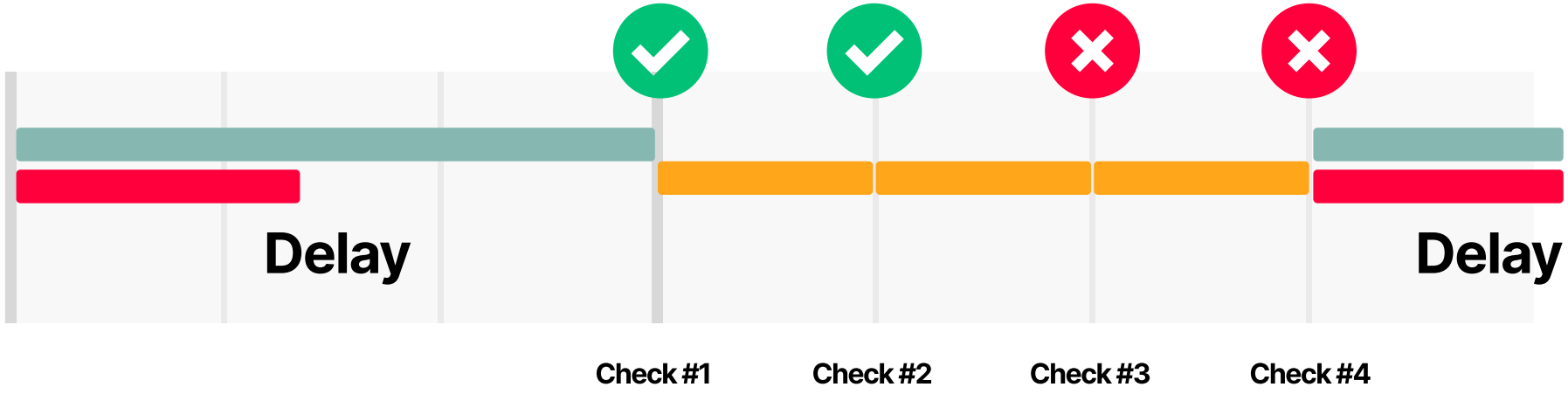
# Liveness probe



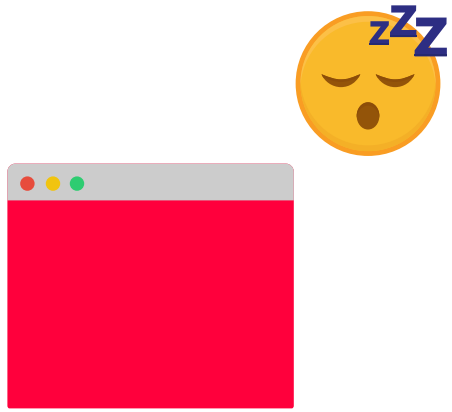

```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 2
  successThreshold: 1
  periodSeconds: 10
  initialDelaySeconds: 30
```



# Liveness probe

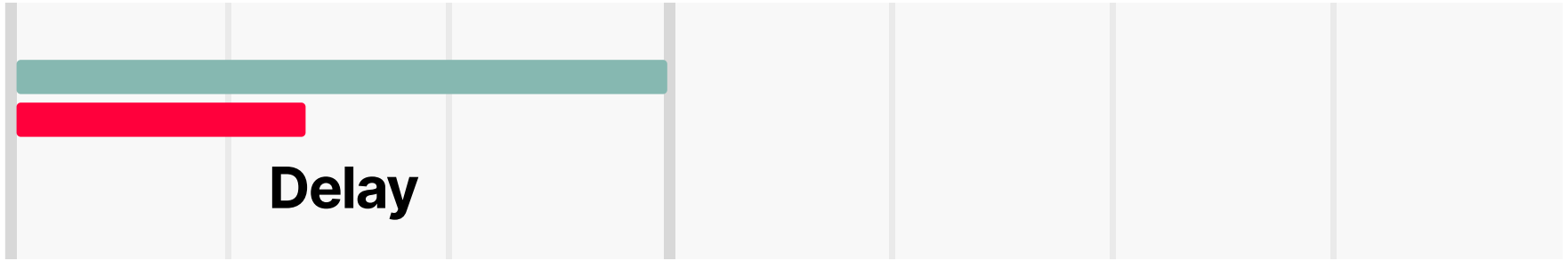



```yaml
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 2
  successThreshold: 1
  periodSeconds: 10
  initialDelaySeconds: 30
```

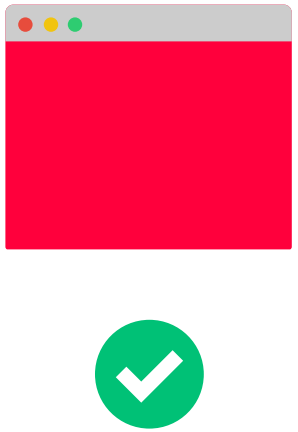


# Startup probe

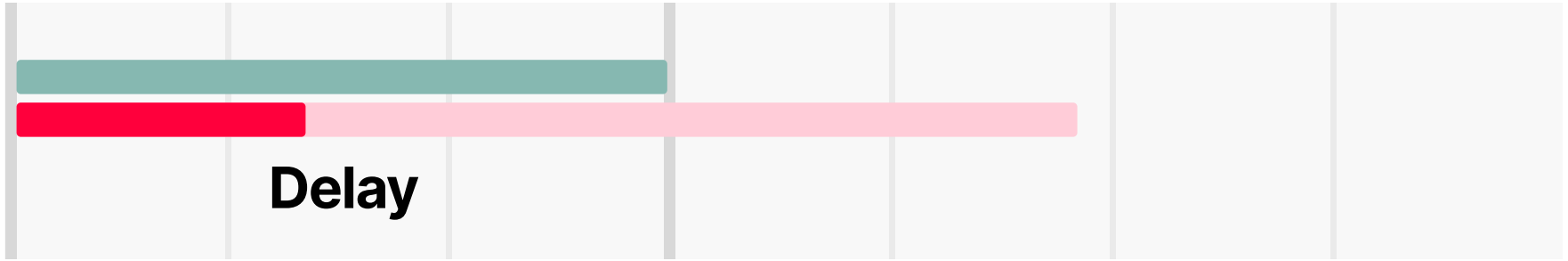
# Startup probe



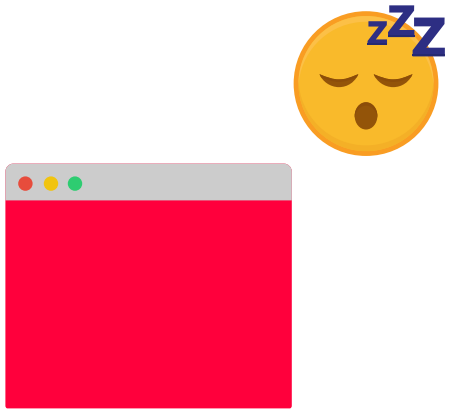

```yaml
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 2
  successThreshold: 1
  periodSeconds: 10
  initialDelaySeconds: 30
```



# Startup probe

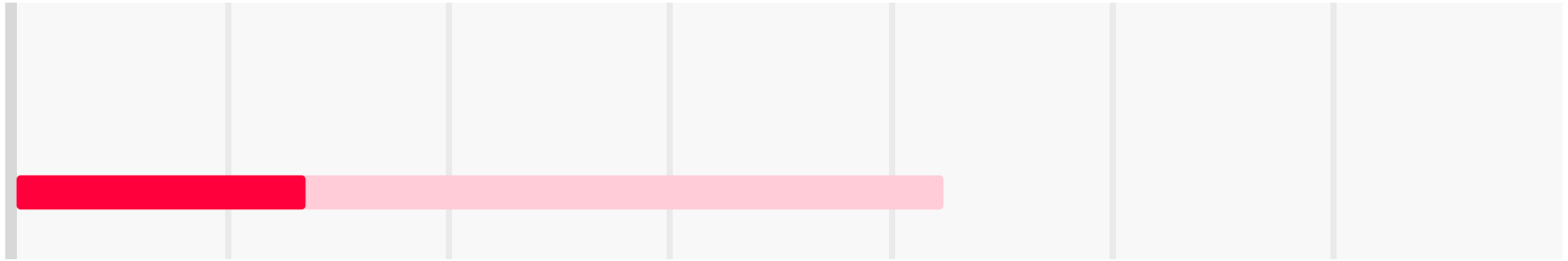



```yaml
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 2
  successThreshold: 1
  periodSeconds: 10
  initialDelaySeconds: 30
```

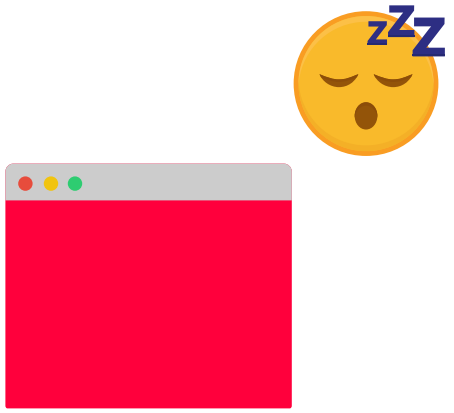


Restart

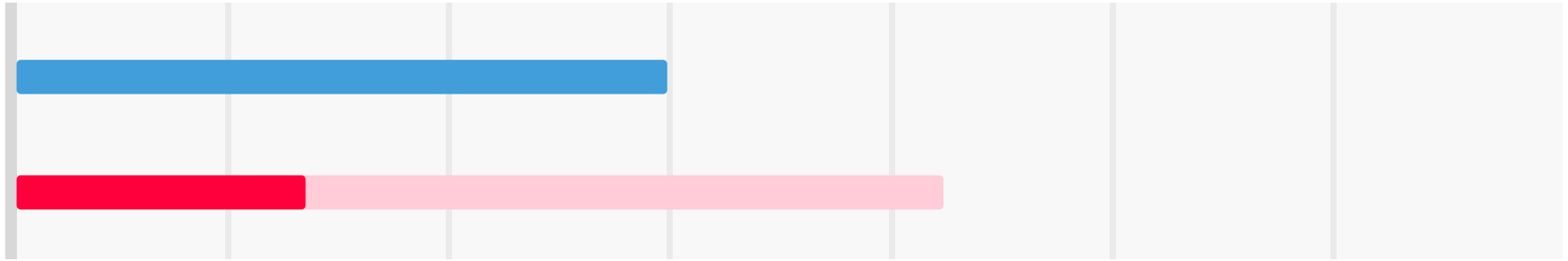
# Startup probe



```yaml
startupProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 6
  successThreshold: 1
  periodSeconds: 5
  initialDelaySeconds: 30
```



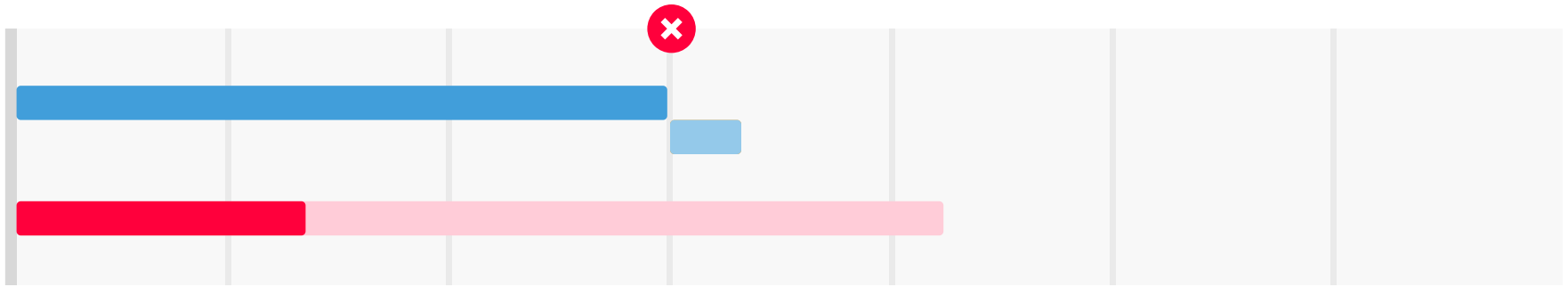
# Startup probe



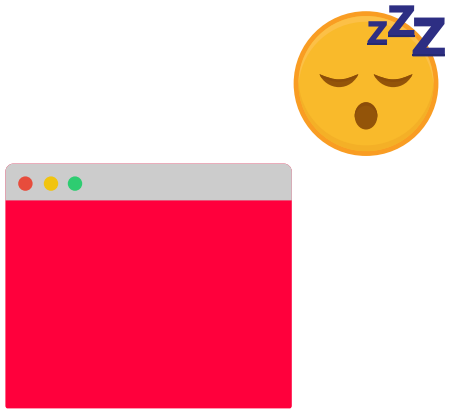
```yaml
startupProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 6
  successThreshold: 1
  periodSeconds: 5
  initialDelaySeconds: 30
```



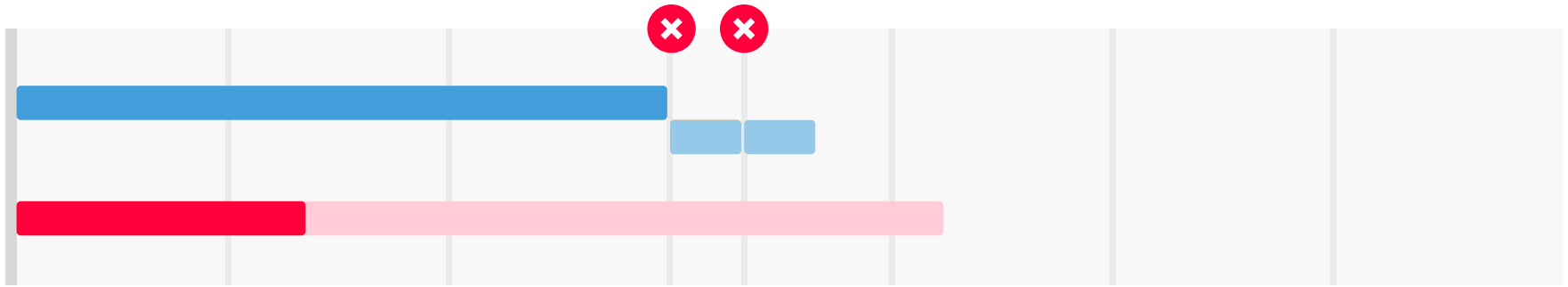
# Startup probe



```
startupProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 6
  successThreshold: 1
  periodSeconds: 5
  initialDelaySeconds: 30
```



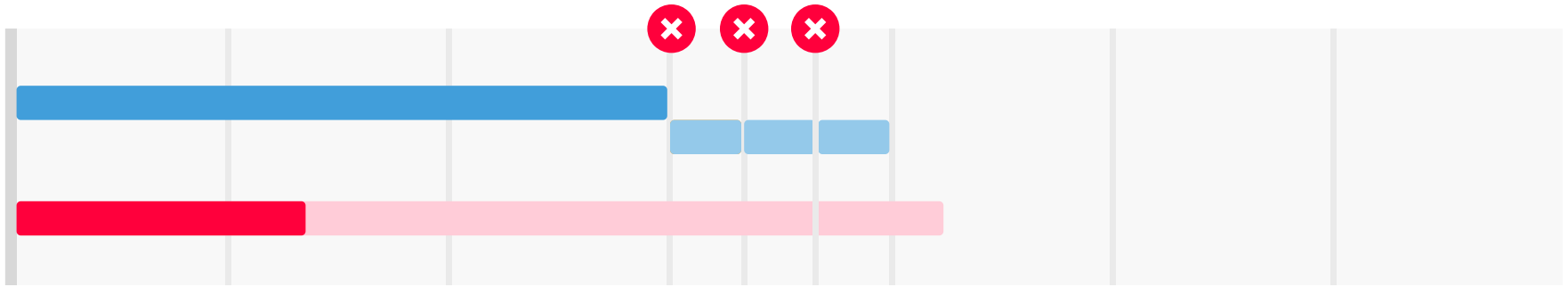
# Startup probe



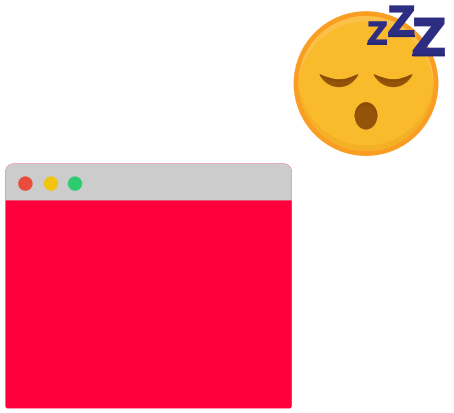
```
startupProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 6
  successThreshold: 1
  periodSeconds: 5
  initialDelaySeconds: 30
```



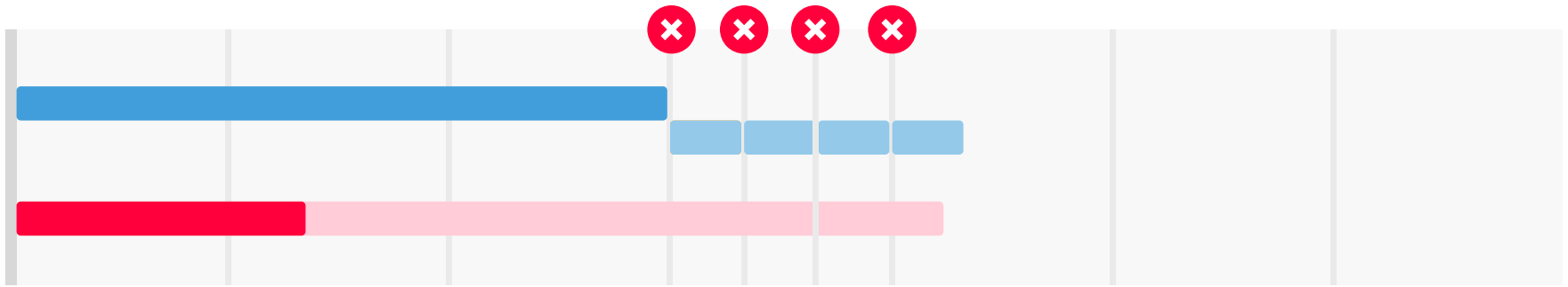
# Startup probe



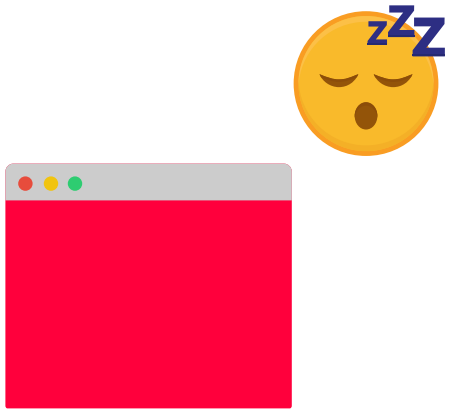
```yaml
startupProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 6
  successThreshold: 1
  periodSeconds: 5
  initialDelaySeconds: 30
```



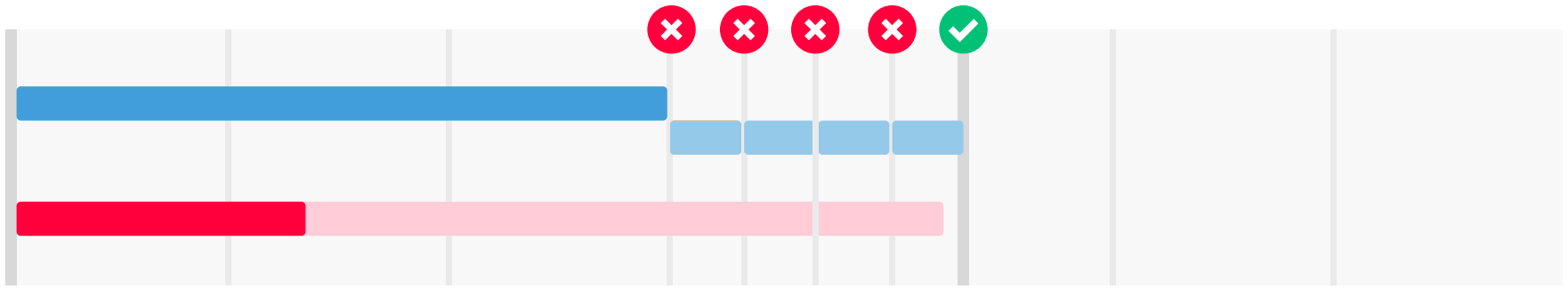
# Startup probe



```yaml
startupProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 6
  successThreshold: 1
  periodSeconds: 5
  initialDelaySeconds: 30
```



# Startup probe

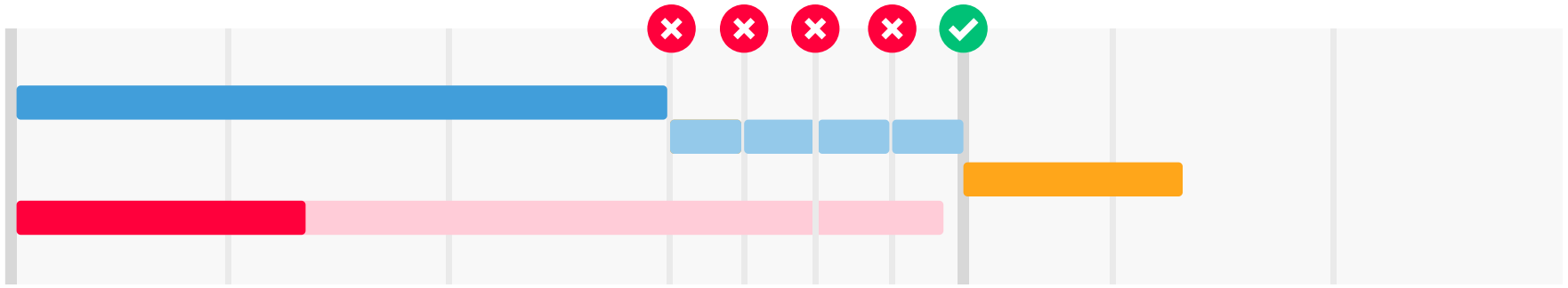


```yaml
startupProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 6
  successThreshold: 1
  periodSeconds: 5
  initialDelaySeconds: 30
```





# Startup probe

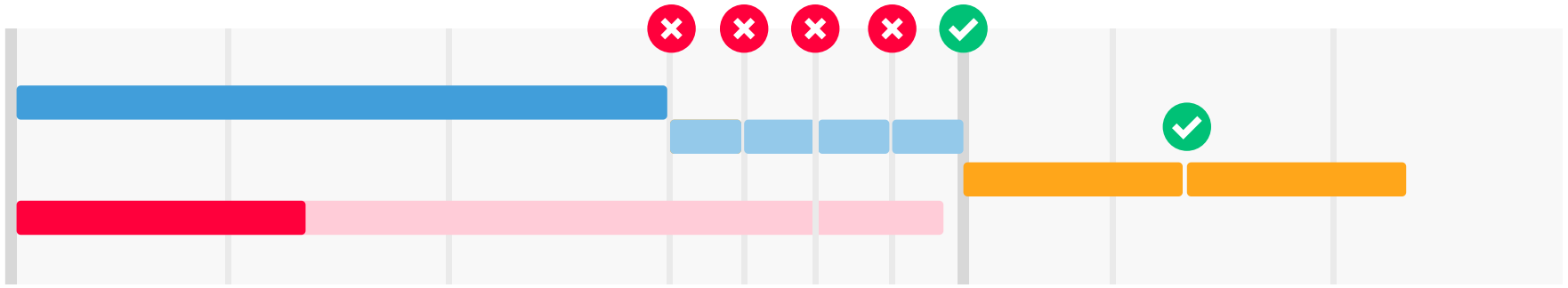


```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 2
  successThreshold: 1
  periodSeconds: 10
```





# Startup probe



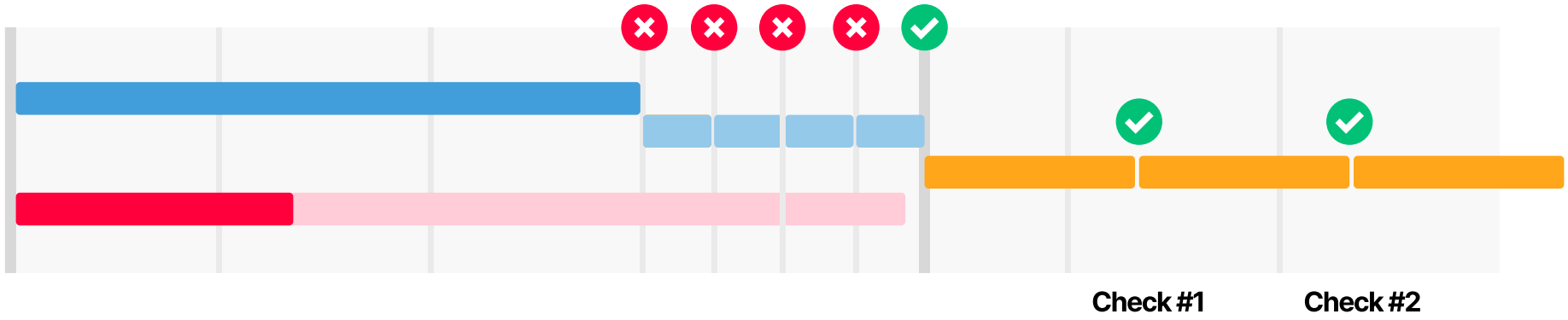
Check #1

```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 2
  successThreshold: 1
  periodSeconds: 10
```





# Startup probe

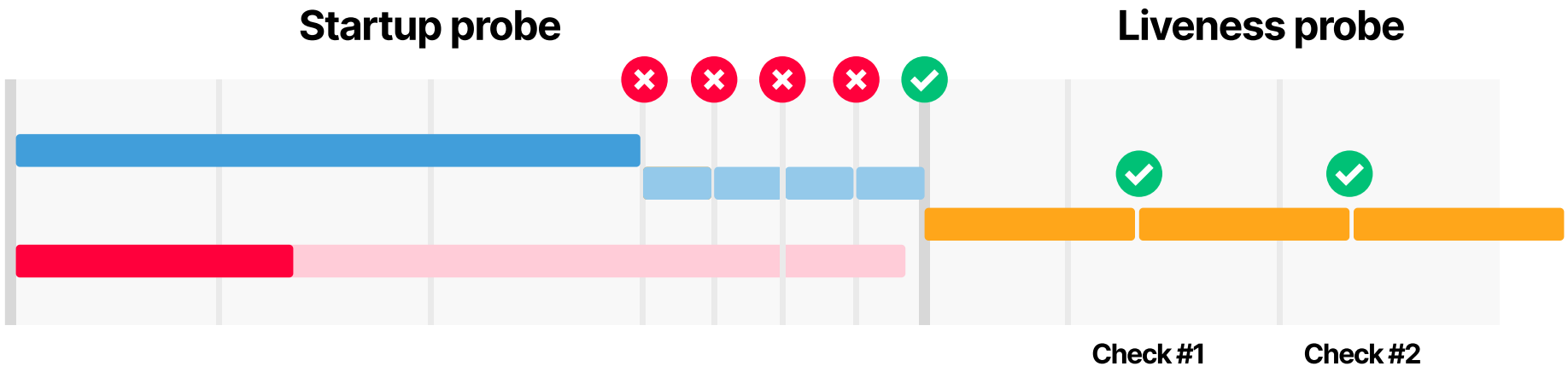



```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 2
  successThreshold: 1
  periodSeconds: 10
```





# Startup probe




```yaml
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 2
  successThreshold: 1
  periodSeconds: 10
```





# Readiness probe

## Readiness probe

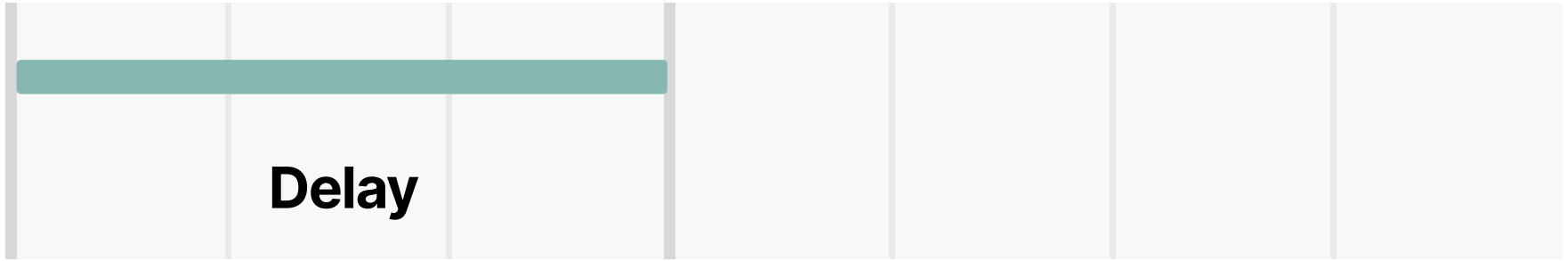
# Can receive traffic

# Load balancer pool

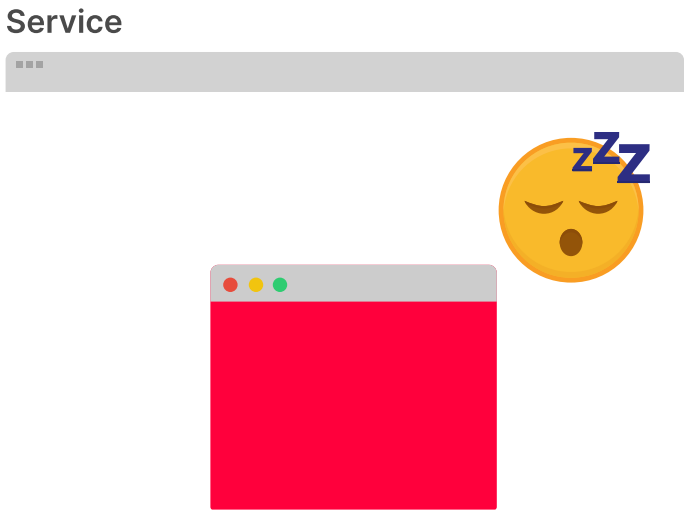
## Readiness probe

# Can receive traffic
# Load balancer pool

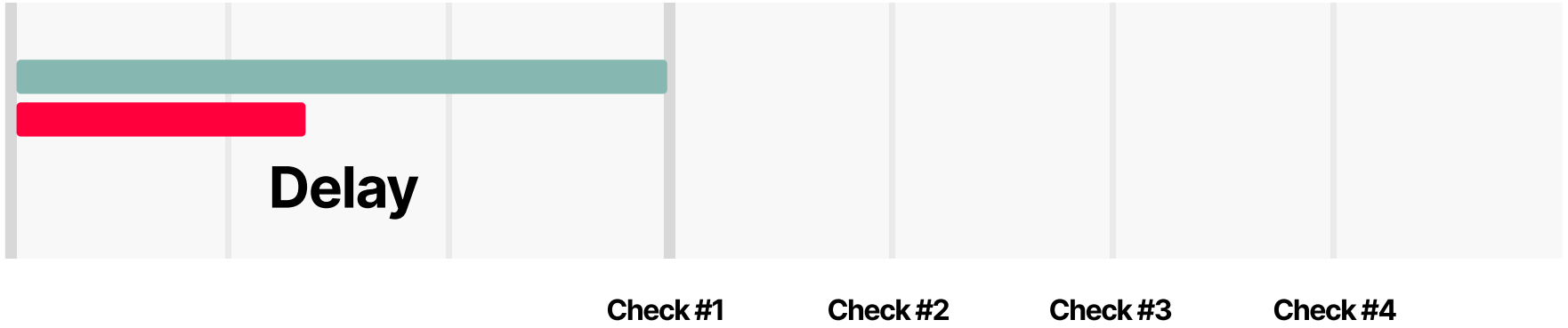
# Readiness probe



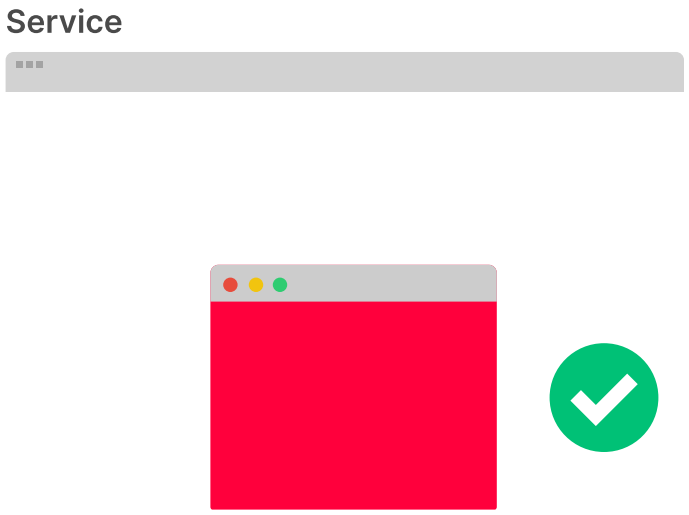

```yaml
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  failureThreshold: 2
  successThreshold: 1
  periodSeconds: 10
  initialDelaySeconds: 30
```



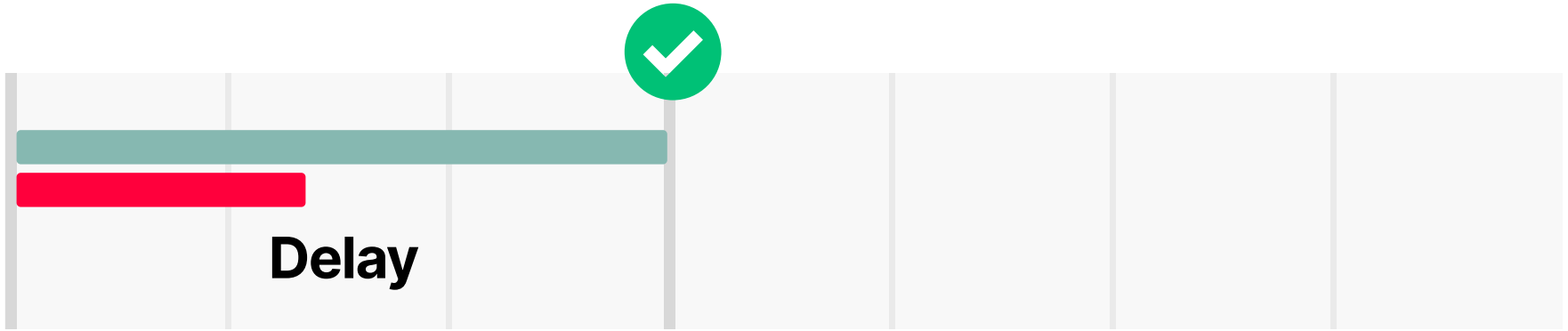
# Readiness probe




```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  failureThreshold: 2
  successThreshold: 1
  periodSeconds: 10
  initialDelaySeconds: 30
```



# Readiness probe



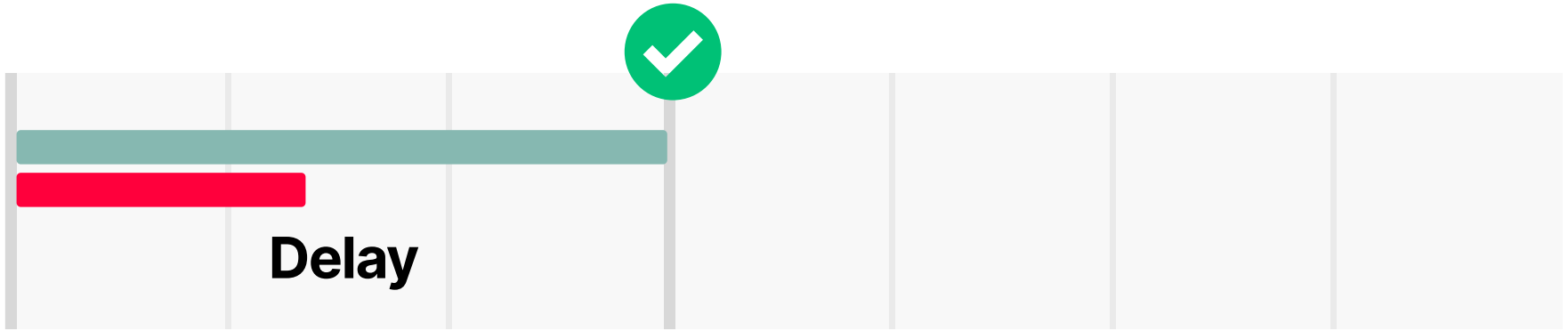

Check #1

```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  failureThreshold: 2
  successThreshold: 1
  periodSeconds: 10
  initialDelaySeconds: 30
```

Service

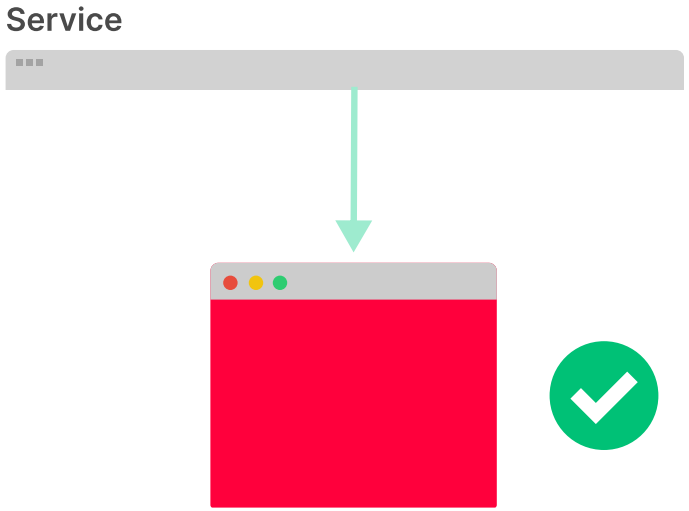


# Readiness probe

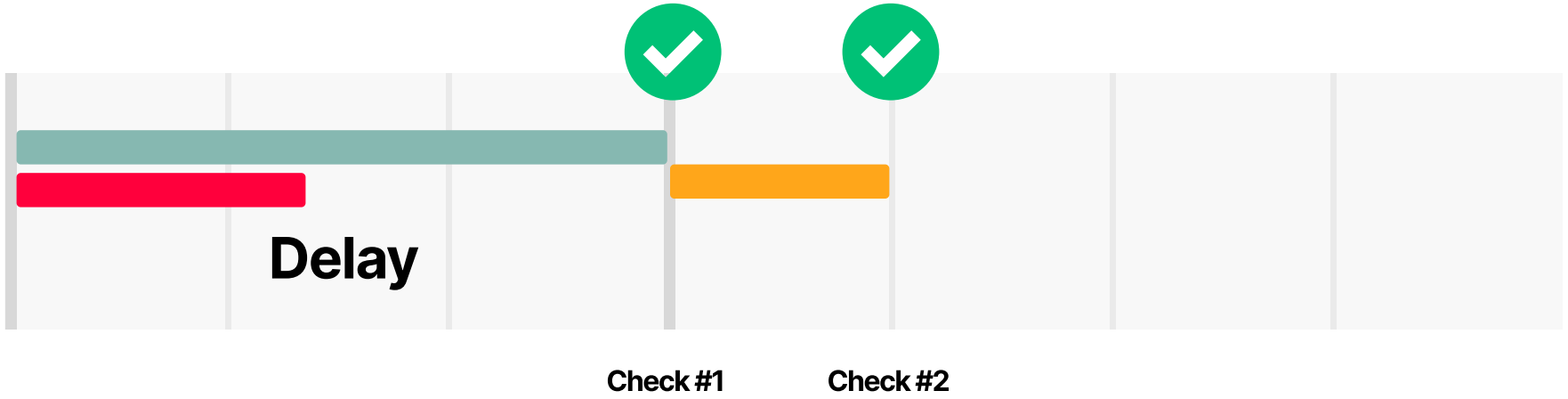



Check #1

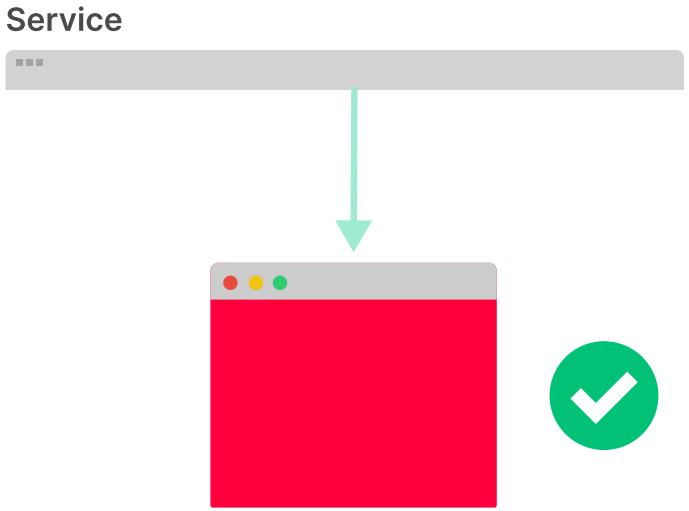
```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  failureThreshold: 2
  successThreshold: 1
  periodSeconds: 10
  initialDelaySeconds: 30
```



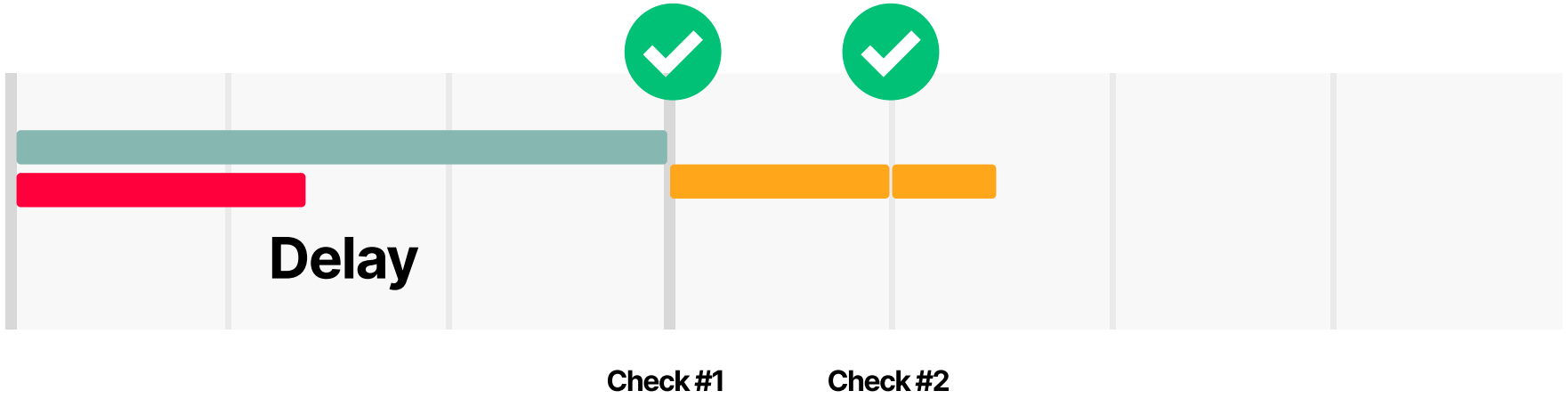
# Readiness probe



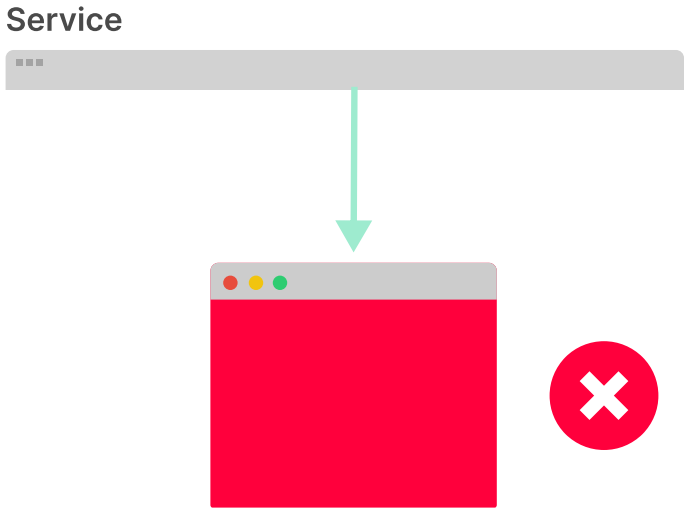

```yaml
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  failureThreshold: 2
  successThreshold: 1
  periodSeconds: 10
  initialDelaySeconds: 30
```



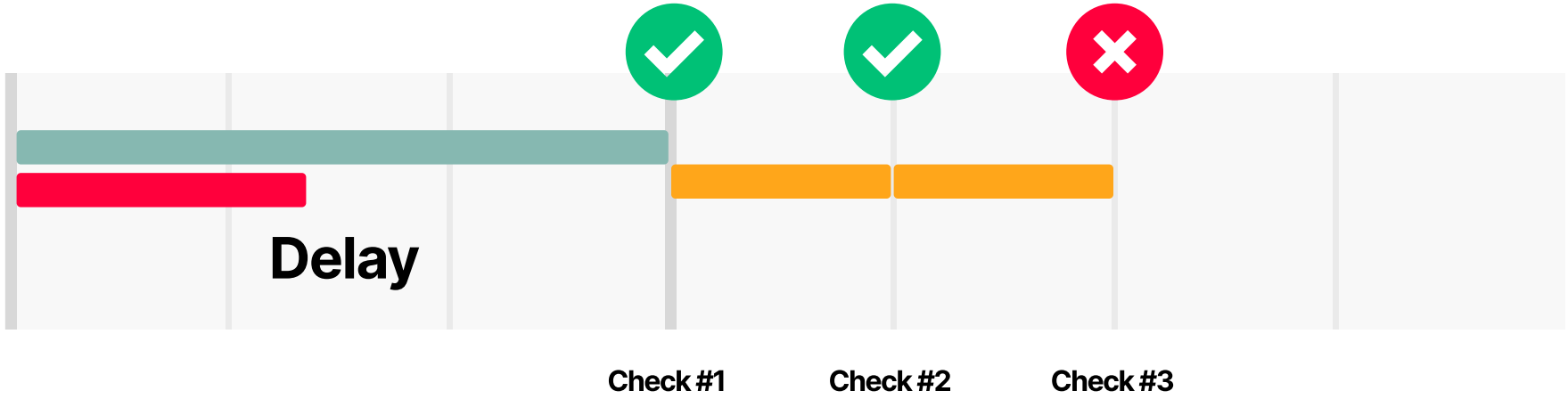
# Readiness probe



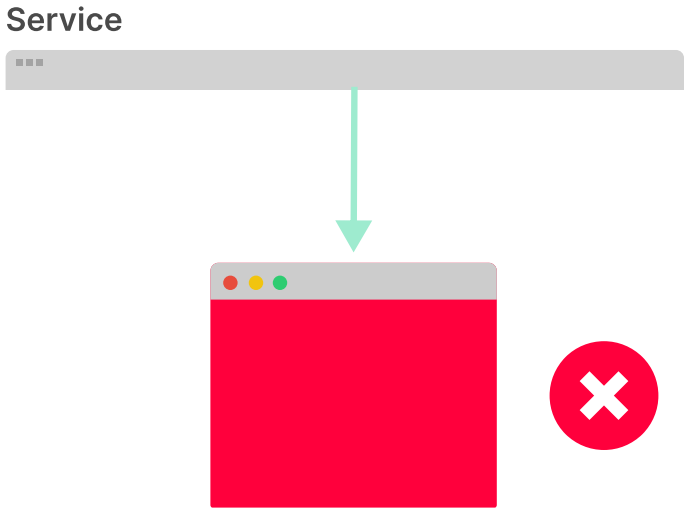

```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  failureThreshold: 2
  successThreshold: 1
  periodSeconds: 10
  initialDelaySeconds: 30
```



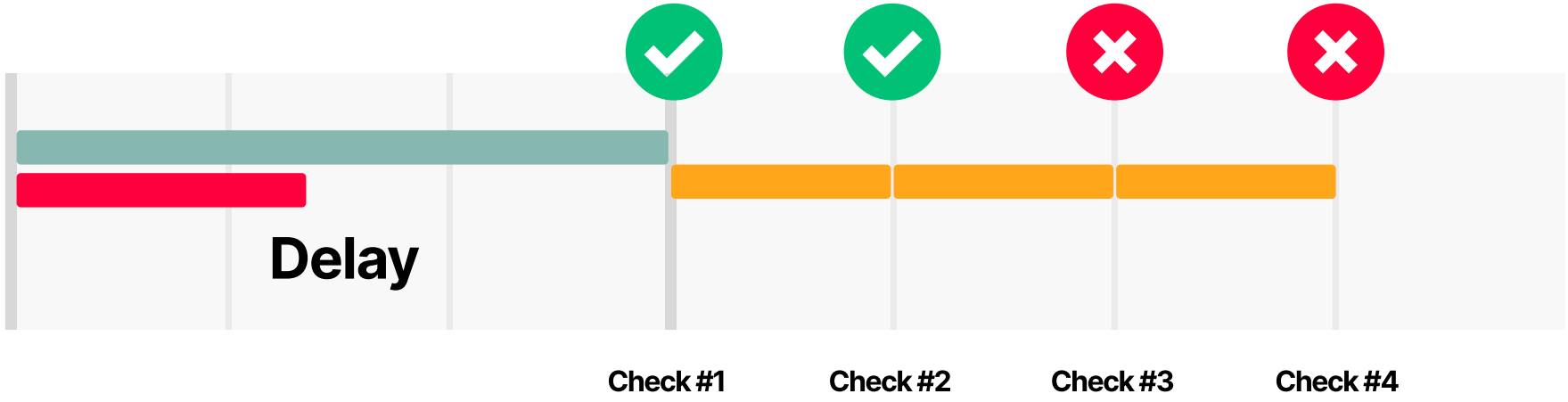
# Readiness probe



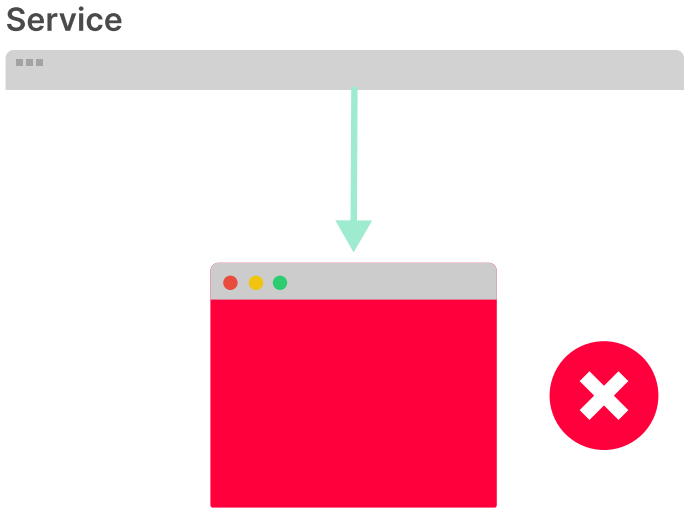

```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  failureThreshold: 2
  successThreshold: 1
  periodSeconds: 10
  initialDelaySeconds: 30
```



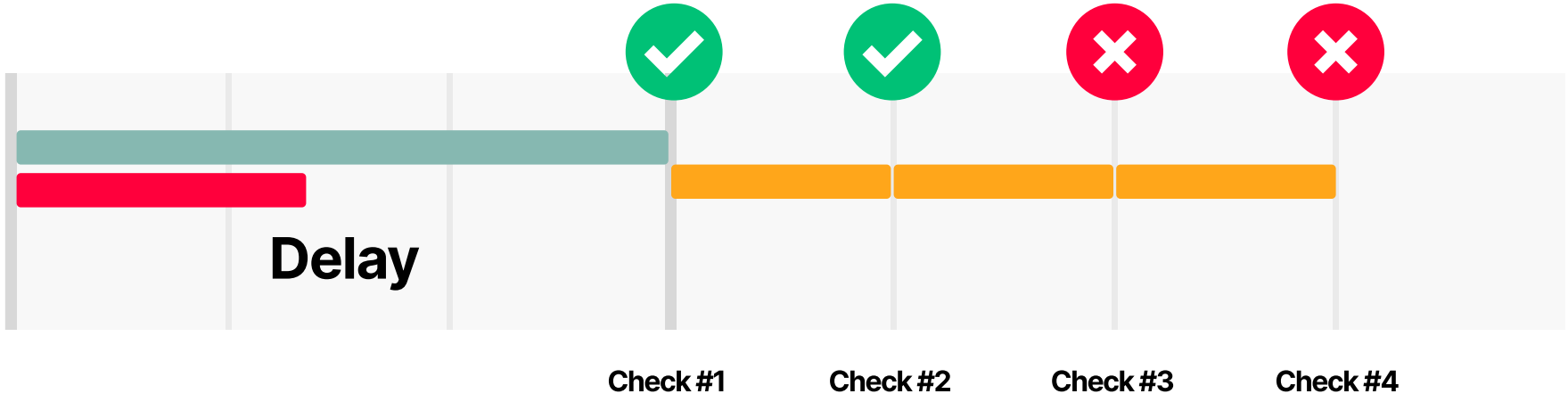
# Readiness probe




```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  failureThreshold: 2
  successThreshold: 1
  periodSeconds: 10
  initialDelaySeconds: 30
```



# Readiness probe

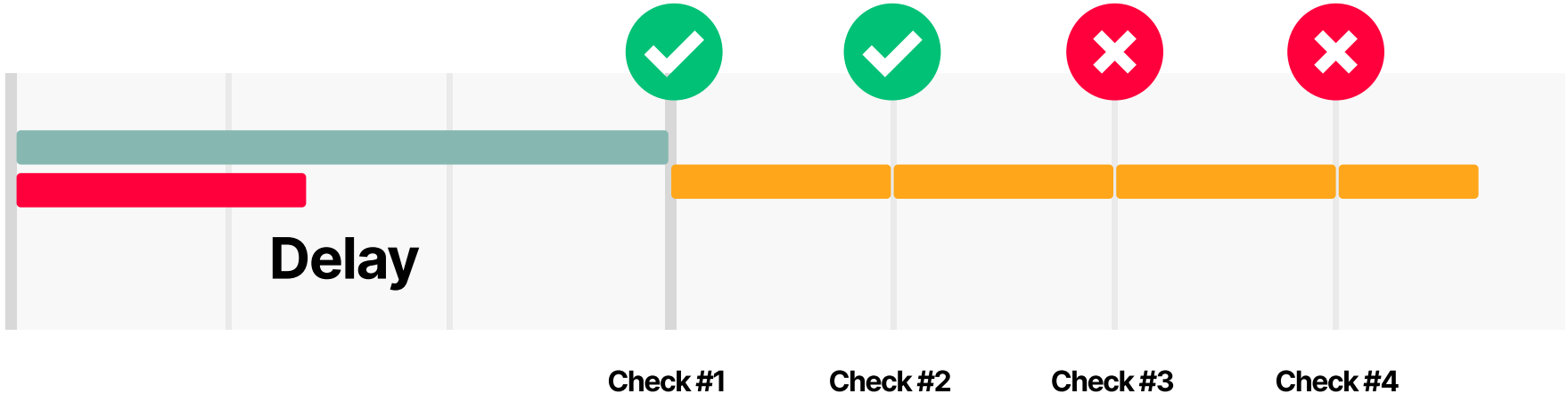



```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  failureThreshold: 2
  successThreshold: 1
  periodSeconds: 10
  initialDelaySeconds: 30
```






# Readiness probe




```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  failureThreshold: 2
  successThreshold: 1
  periodSeconds: 10
  initialDelaySeconds: 30
```



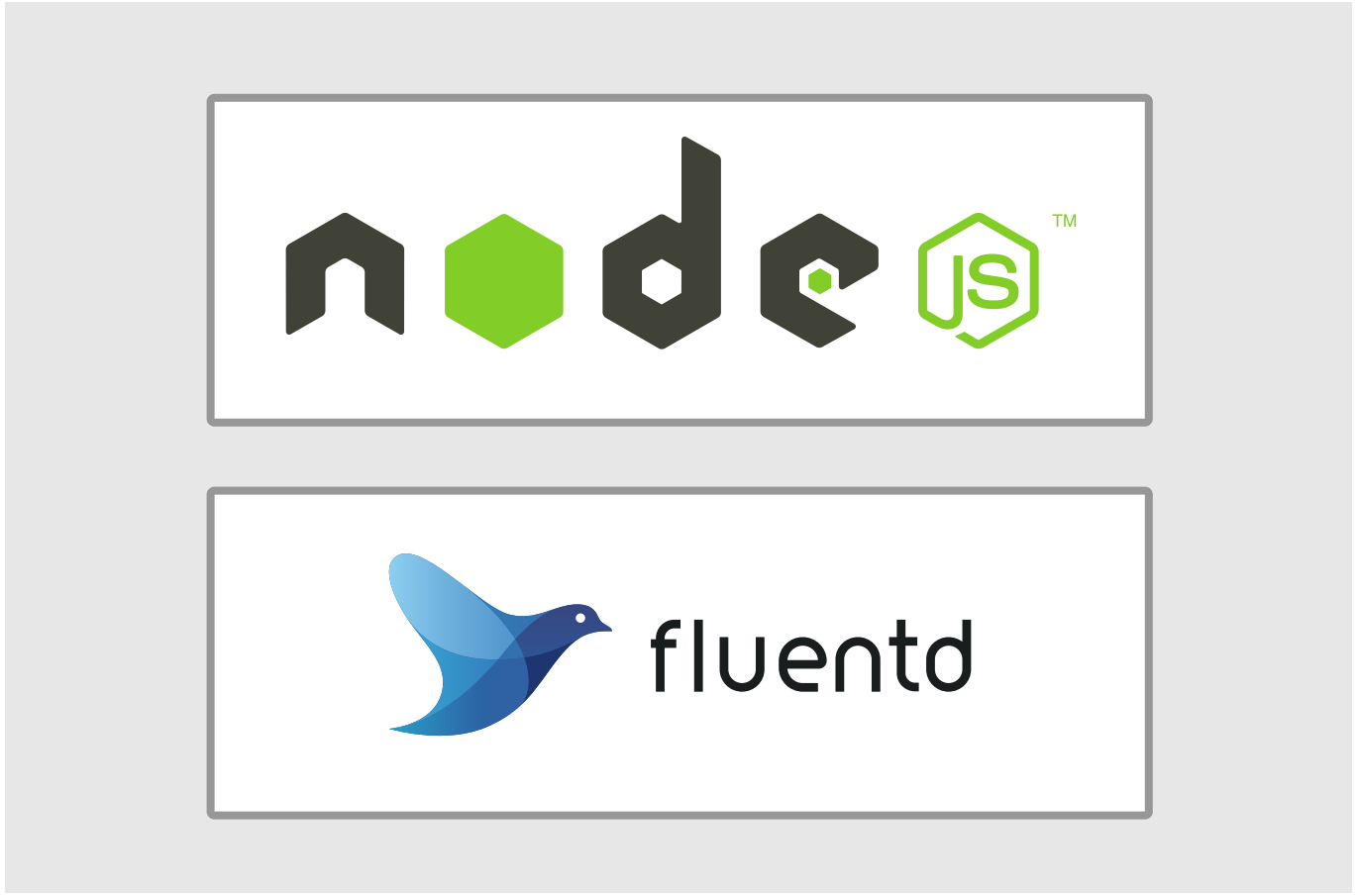



# Container probes

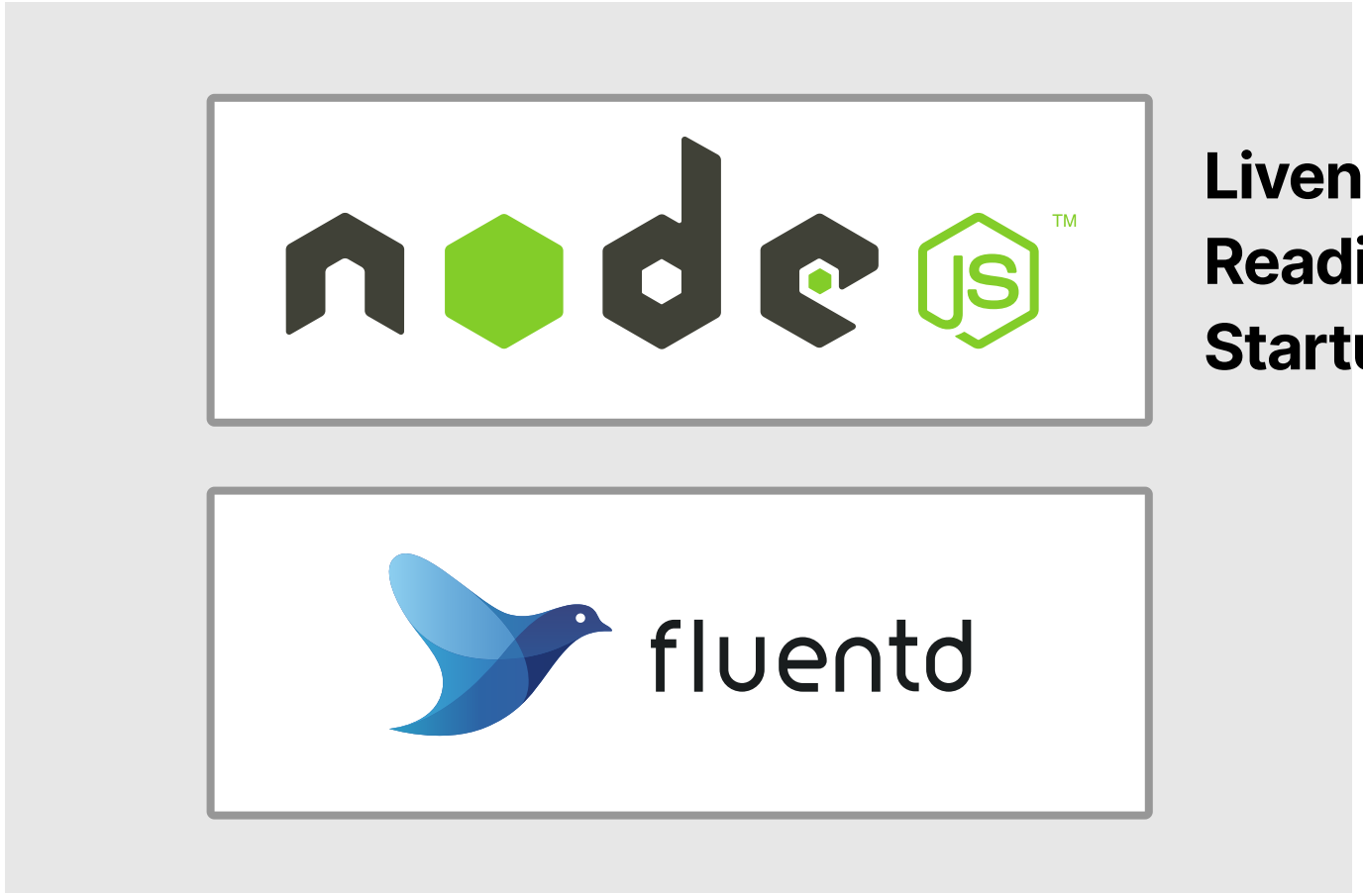
```
~$ cat pod.json
apiVersion: v1
kind: Pod
metadata:
  name: goproxy
spec:
  containers:
  - name: goproxy
    image: k8s.gcr.io/goproxy:0.1
    ports:
    - containerPort: 8080
    readinessProbe:
      tcpSocket:
        port: 8080
      initialDelaySeconds: 5
      periodSeconds: 10
    livenessProbe:
      tcpSocket:
        port: 8080
      initialDelaySeconds: 15
      periodSeconds: 20
```

```
~$ cat pod.json
apiVersion: v1
kind: Pod
metadata:
  name: goproxy
spec:
  containers:
  - name: goproxy
    image: k8s.gcr.io/goproxy:0.1
    ports:
    - containerPort: 8080
    readinessProbe:
      tcpSocket:
        port: 8080
      initialDelaySeconds: 5
      periodSeconds: 10
    livenessProbe:
      tcpSocket:
        port: 8080
      initialDelaySeconds: 15
      periodSeconds: 20
```

# Pod

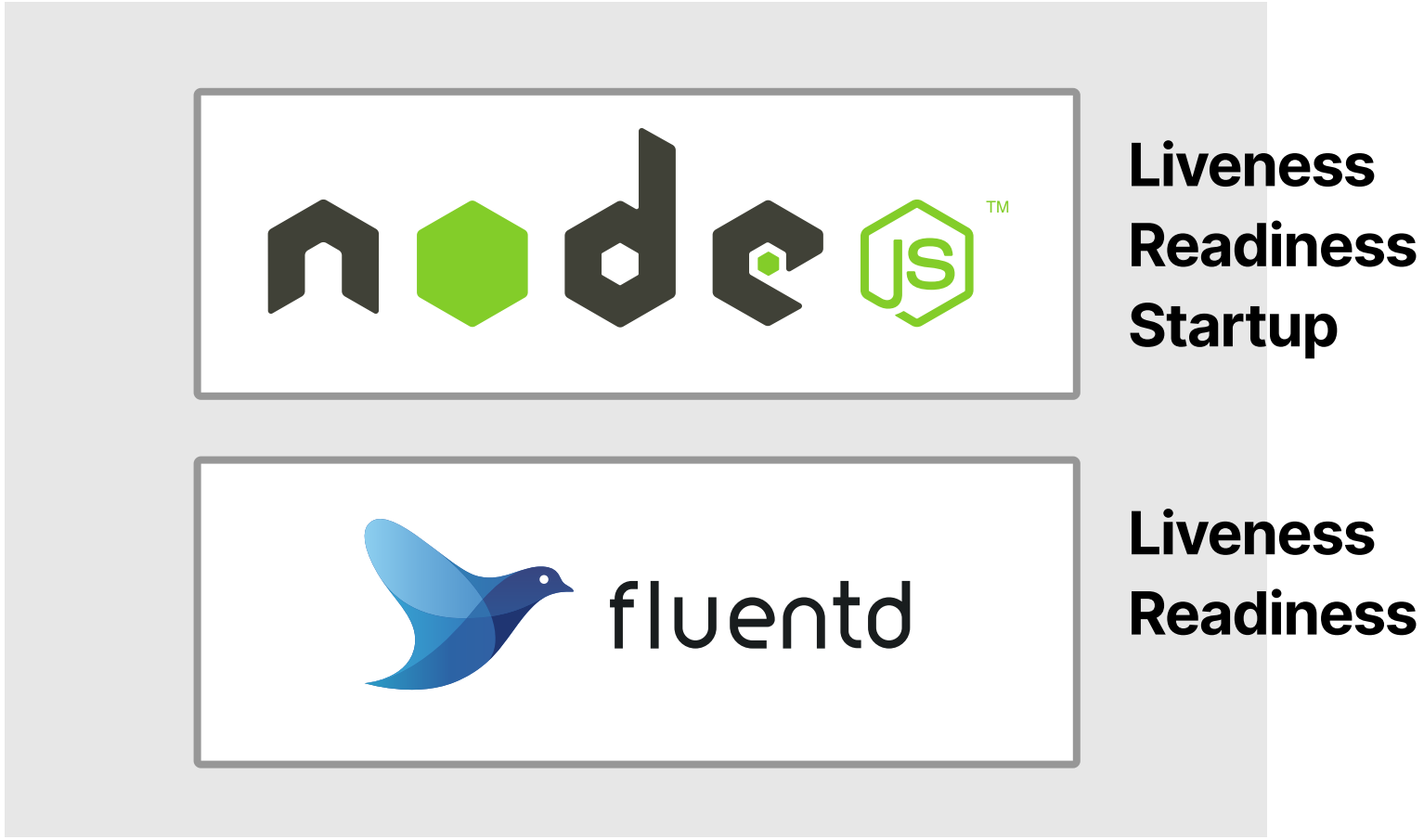


## Pod




**Liveness**
**Readiness**
**Startup**

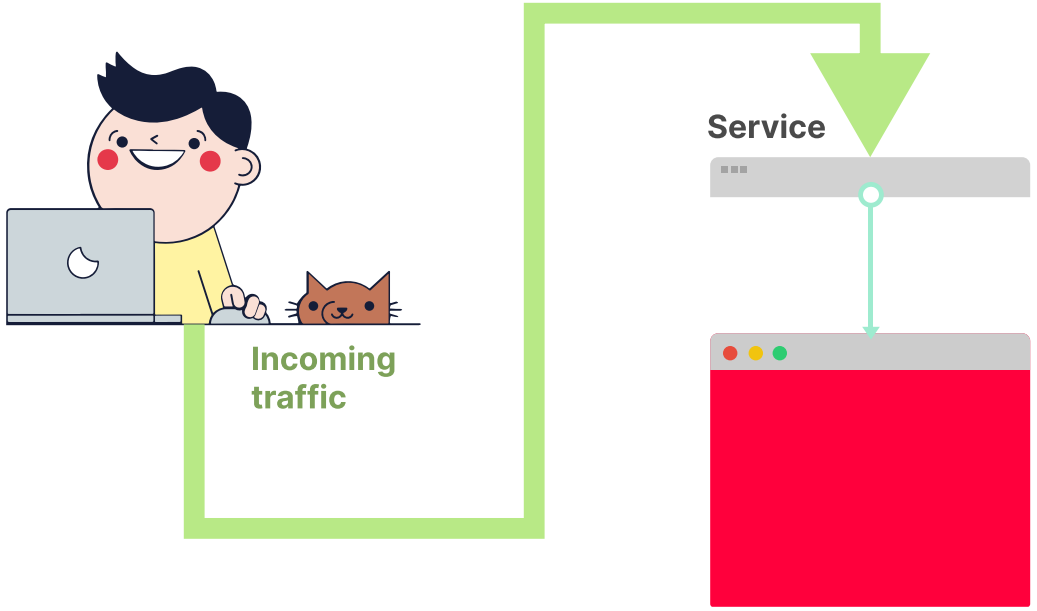
Pod




Liveness
Readiness
Startup

Liveness
Readiness

# Probes in practice

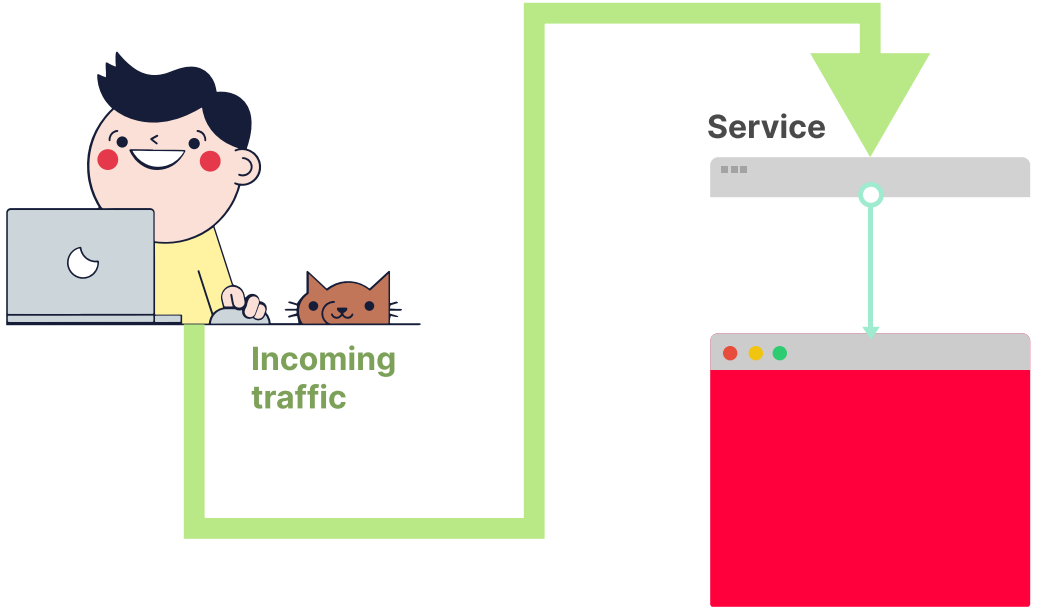



/health
ENDPOINT

Liveness
PROBE

/ready
ENDPOINT

Readiness
PROBE

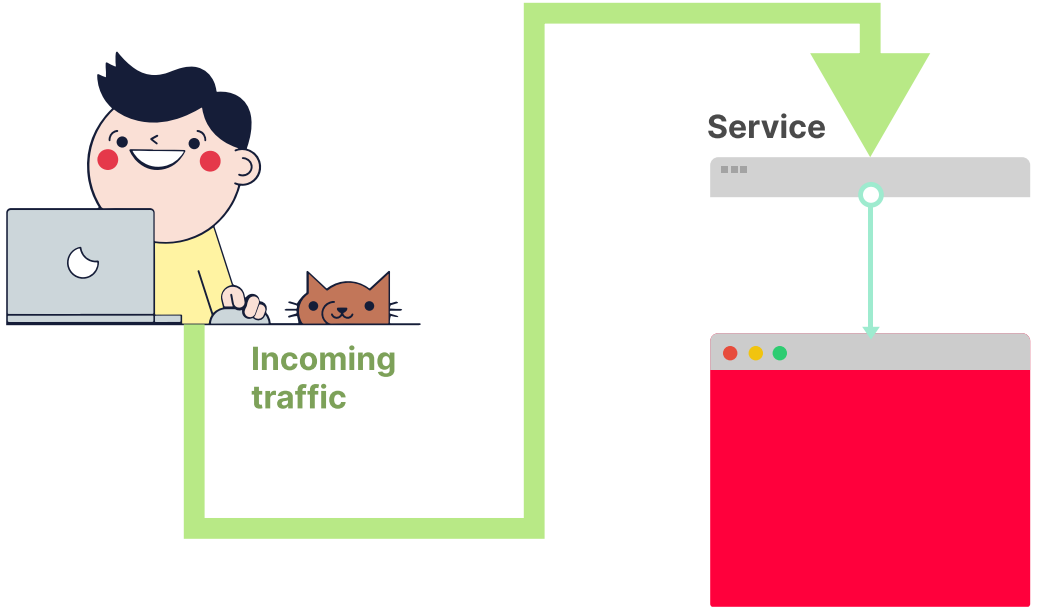



/health
ENDPOINT

Liveness
PROBE

/ready
ENDPOINT

Readiness
PROBE

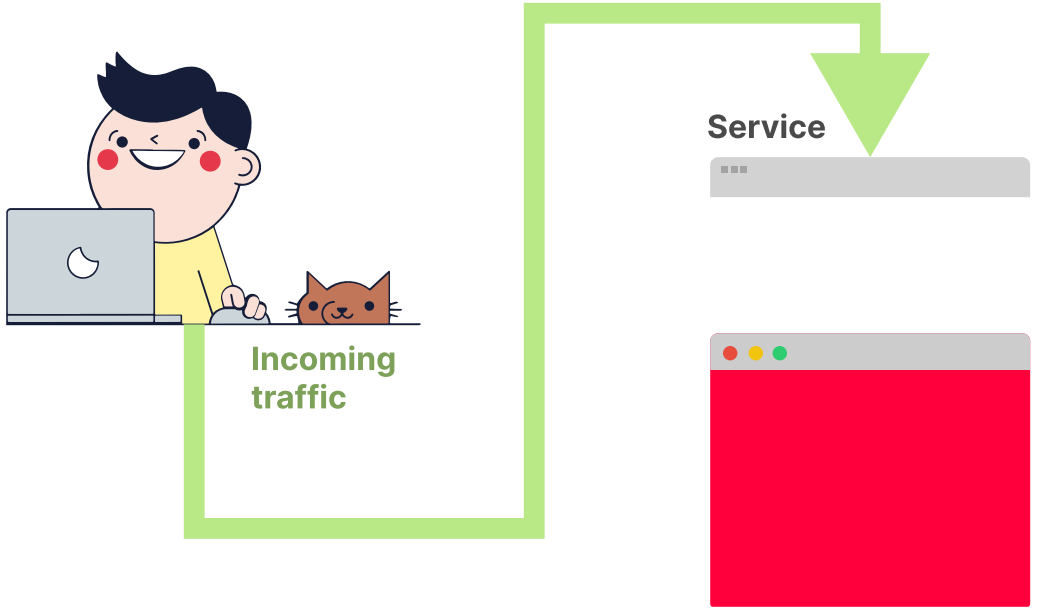



/health
ENDPOINT

Liveness
PROBE

/ready
ENDPOINT

Readiness
PROBE

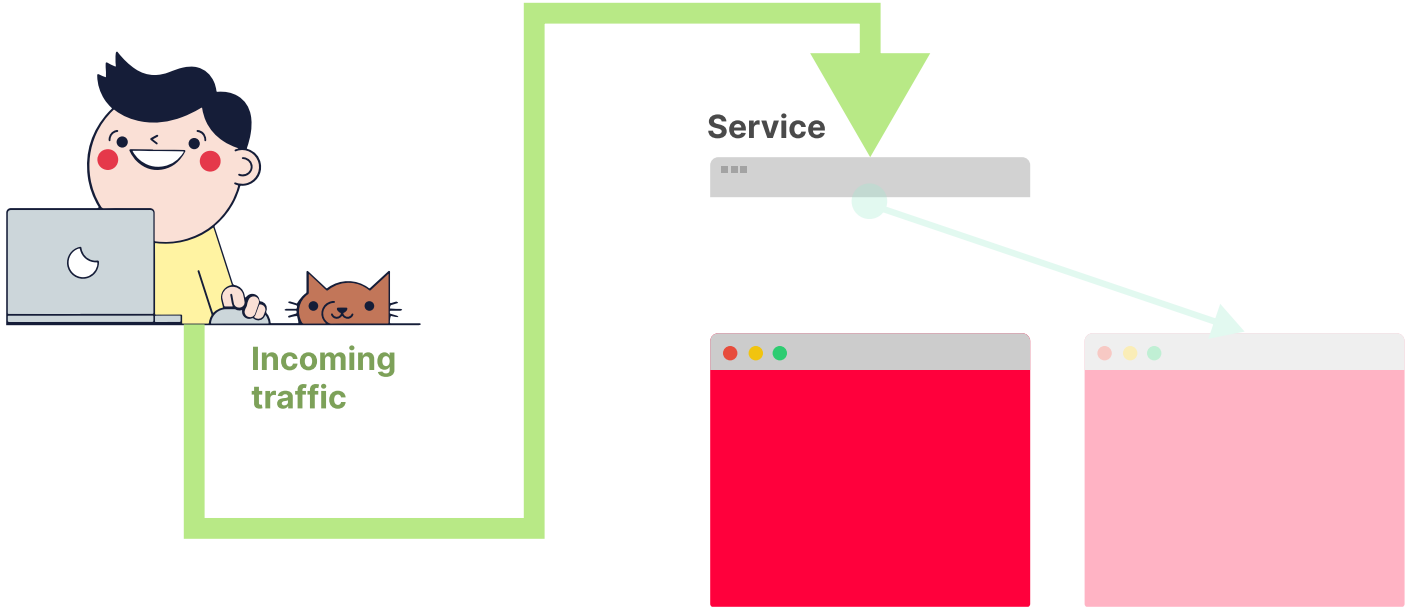



/health
ENDPOINT

Liveness
PROBE

/ready
ENDPOINT

Readiness
PROBE

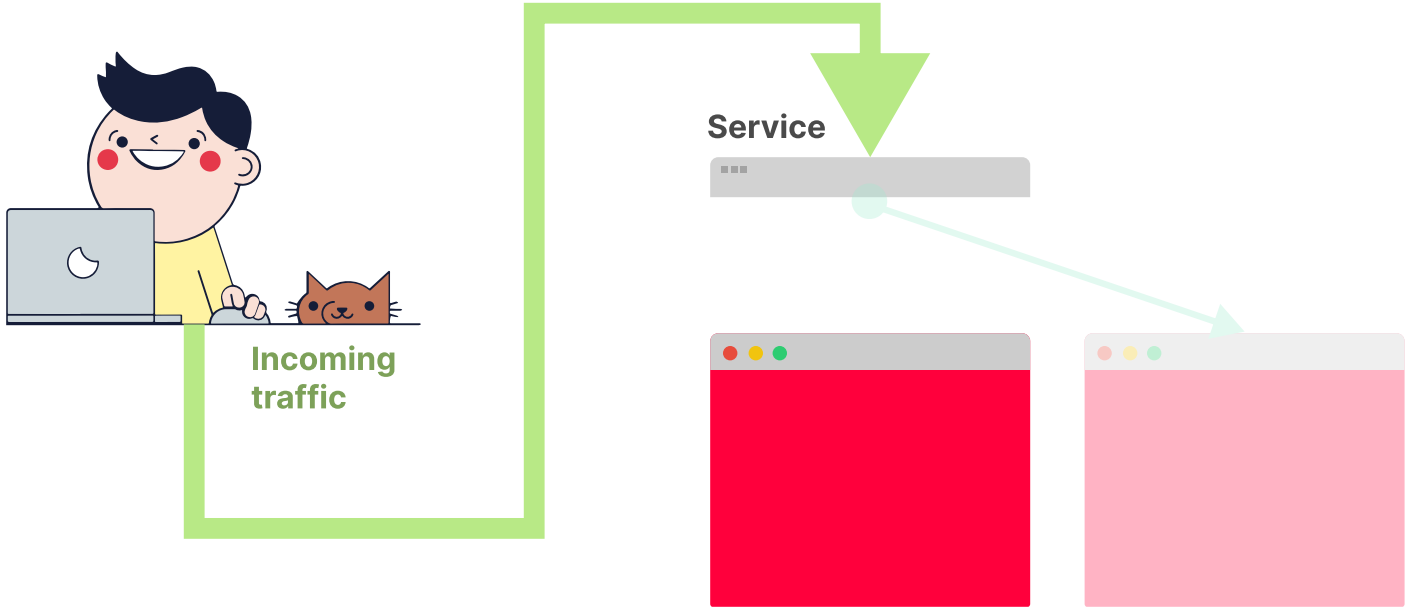



/health
ENDPOINT

Liveness
PROBE

/ready
ENDPOINT

Readiness
PROBE

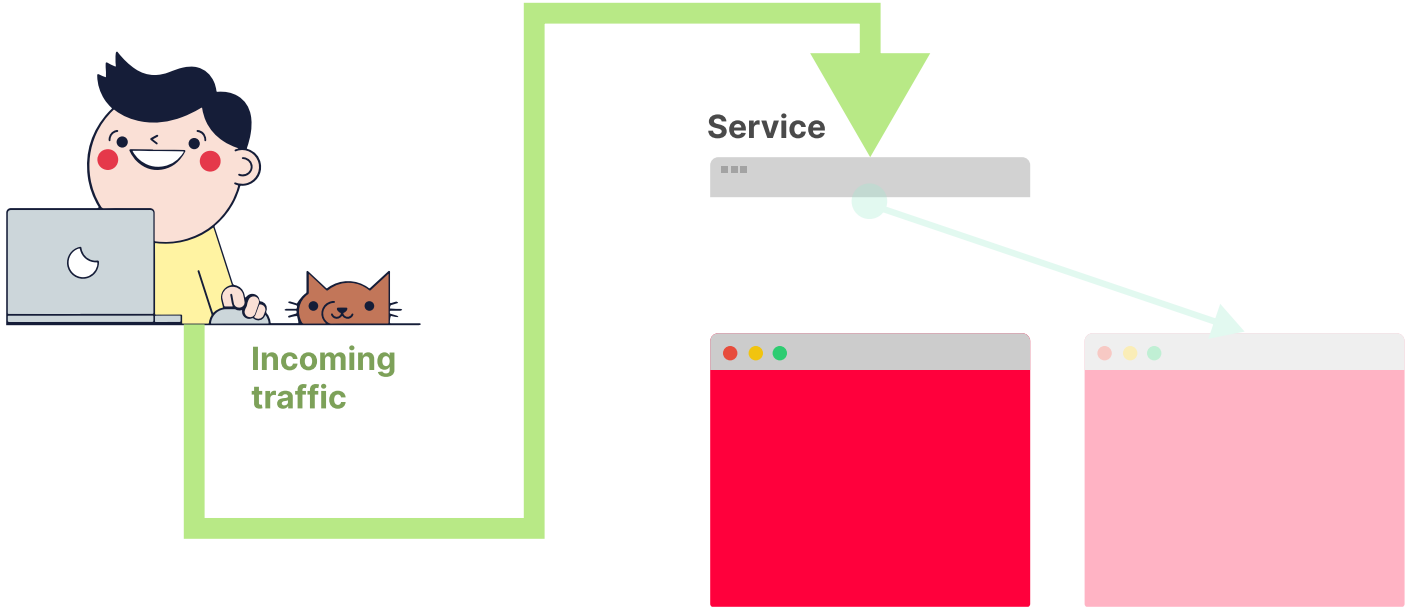



/health
ENDPOINT

Liveness
PROBE

/ready
ENDPOINT

Readiness
PROBE

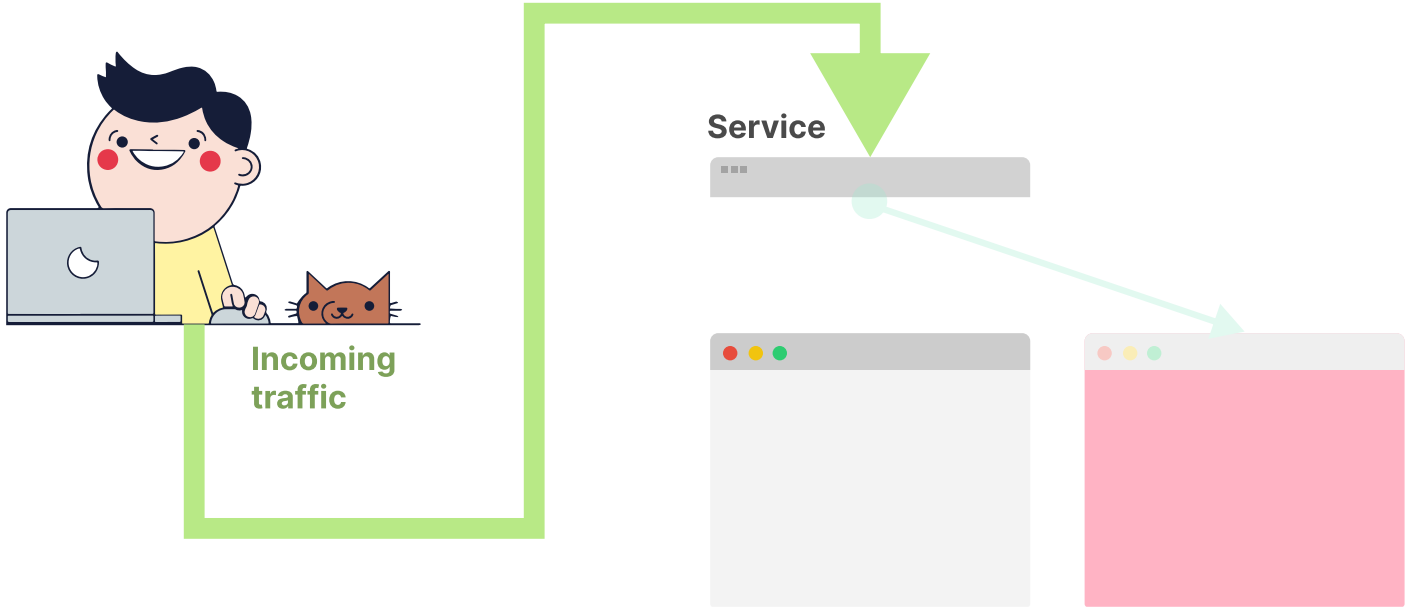



/health
ENDPOINT

Liveness
PROBE

/ready
ENDPOINT

Readiness
PROBE

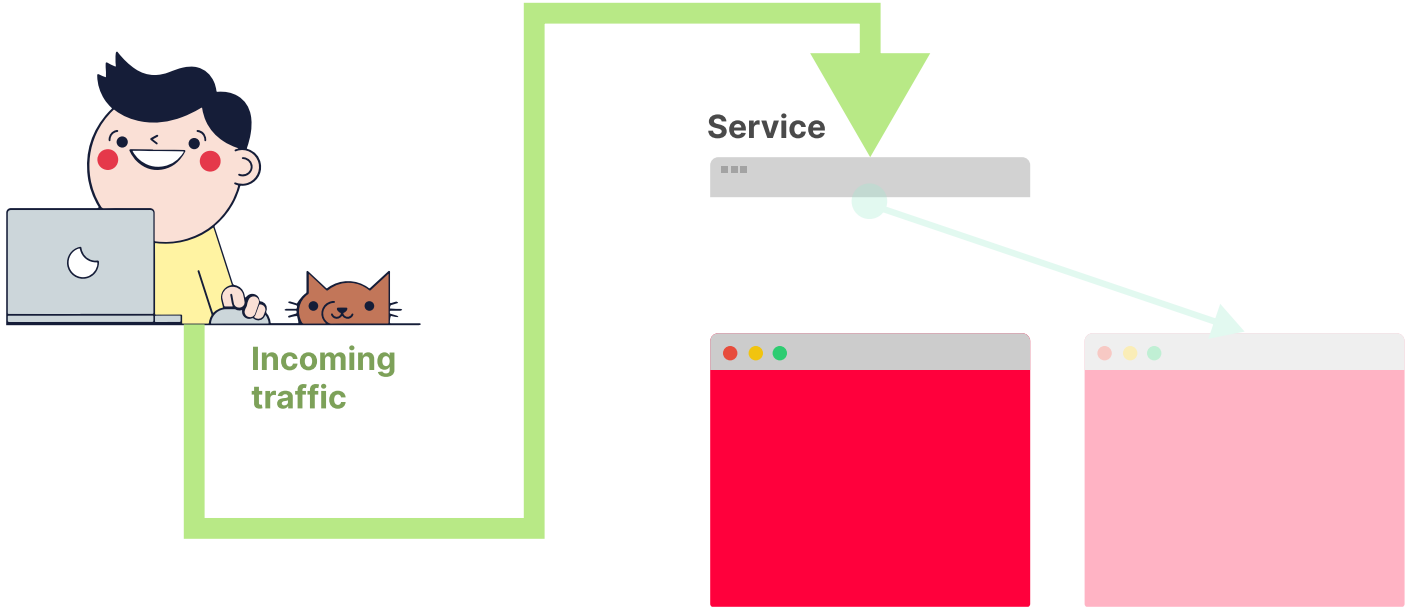



/health
ENDPOINT

Liveness
PROBE

/ready
ENDPOINT

Readiness
PROBE

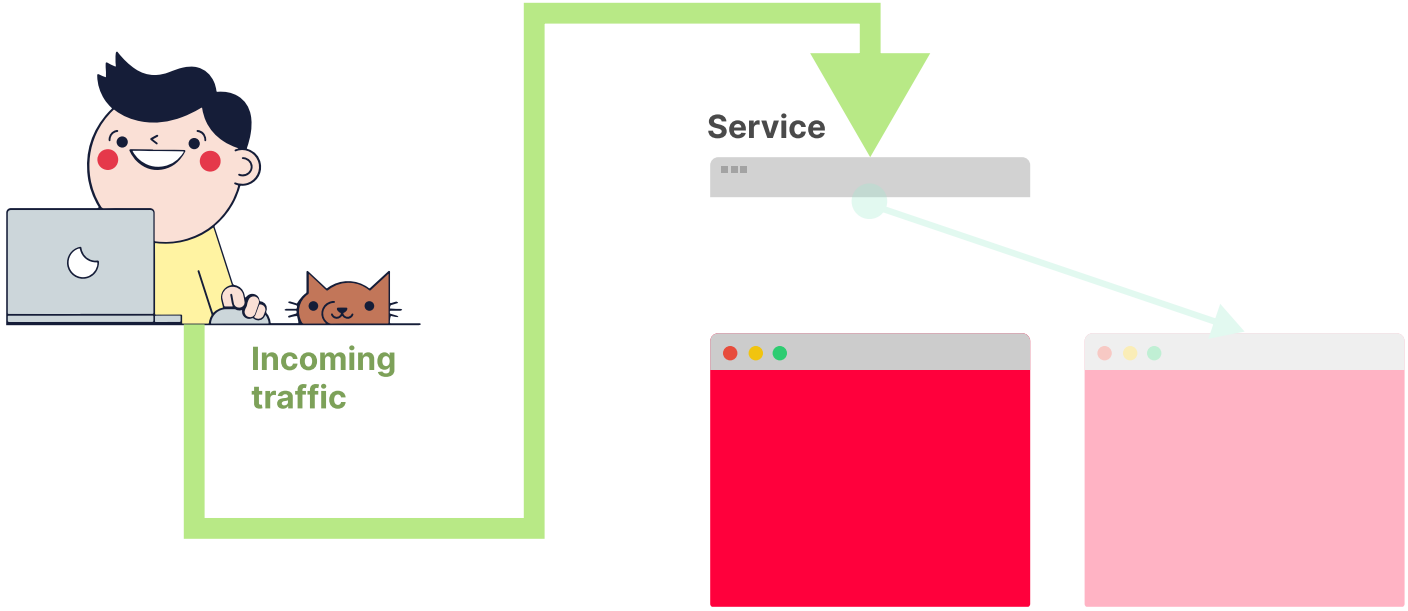



/health
ENDPOINT

Liveness
PROBE

/ready
ENDPOINT

Readiness
PROBE

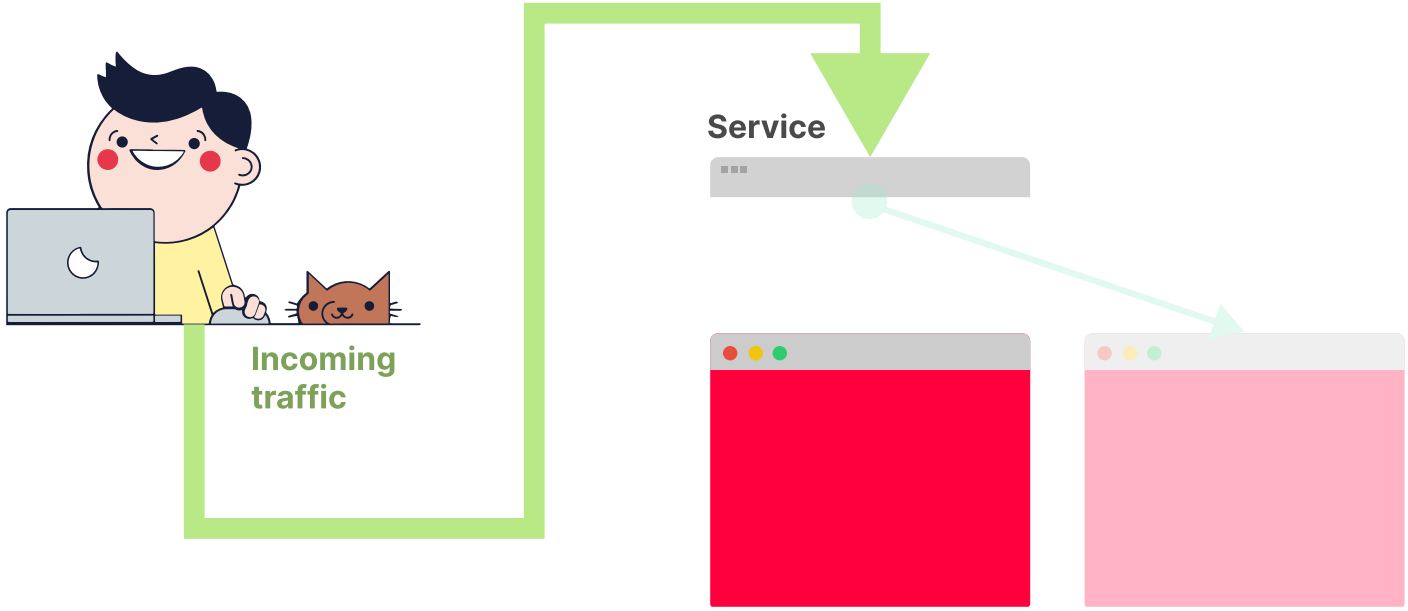



/health
ENDPOINT

Liveness
PROBE

/ready
ENDPOINT

Readiness
PROBE

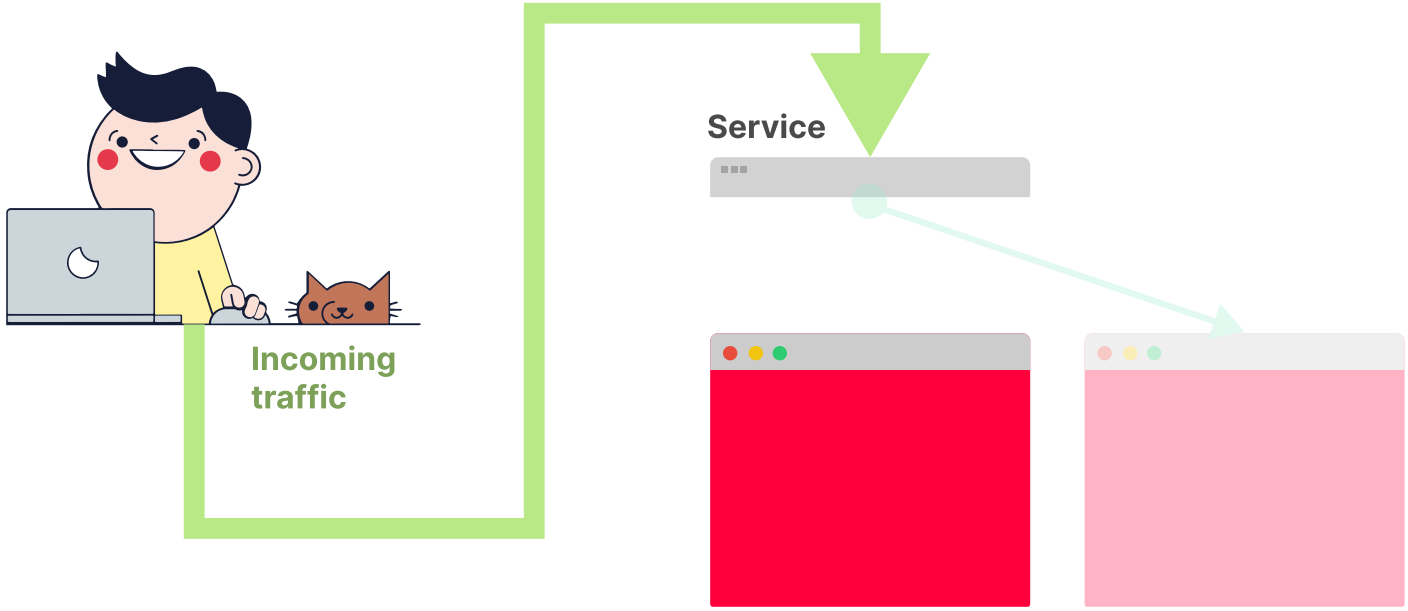



/health
ENDPOINT

Liveness
PROBE

/ready
ENDPOINT

Readiness
PROBE

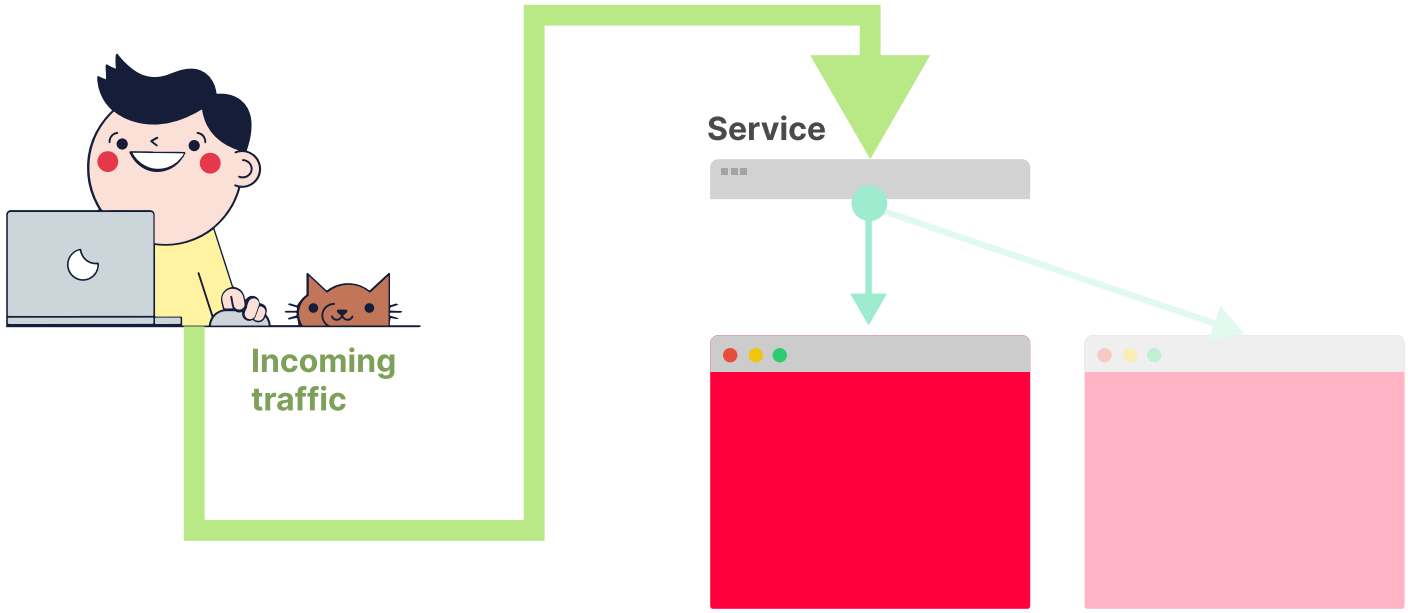



/health
ENDPOINT

Liveness
PROBE

/ready
ENDPOINT

Readiness
PROBE




/health
ENDPOINT

Liveness
PROBE

/ready
ENDPOINT

Readiness
PROBE

# Recap

Probes

# Recap — Probes

1. Push vs Pull
2. Liveness probe
3. Startup probe
4. Readiness probe
5. Customizable on a container level

# Recap — Probes

1. Push vs Pull

2. Liveness probe

3. Startup probe

4. Readiness probe

5. Customizable on a container level

# Recap — Probes

1. Push vs Pull

2. Liveness probe

3. Startup probe

4. Readiness probe

5. Customizable on a container level

# Recap — Probes

1. Push vs Pull

2. Liveness probe

3. Startup probe

**4. Readiness probe**

5. Customizable on a container level

# Recap — Probes

1. Push vs Pull
2. Liveness probe
3. Startup probe
4. Readiness probe
5. **Customizable on a container level**

# Deployment strategies in Kubernetes

## Deployment strategies

1. Rolling update
2. Canary
3. Blue-green

## Deployment strategies

1. Rolling update
2. **Canary**
3. Blue-green

## Deployment strategies

1. Rolling update
2. Canary
3. **Blue-green**

# Rolling updates

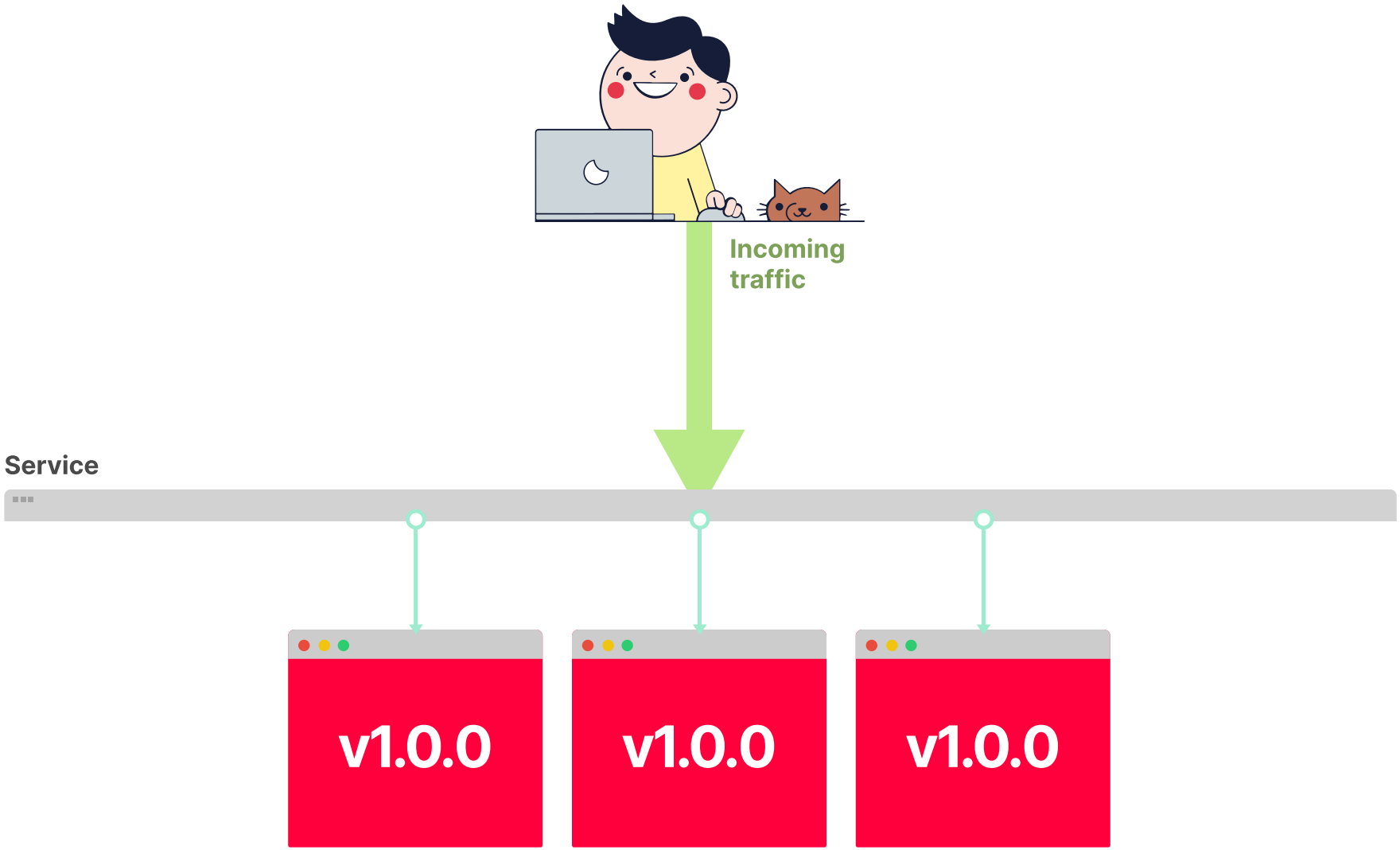
01

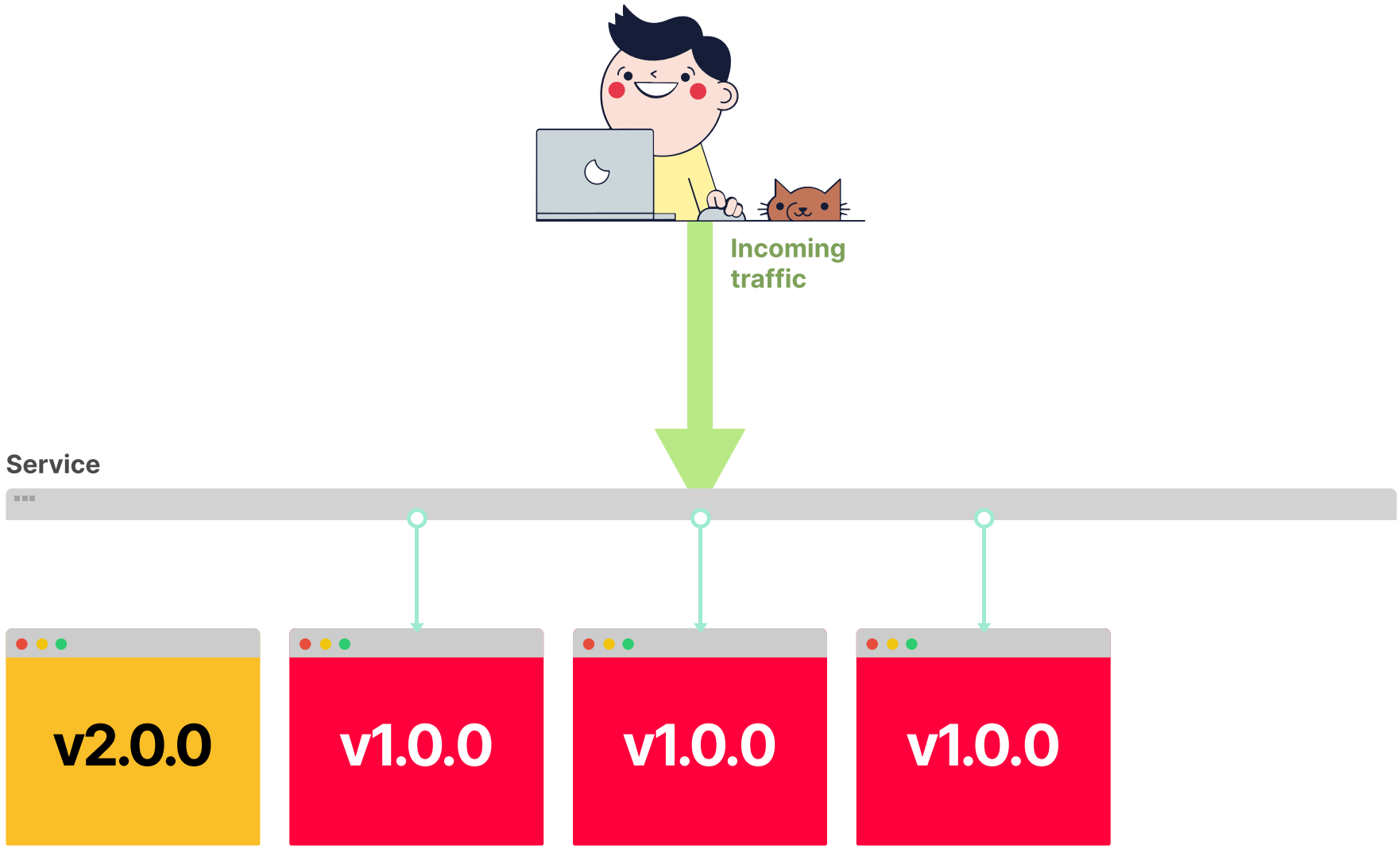


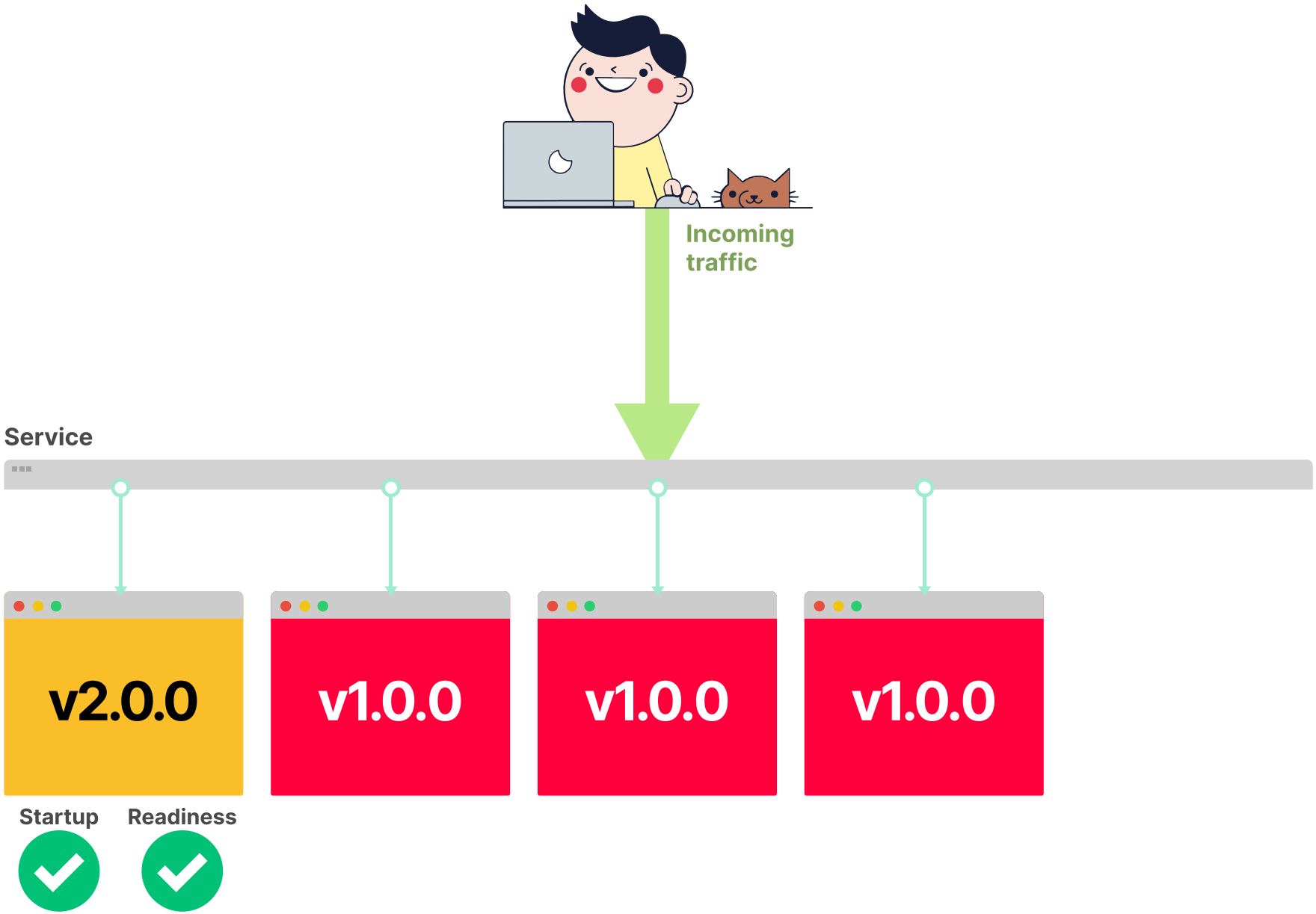

v1.0.0

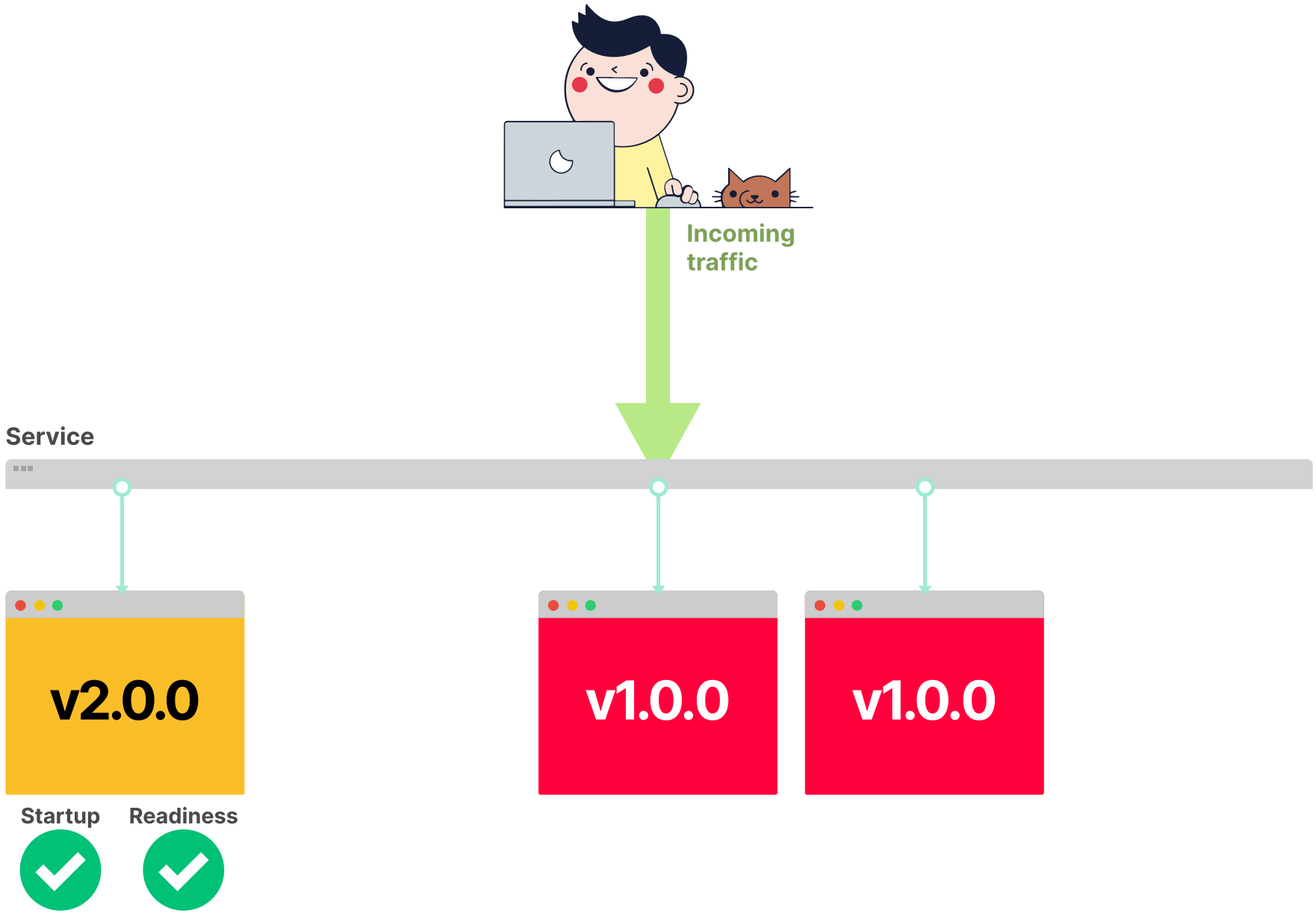


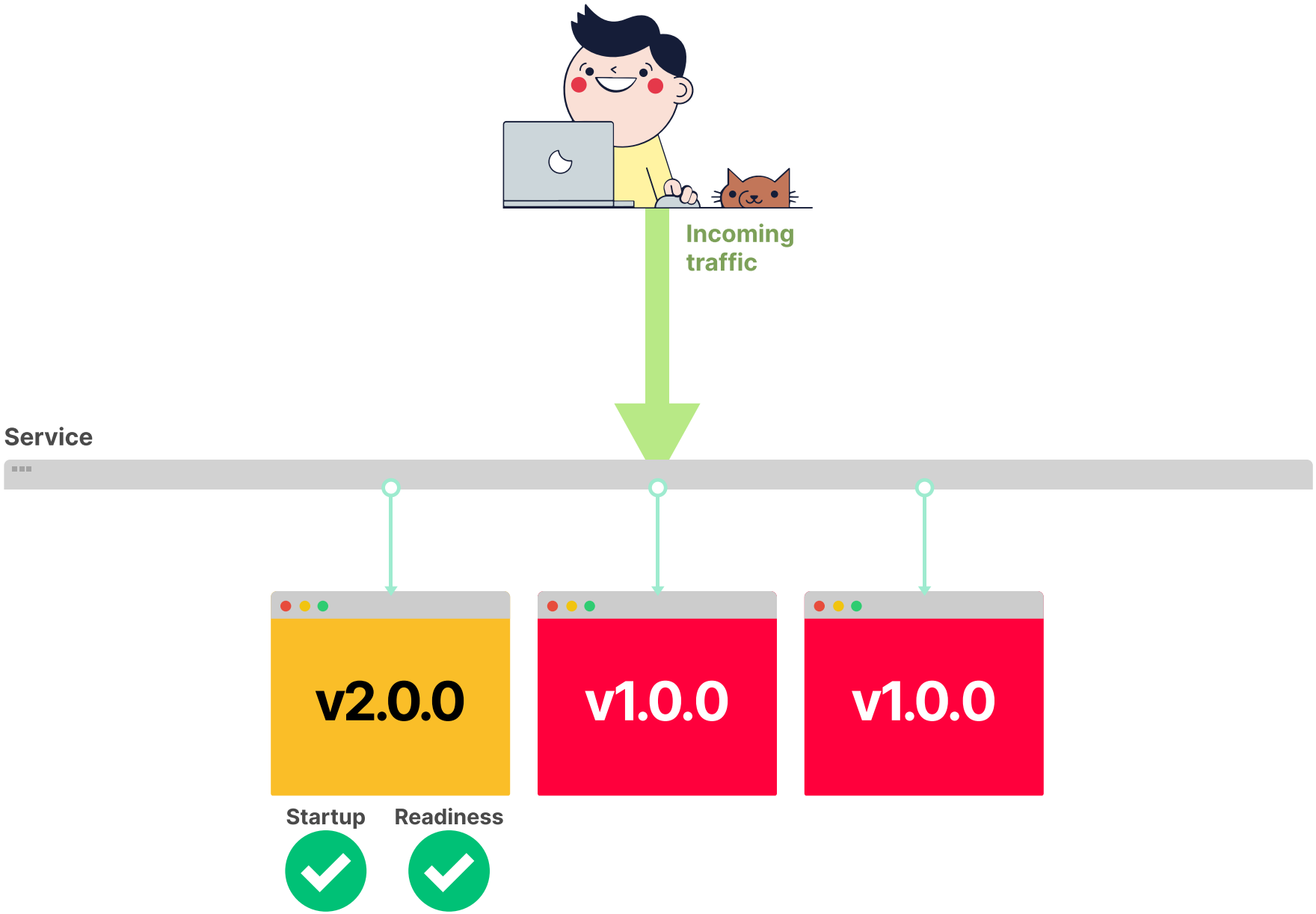

v2.0.0

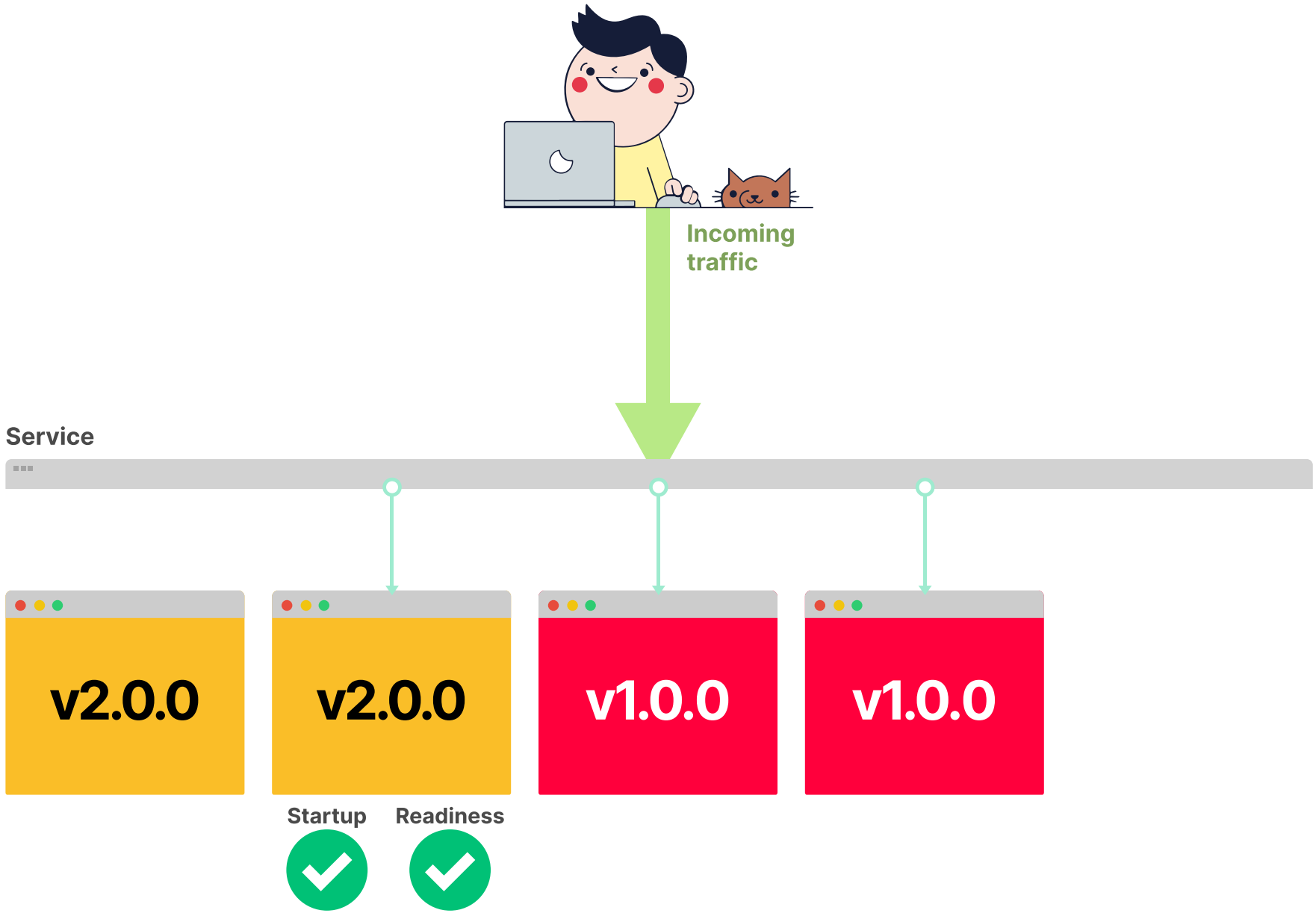

Incoming
traffic
Service
v1.0.0
v1.0.0
v1.0.0

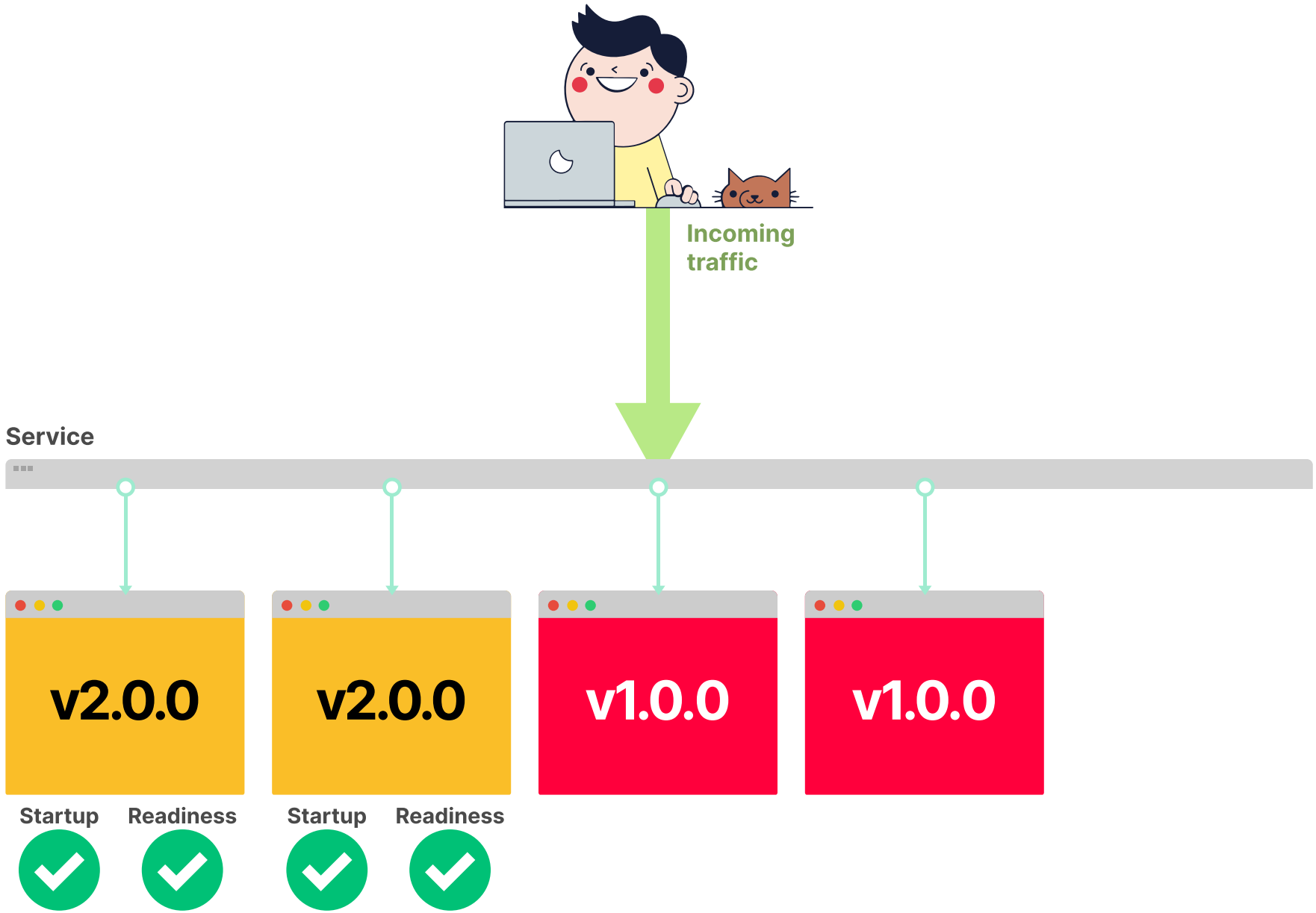

Incoming
traffic
Service
v2.0.0
v1.0.0
v1.0.0
v1.0.0

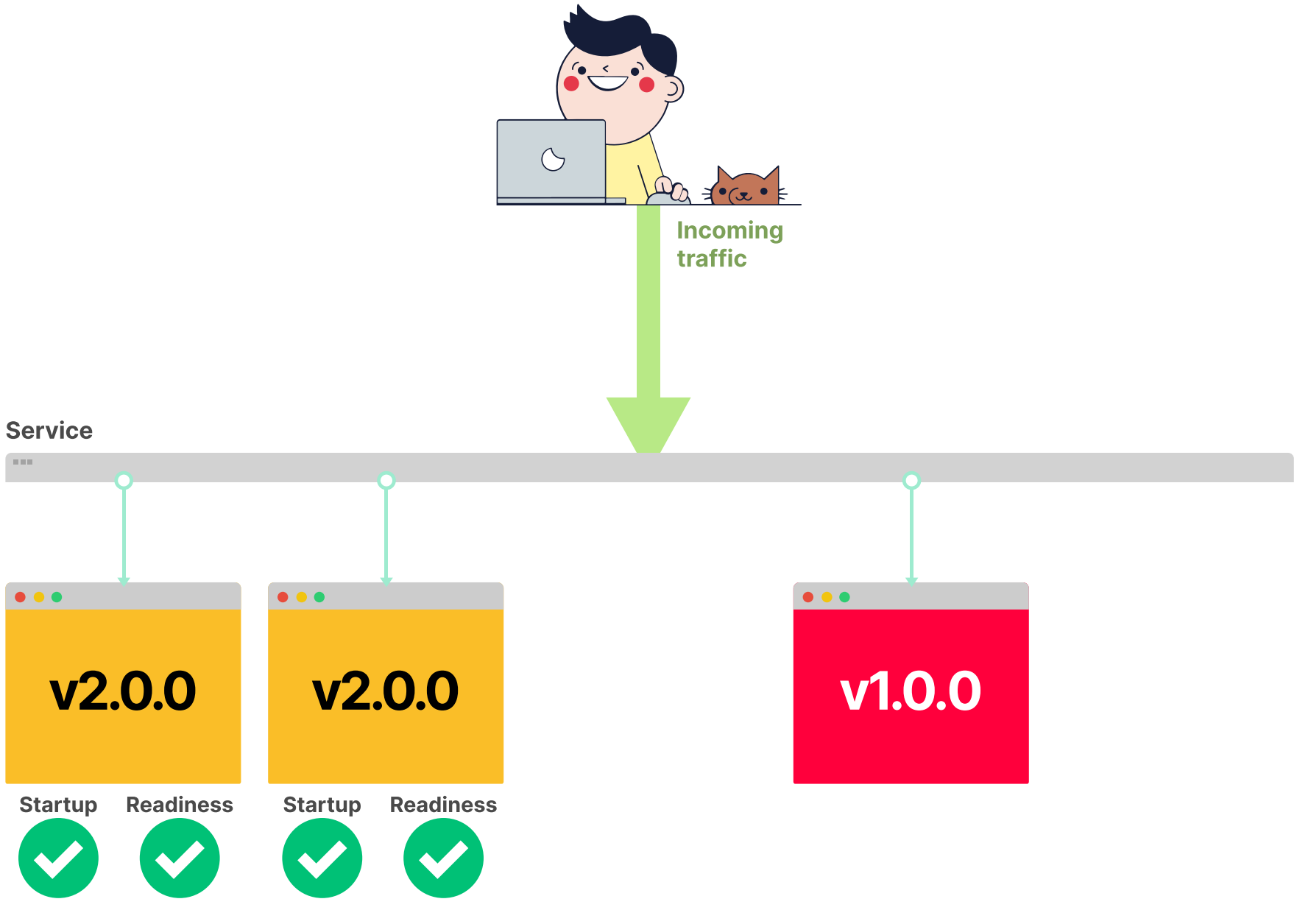

Incoming
traffic
Service
v2.0.0
v1.0.0
v1.0.0
v1.0.0
Startup
Readiness

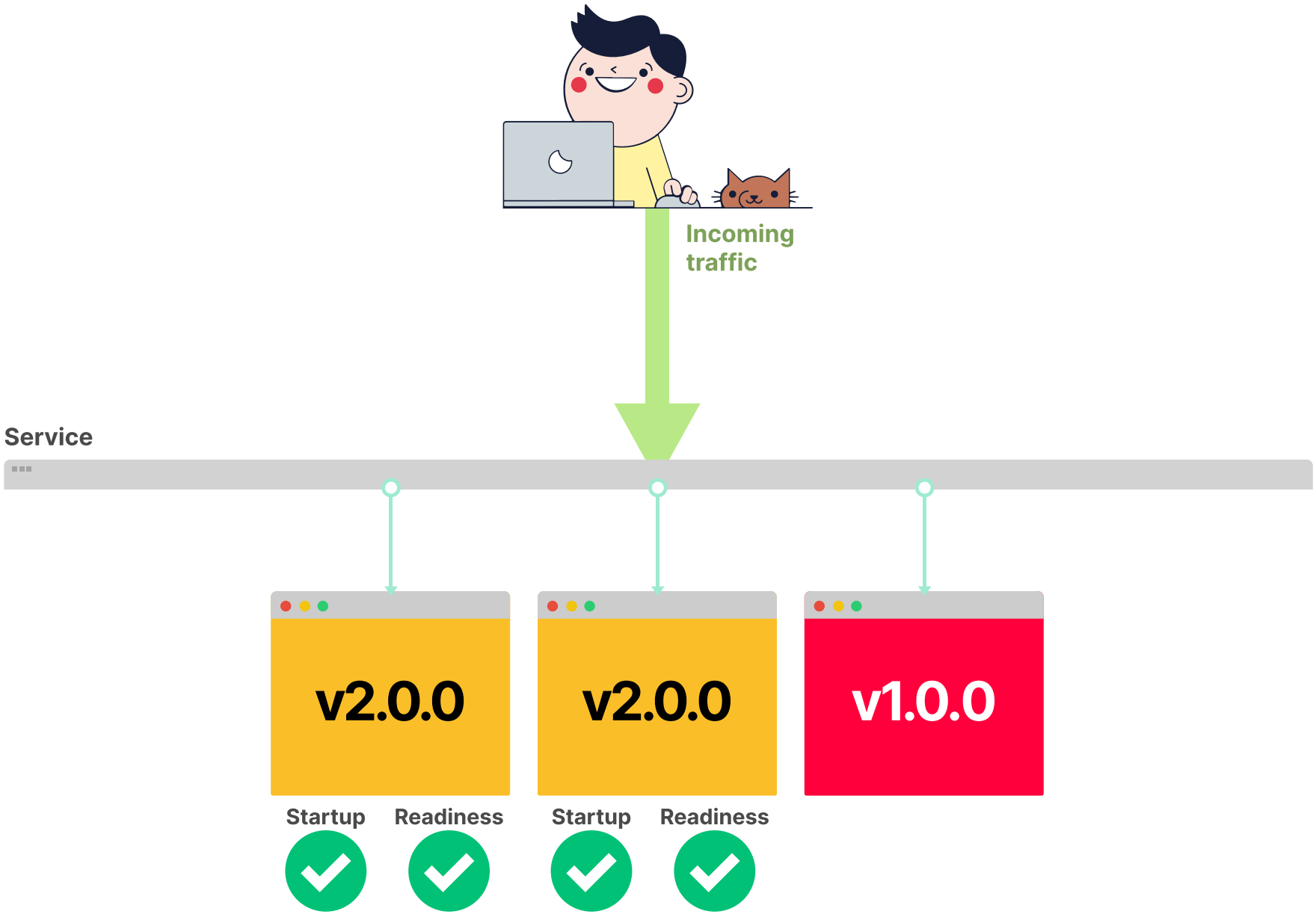

Incoming
traffic
Service
v2.0.0
v1.0.0
v1.0.0
Startup
Readiness

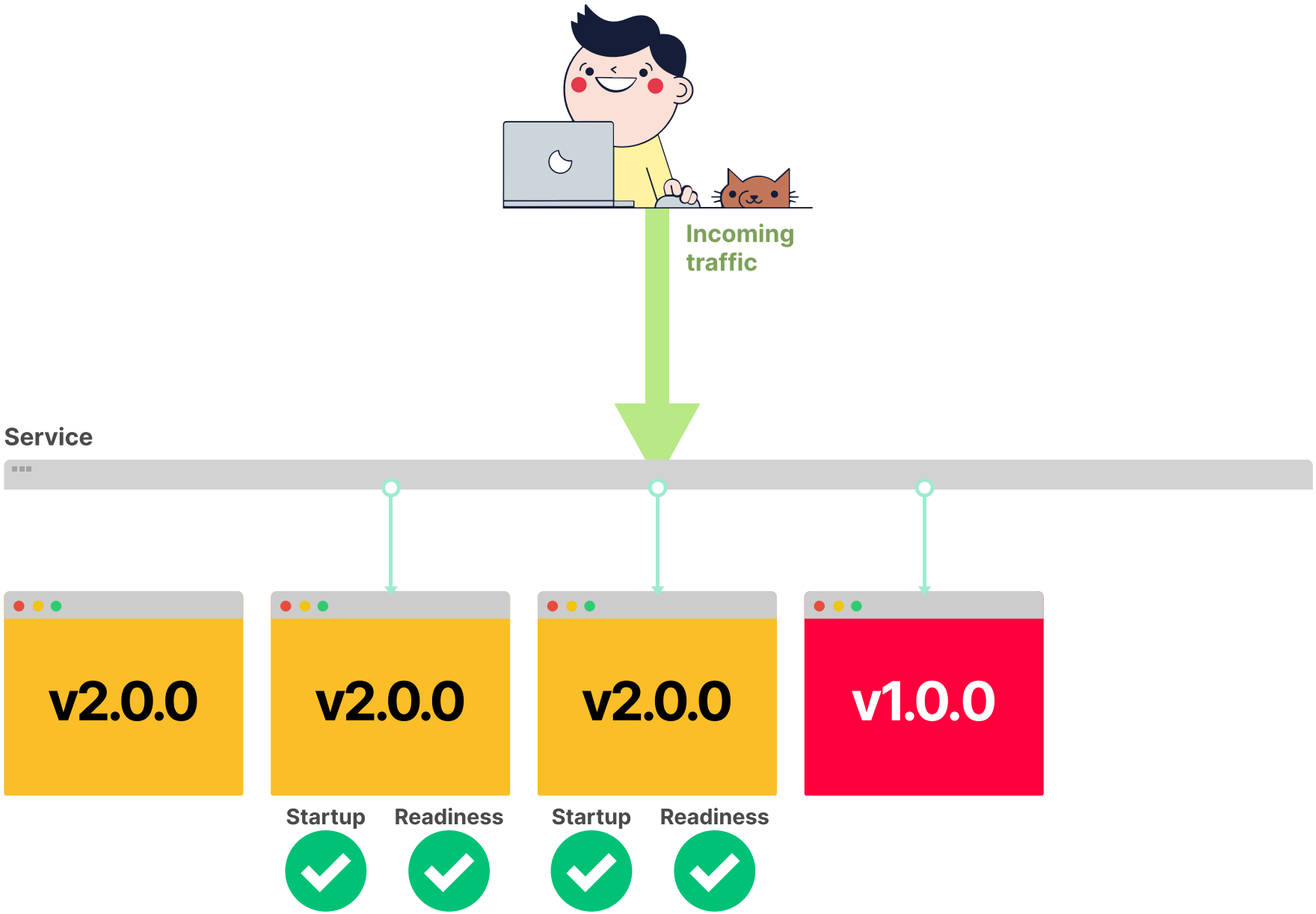

Incoming
traffic
Service
v2.0.0
v1.0.0
v1.0.0
Startup
Readiness

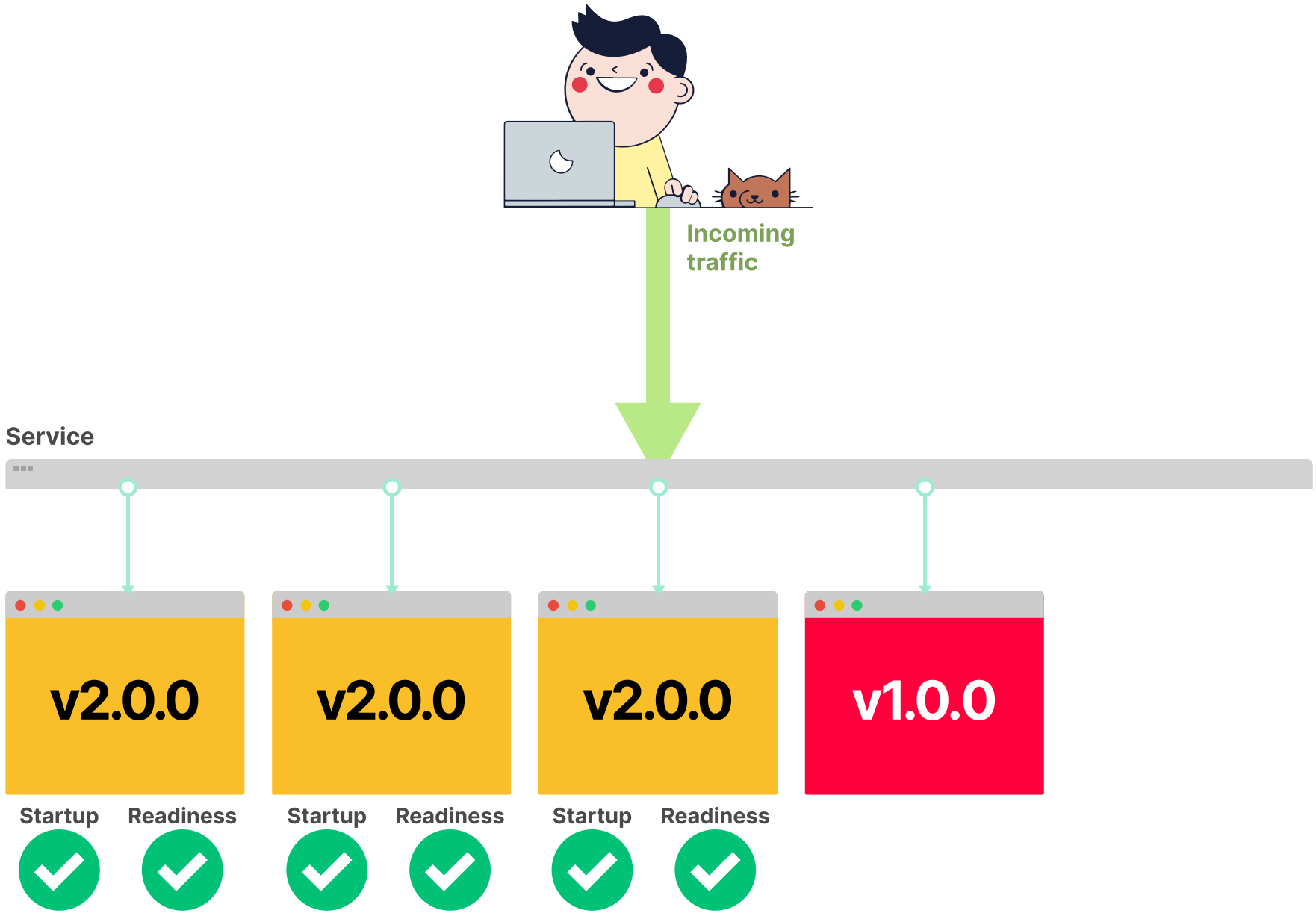

Incoming
traffic
Service
v2.0.0
v2.0.0
v1.0.0
v1.0.0
Startup
Readiness

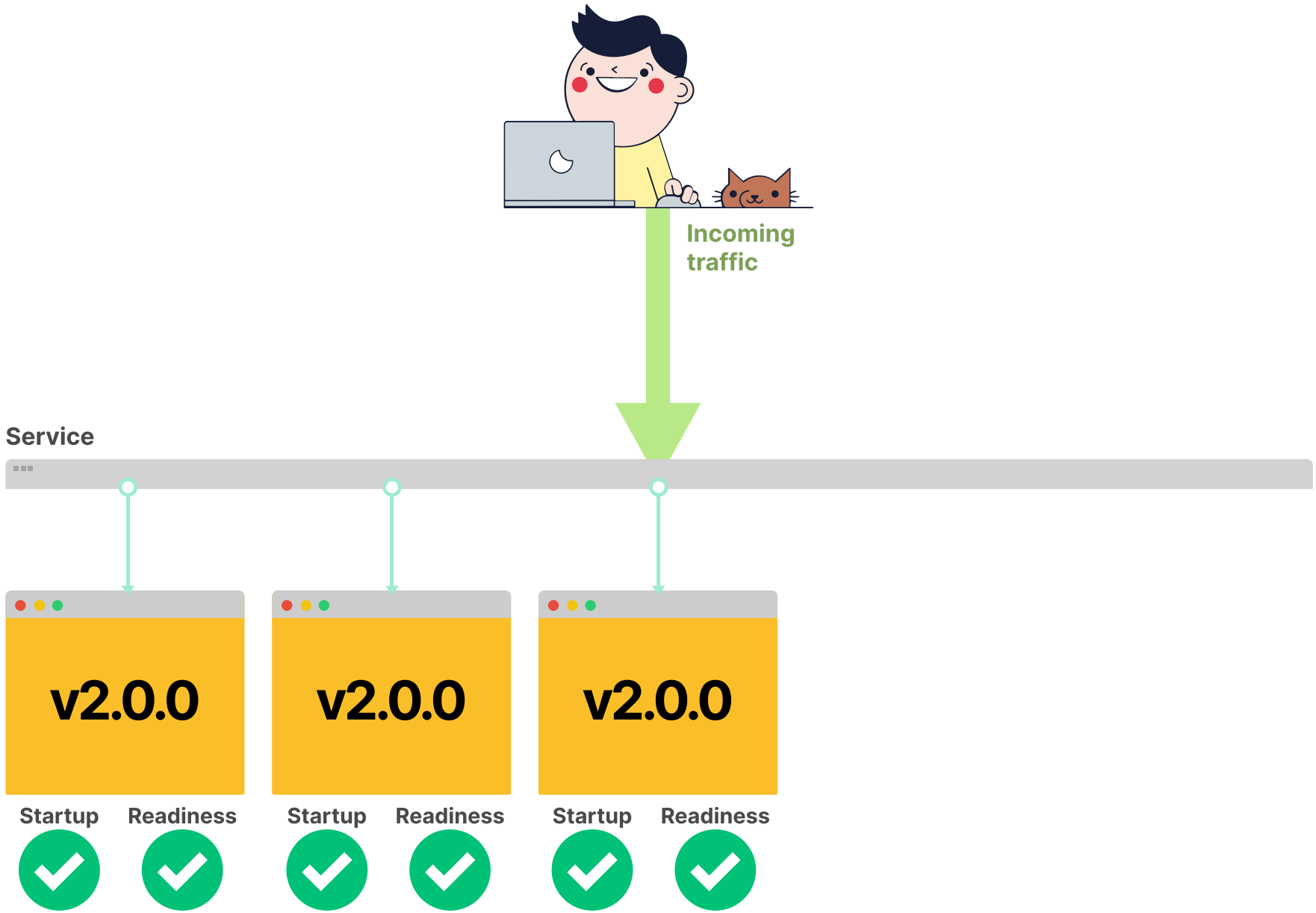

Incoming
traffic
Service
v2.0.0
v2.0.0
v1.0.0
v1.0.0
Startup
Readiness
Startup
Readiness

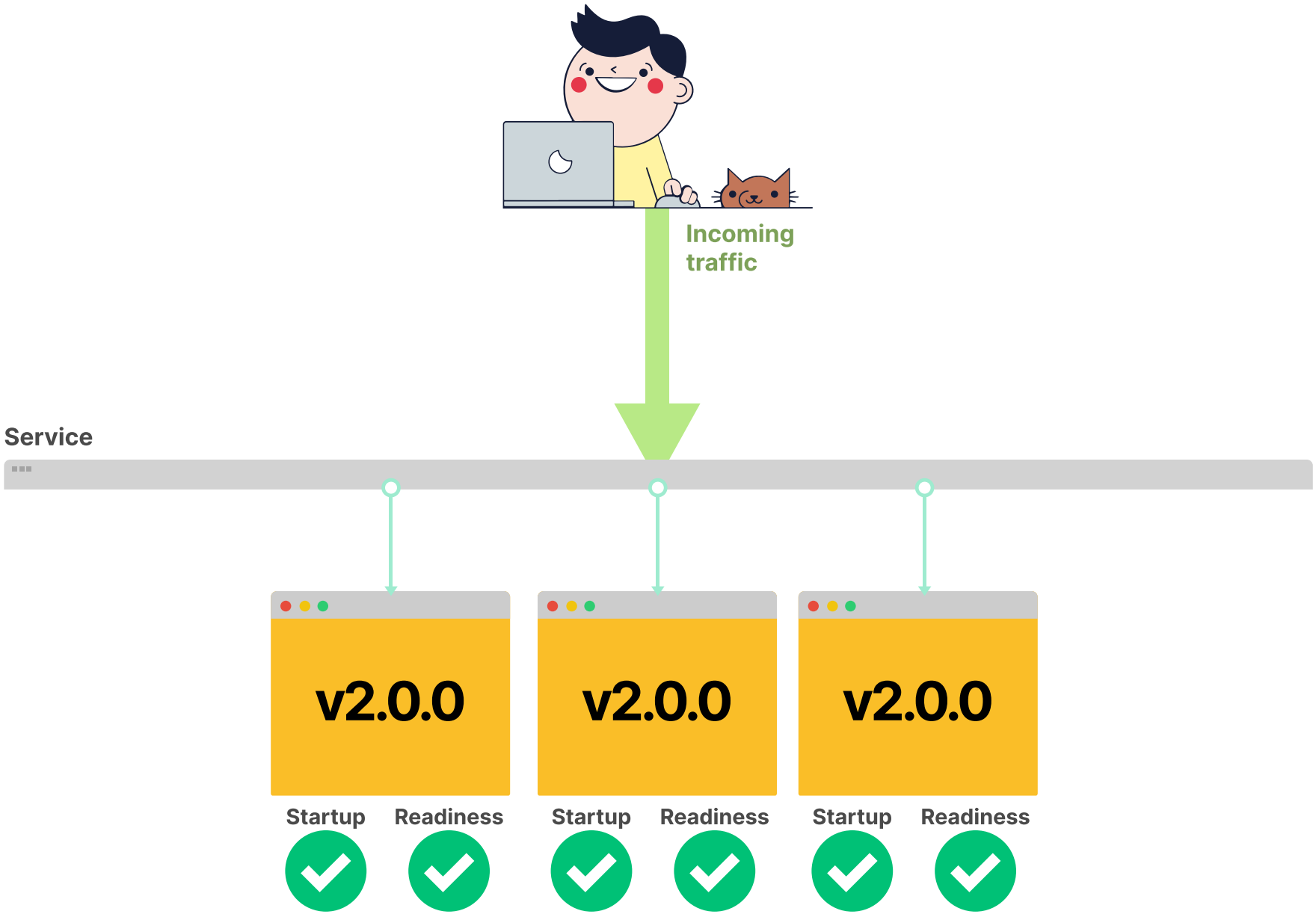

Incoming
traffic
Service
v2.0.0
v2.0.0
v1.0.0
Startup
Readiness
Startup
Readiness


Incoming traffic
Service
v2.0.0
v2.0.0
v1.0.0
Startup
Readiness
Startup
Readiness


Incoming traffic
Service
v2.0.0
v2.0.0
v2.0.0
v1.0.0
Startup
Readiness
Startup
Readiness

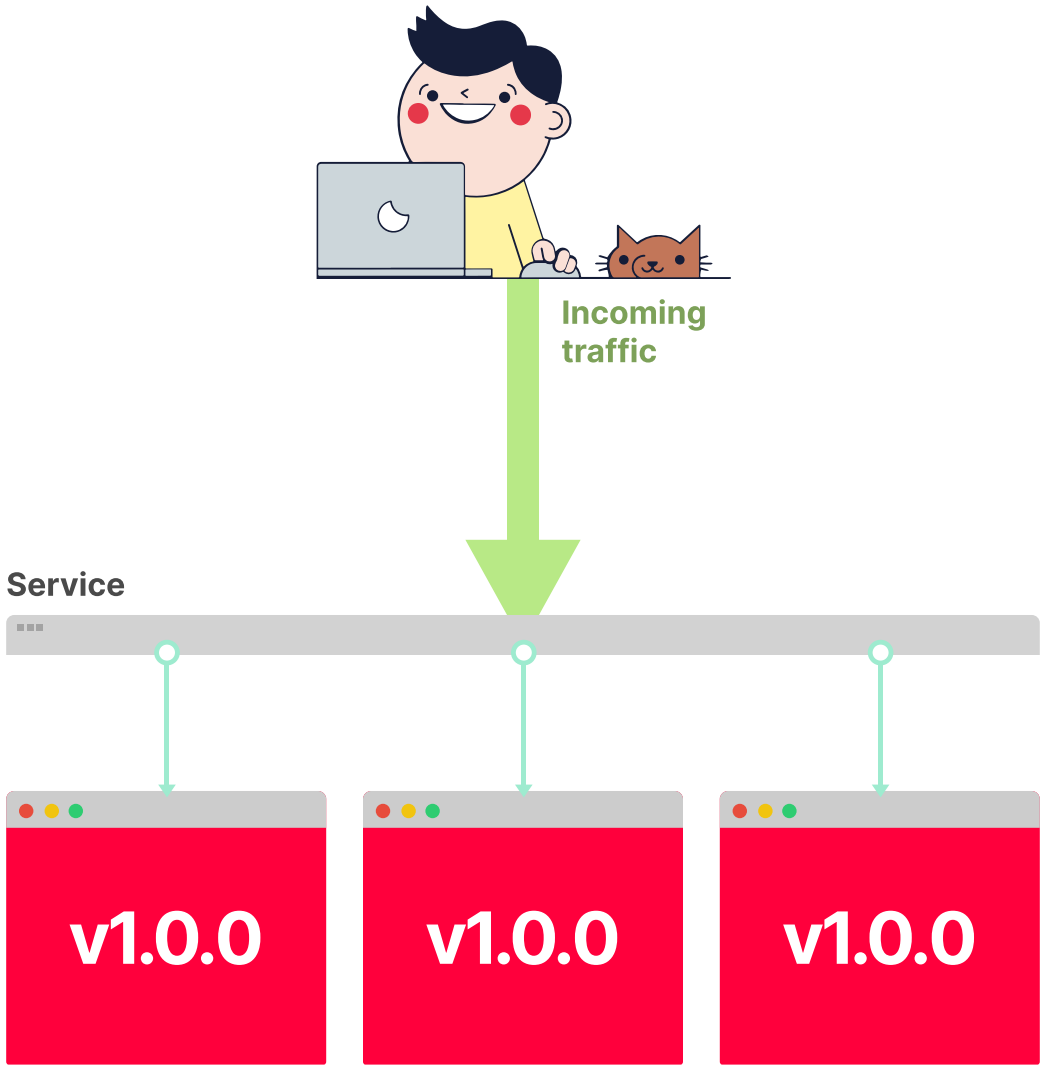

Incoming traffic
Service
v2.0.0
v2.0.0
v2.0.0
v1.0.0
Startup
Readiness
Startup
Readiness
Startup
Readiness

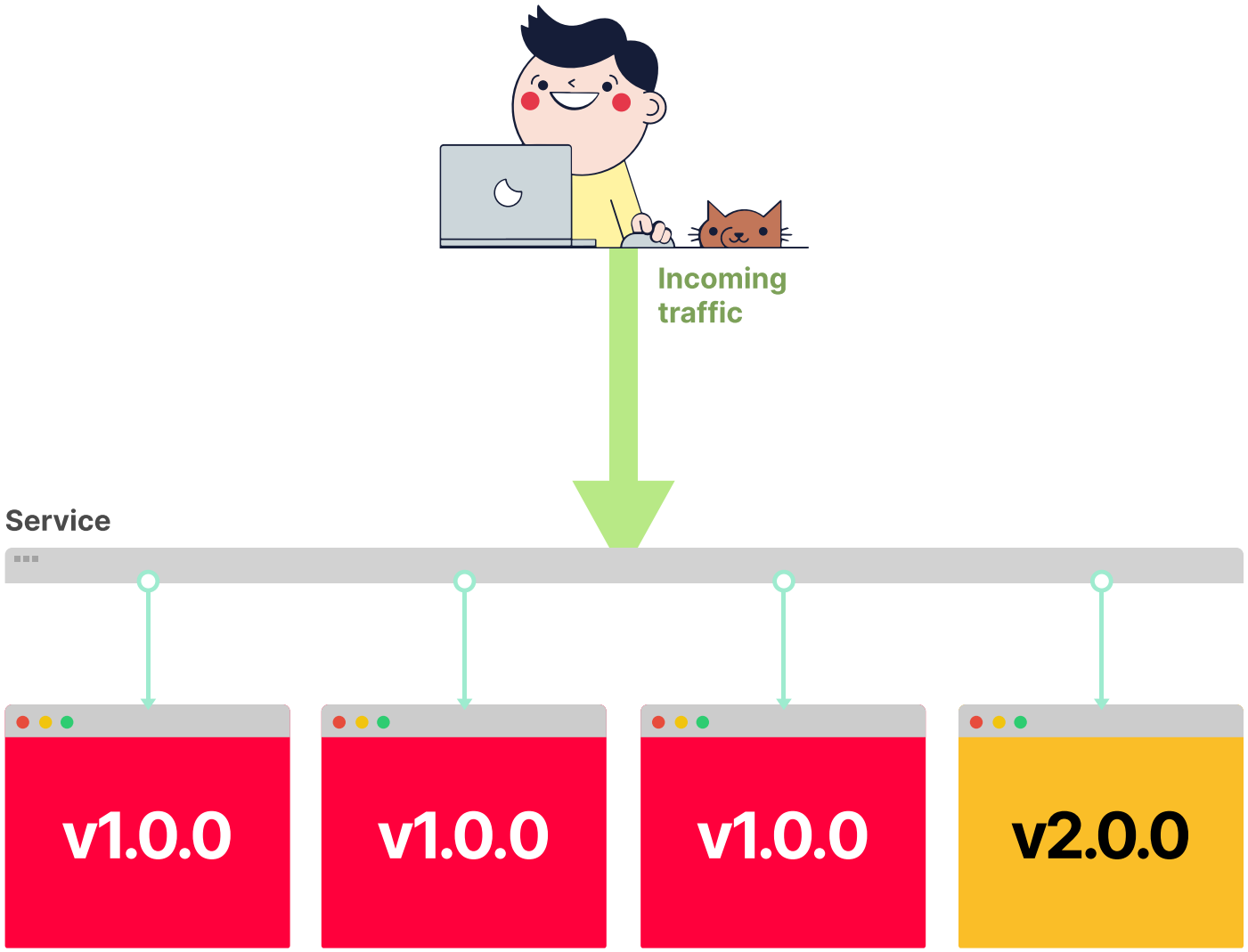

Incoming
traffic
Service
v2.0.0
v2.0.0
v2.0.0
Startup
Readiness
Startup
Readiness
Startup
Readiness

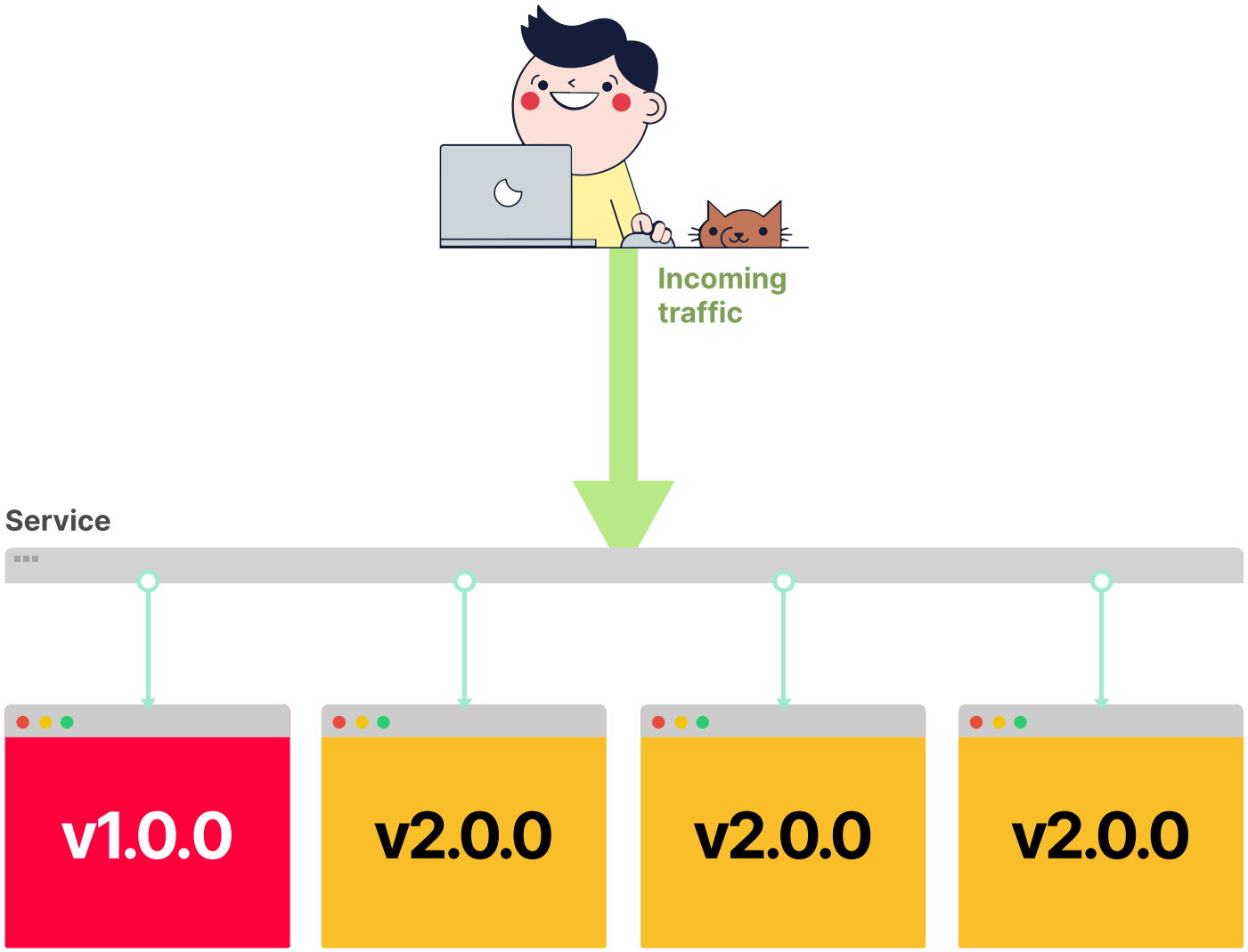

Incoming
traffic
Service
v2.0.0
v2.0.0
v2.0.0
Startup
Readiness
Startup
Readiness
Startup
Readiness

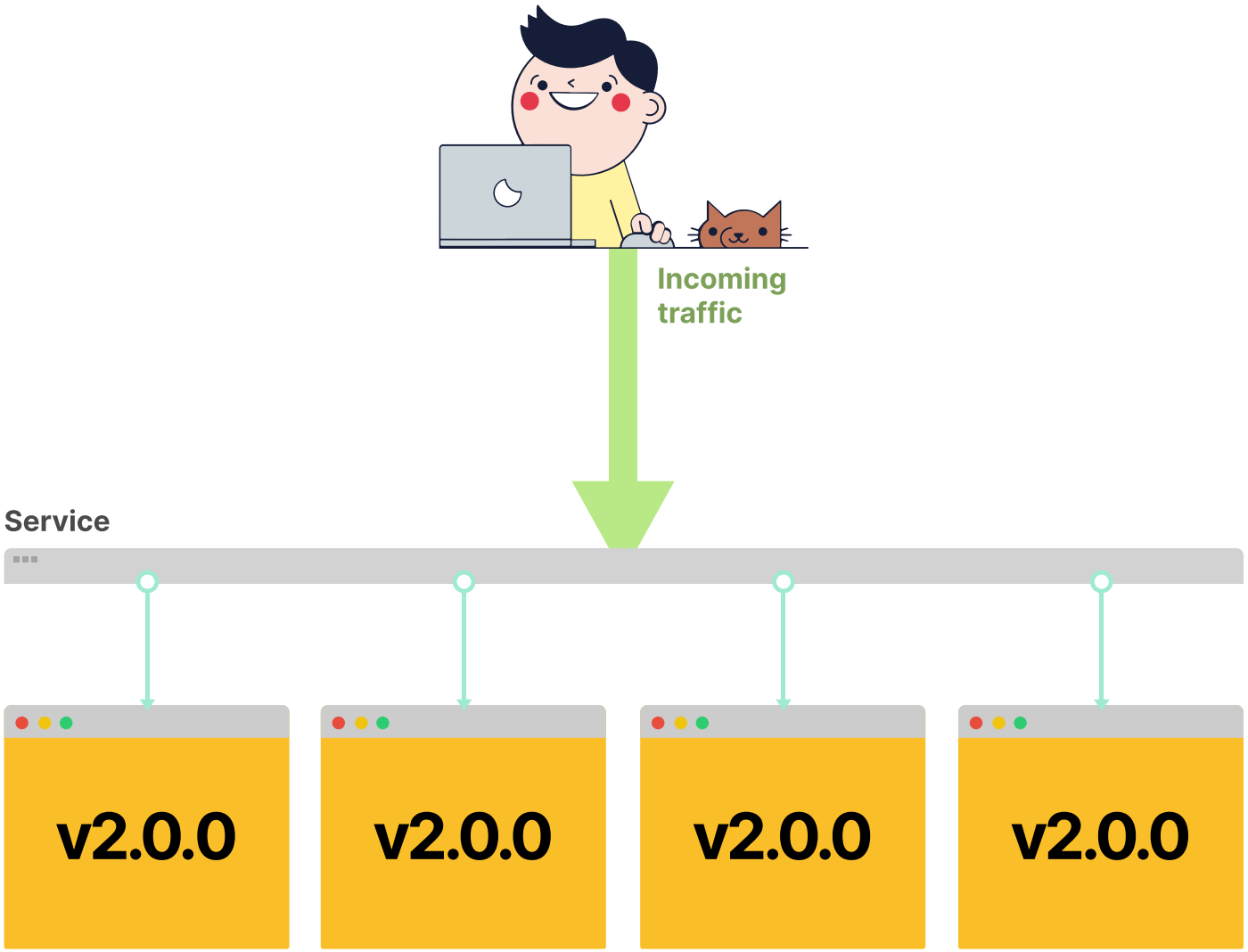
# Canary deployment

02

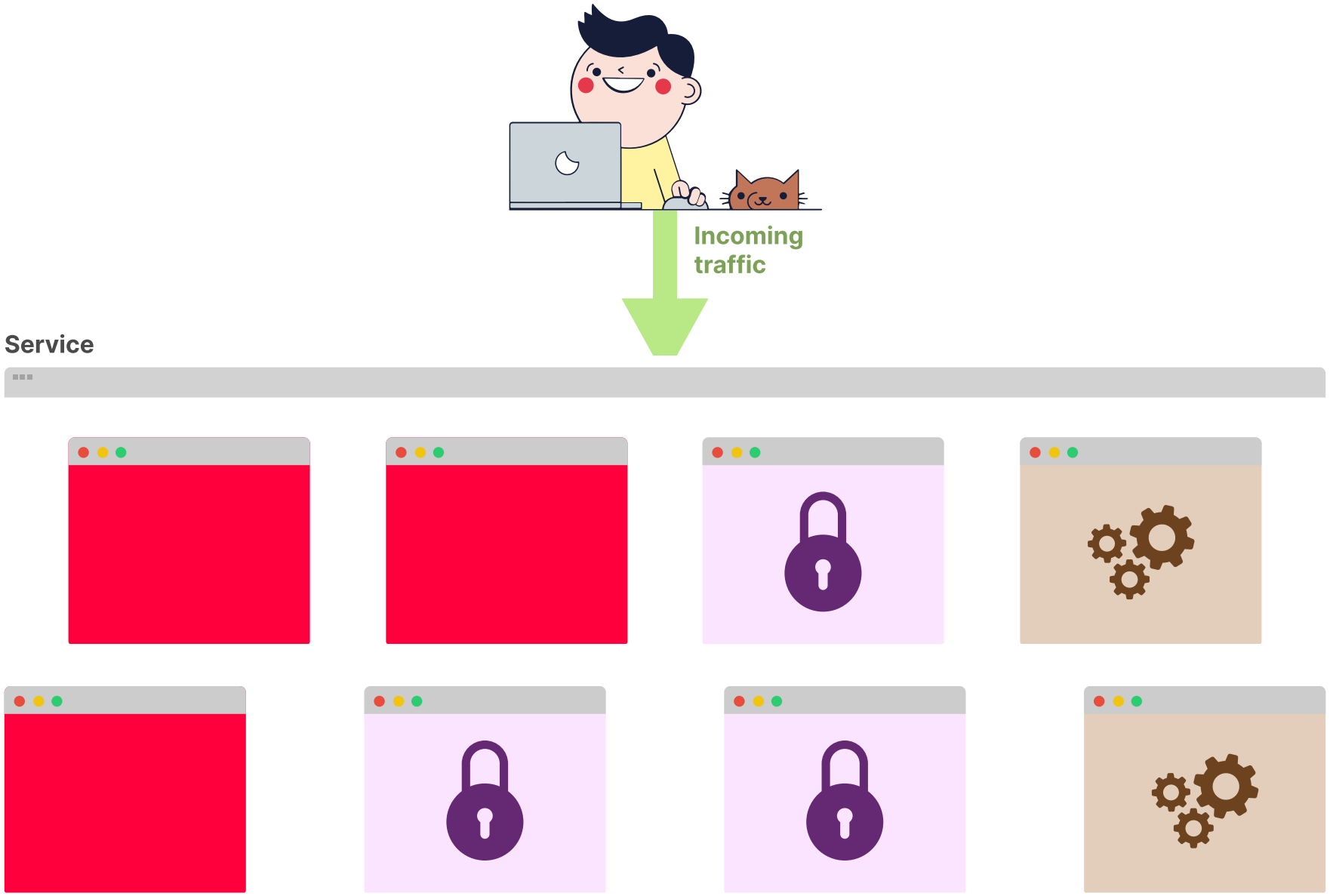

Incoming
traffic
Service
v1.0.0
v1.0.0
v1.0.0

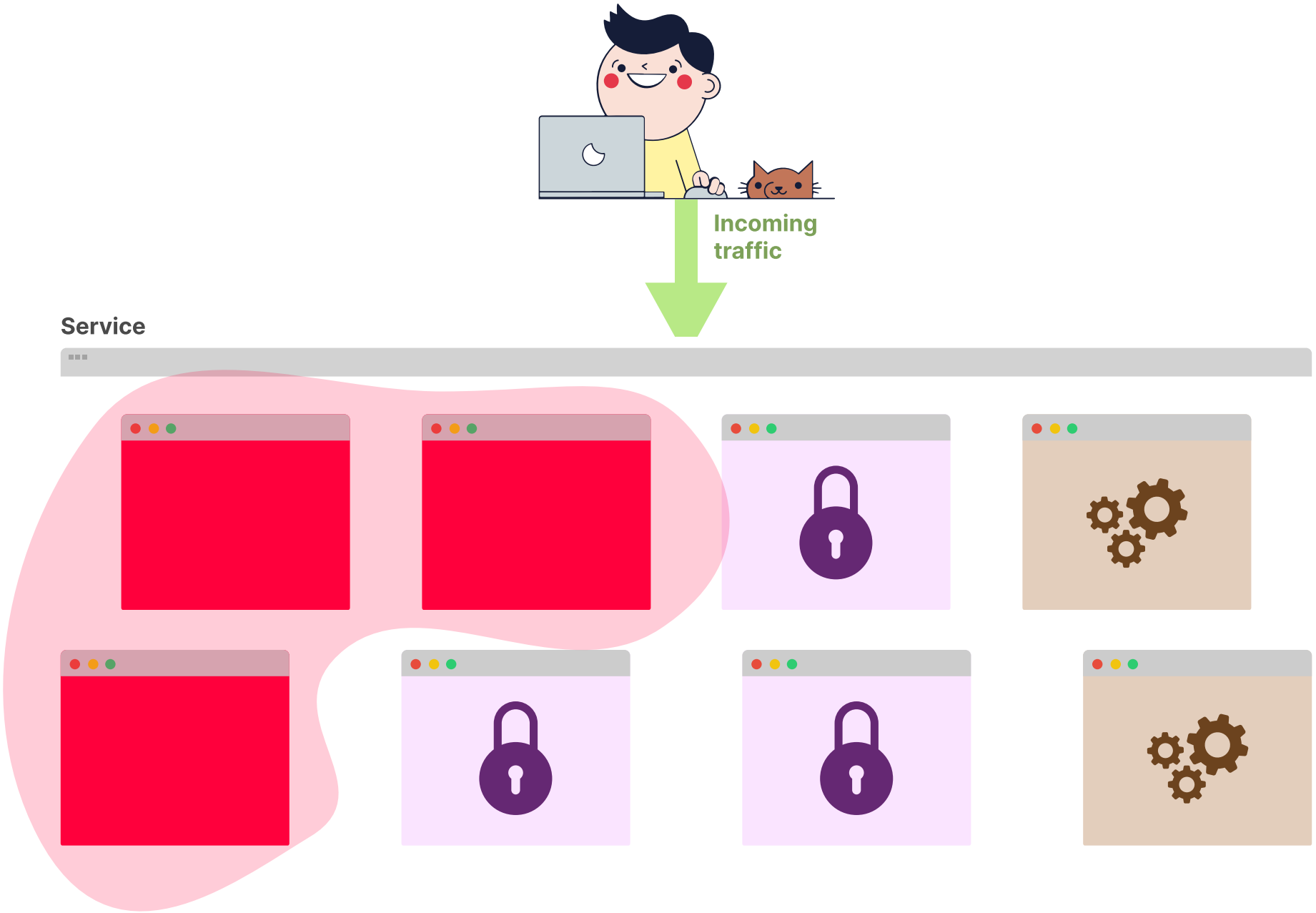

Incoming traffic
Service
v1.0.0
v1.0.0
v1.0.0
v2.0.0

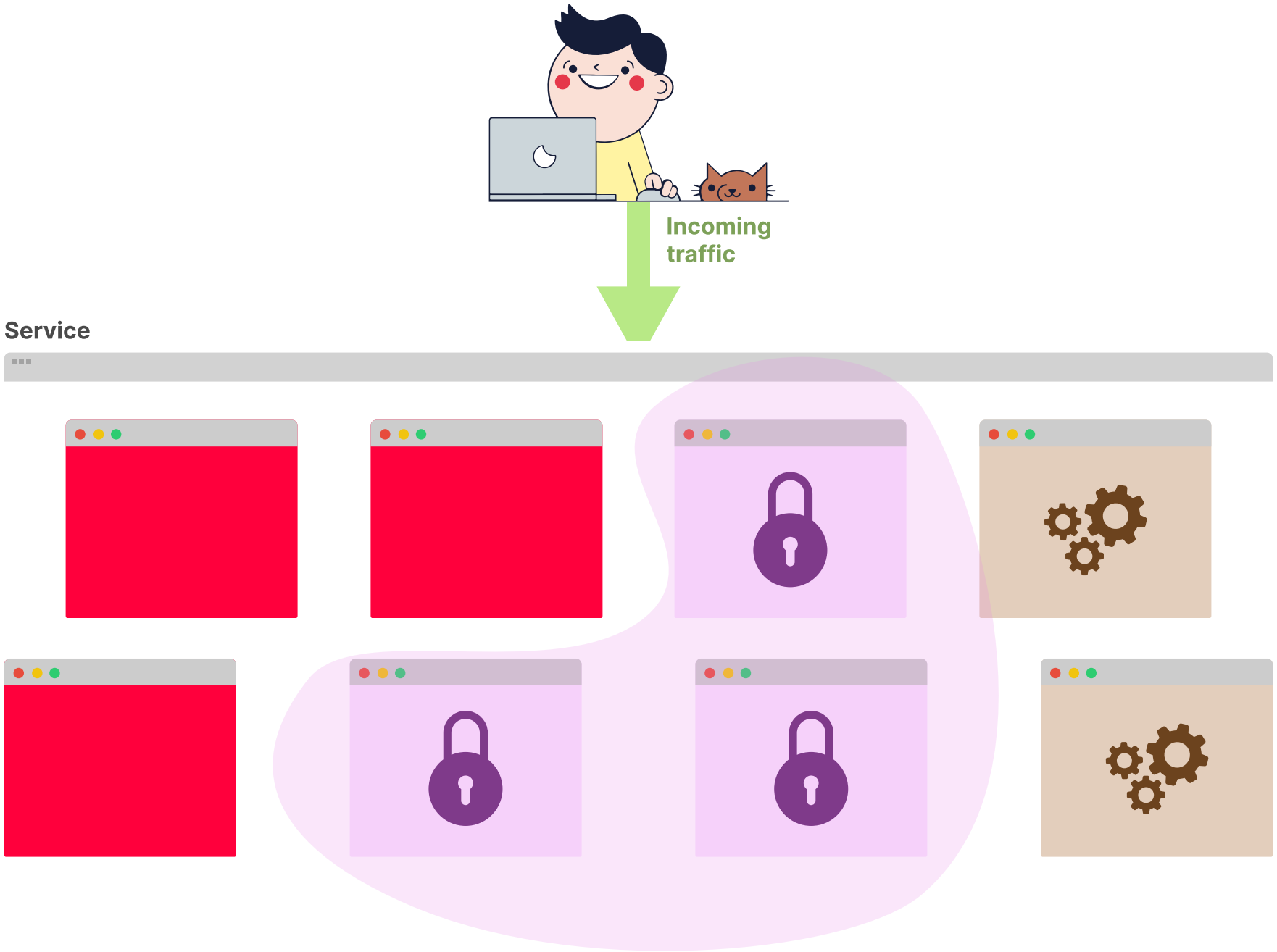

Incoming traffic
Service
v1.0.0
v2.0.0
v2.0.0
v2.0.0

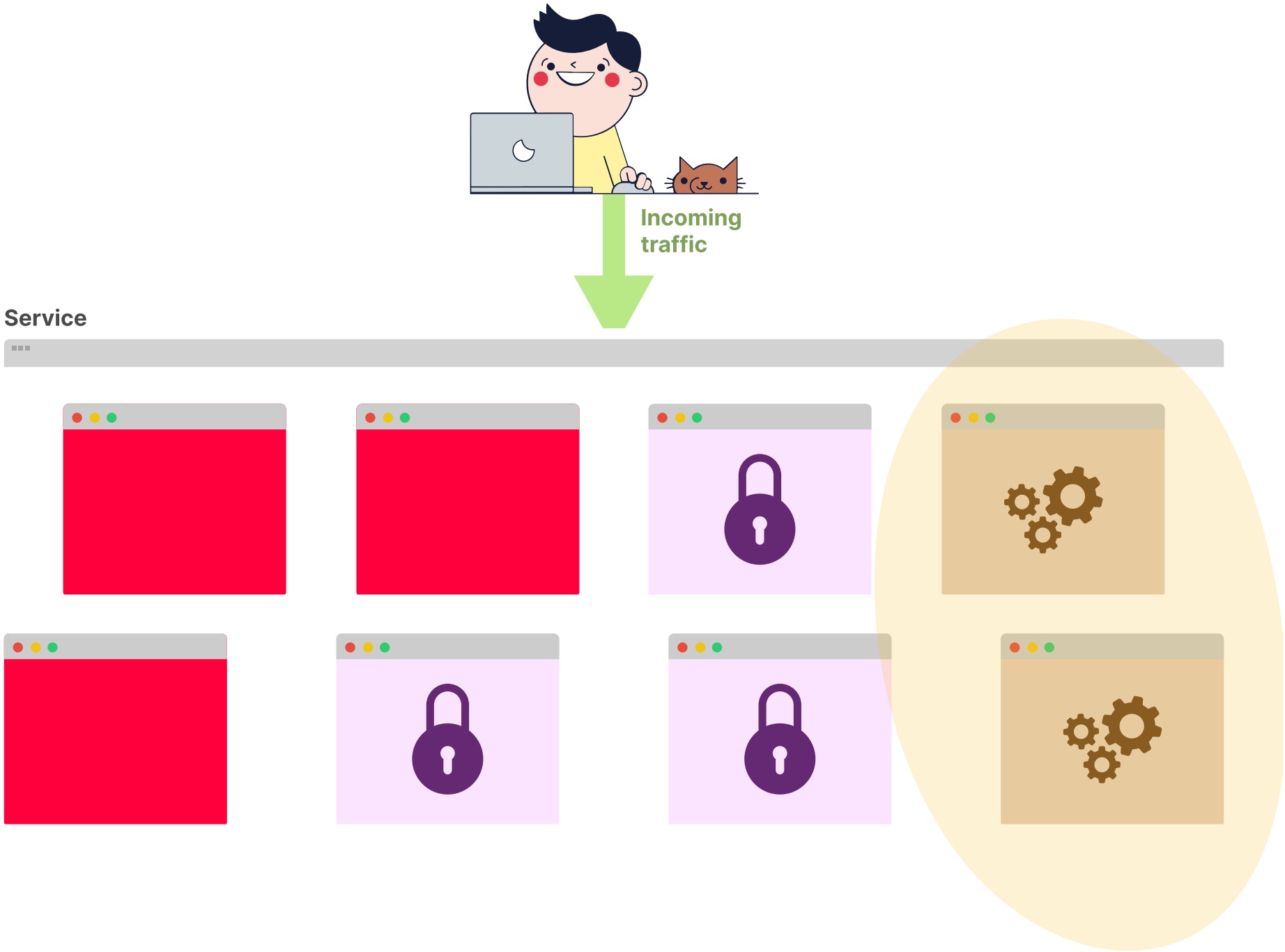

Incoming
traffic
Service
v2.0.0
v2.0.0
v2.0.0
v2.0.0

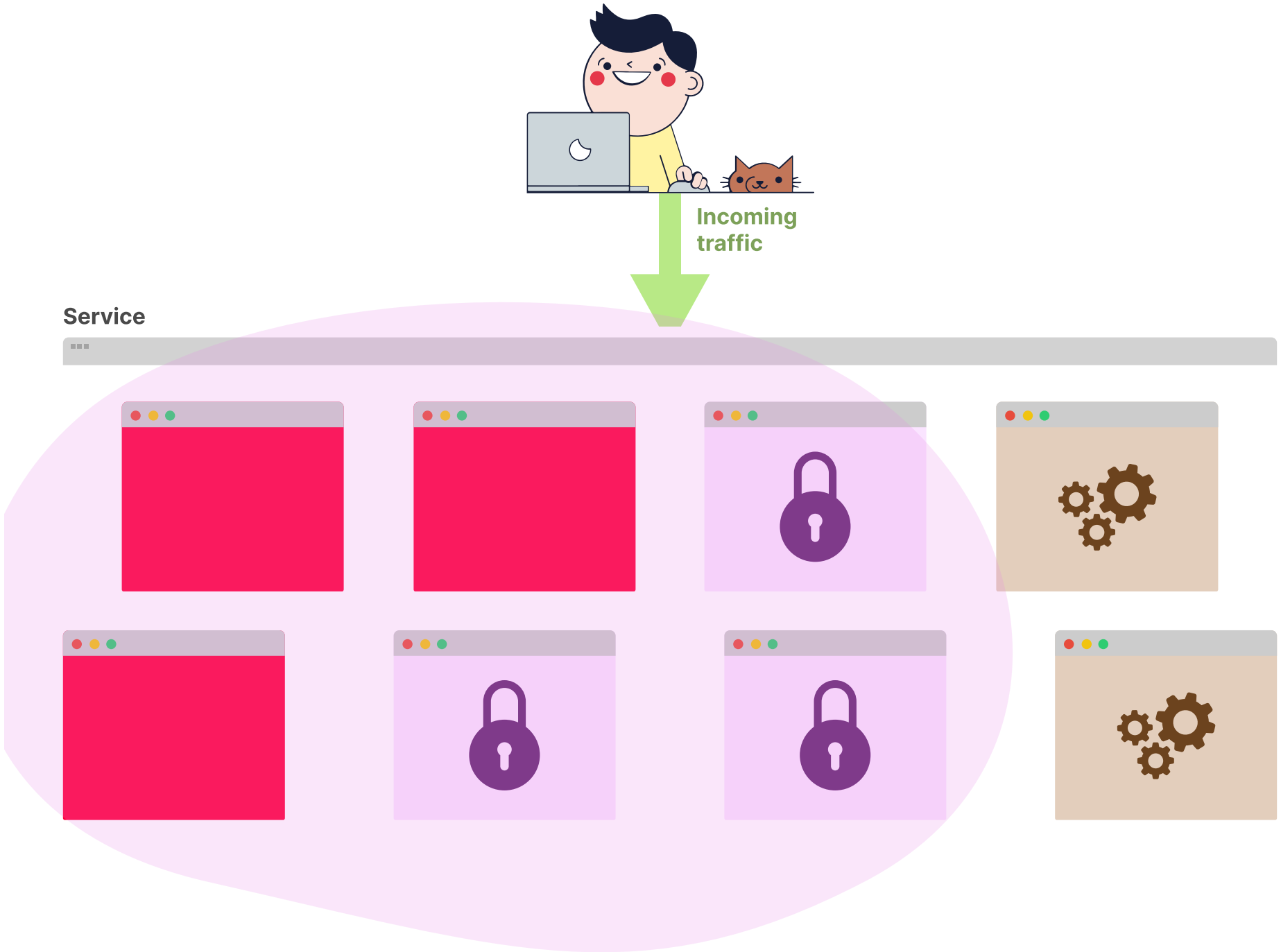
# Services as dynamic load balancers

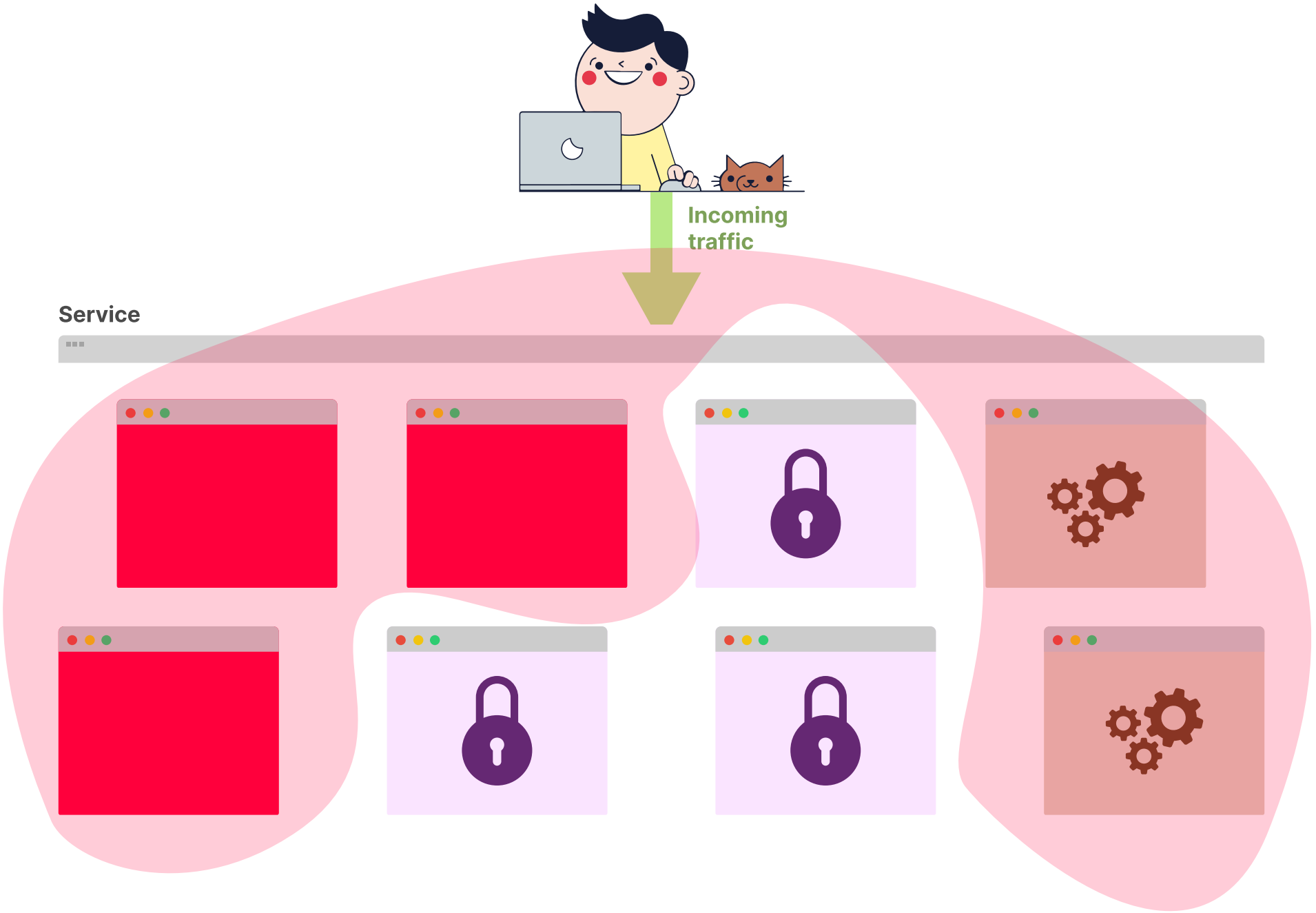

Incoming
traffic
Service


Incoming
traffic
Service


Incoming
traffic
Service


Incoming
traffic
Service


Incoming
traffic
Service


Incoming
traffic
Service

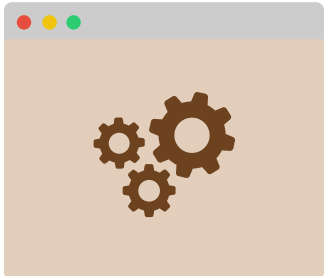
# Labels and selectors

# LABELS



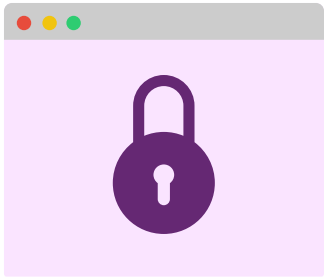
component: frontend

name: app



component: backend

name: app



version: 1.0.0

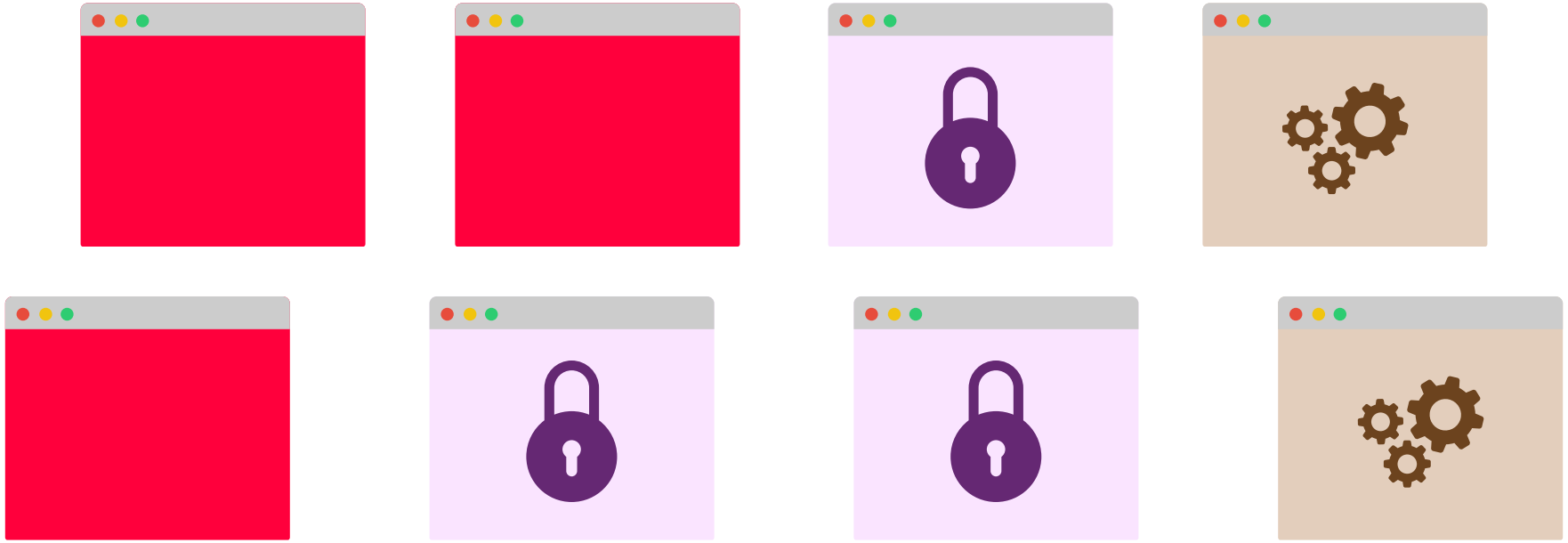
name: app

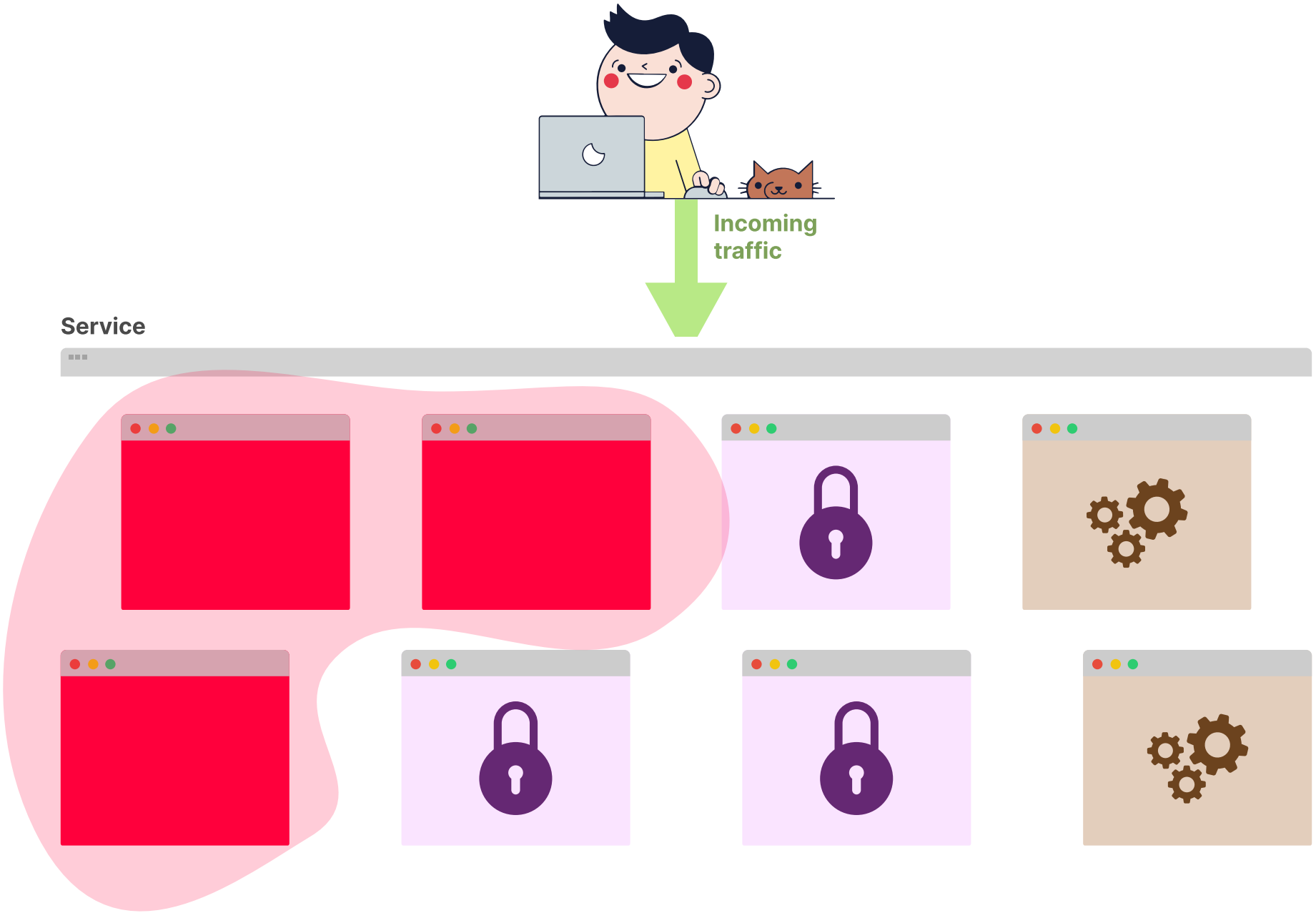
# SELECTOR

## component: frontend




Service



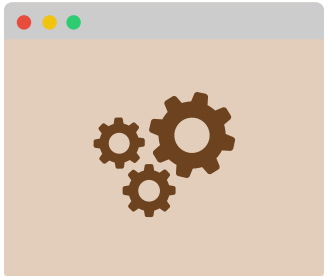

Incoming
traffic
Service

# LABELS



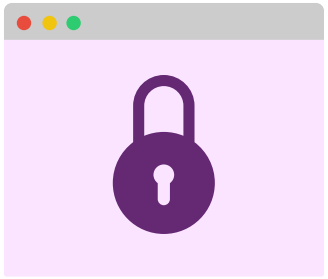
component: frontend

name: app



component: backend

name: app



version: 1.0.0

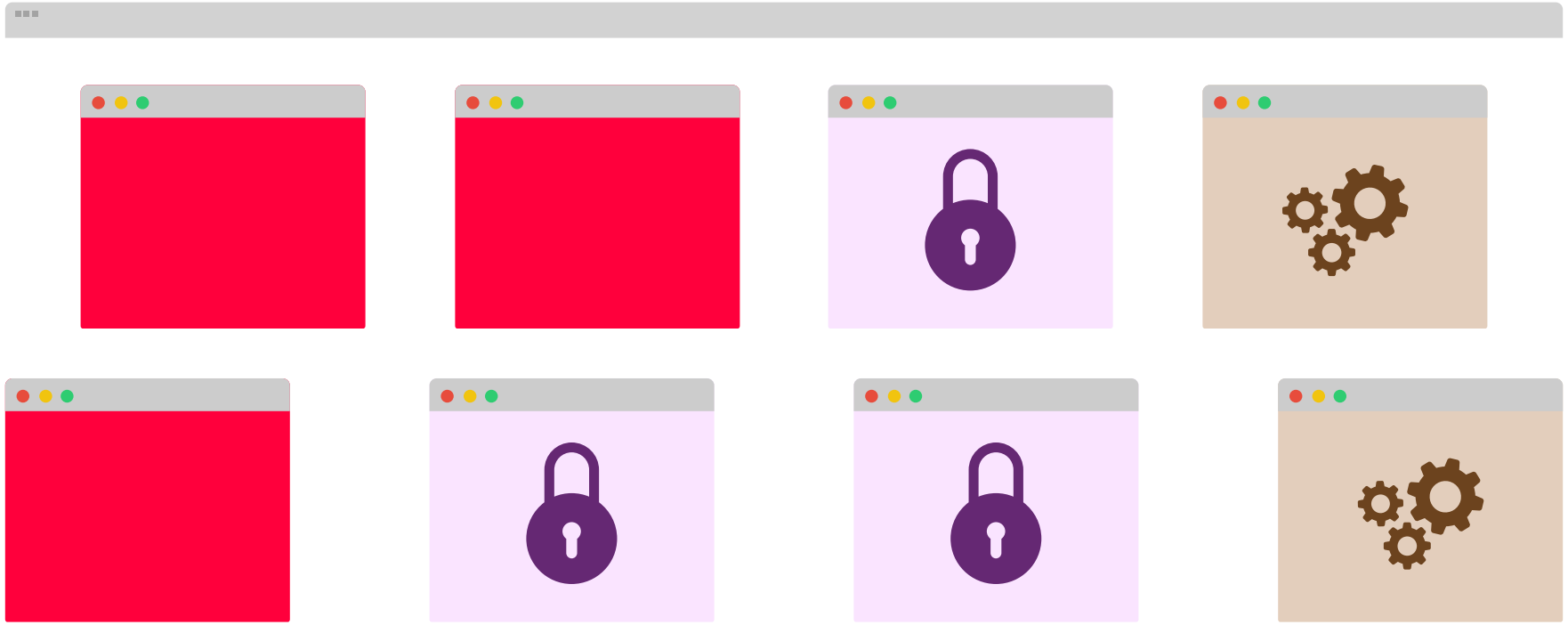
name: app

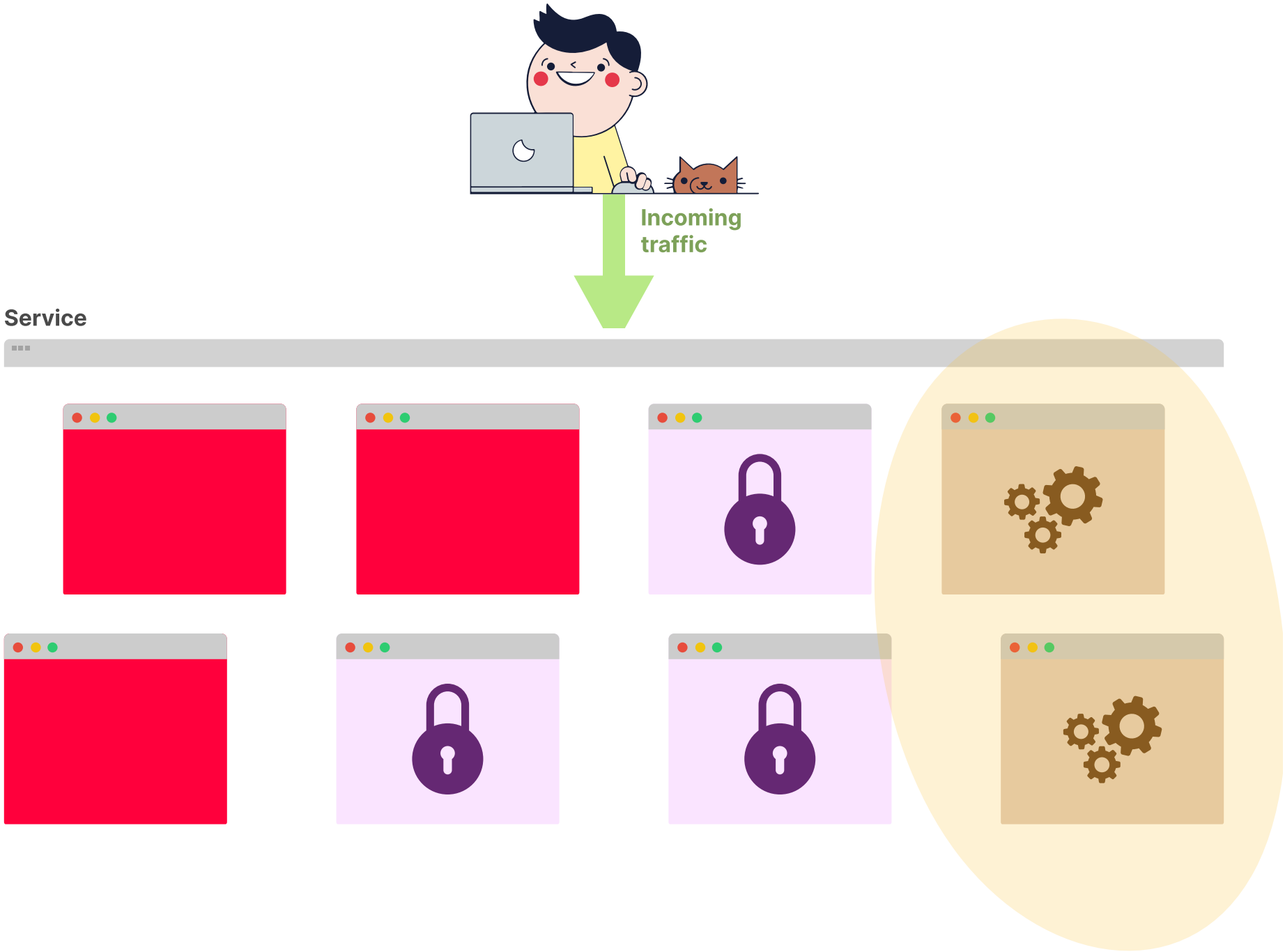
# SELECTOR

## component: backend




Service



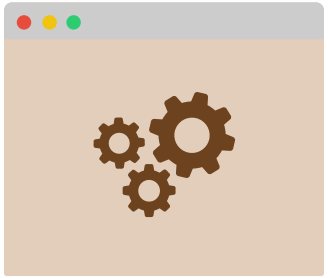

Incoming
traffic
Service

# LABELS



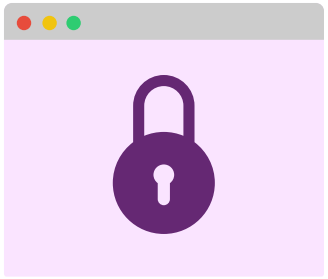
component: frontend

name: app



component: backend

name: app



version: 1.0.0

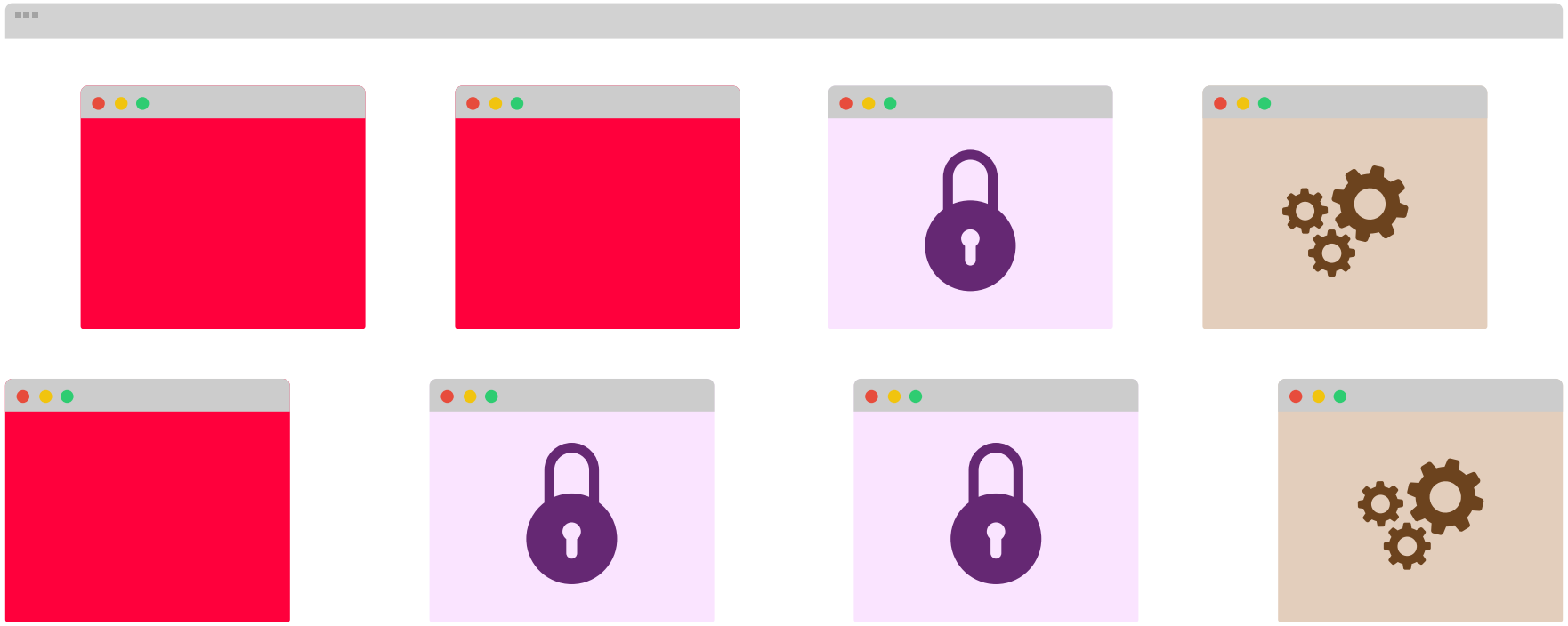
name: app

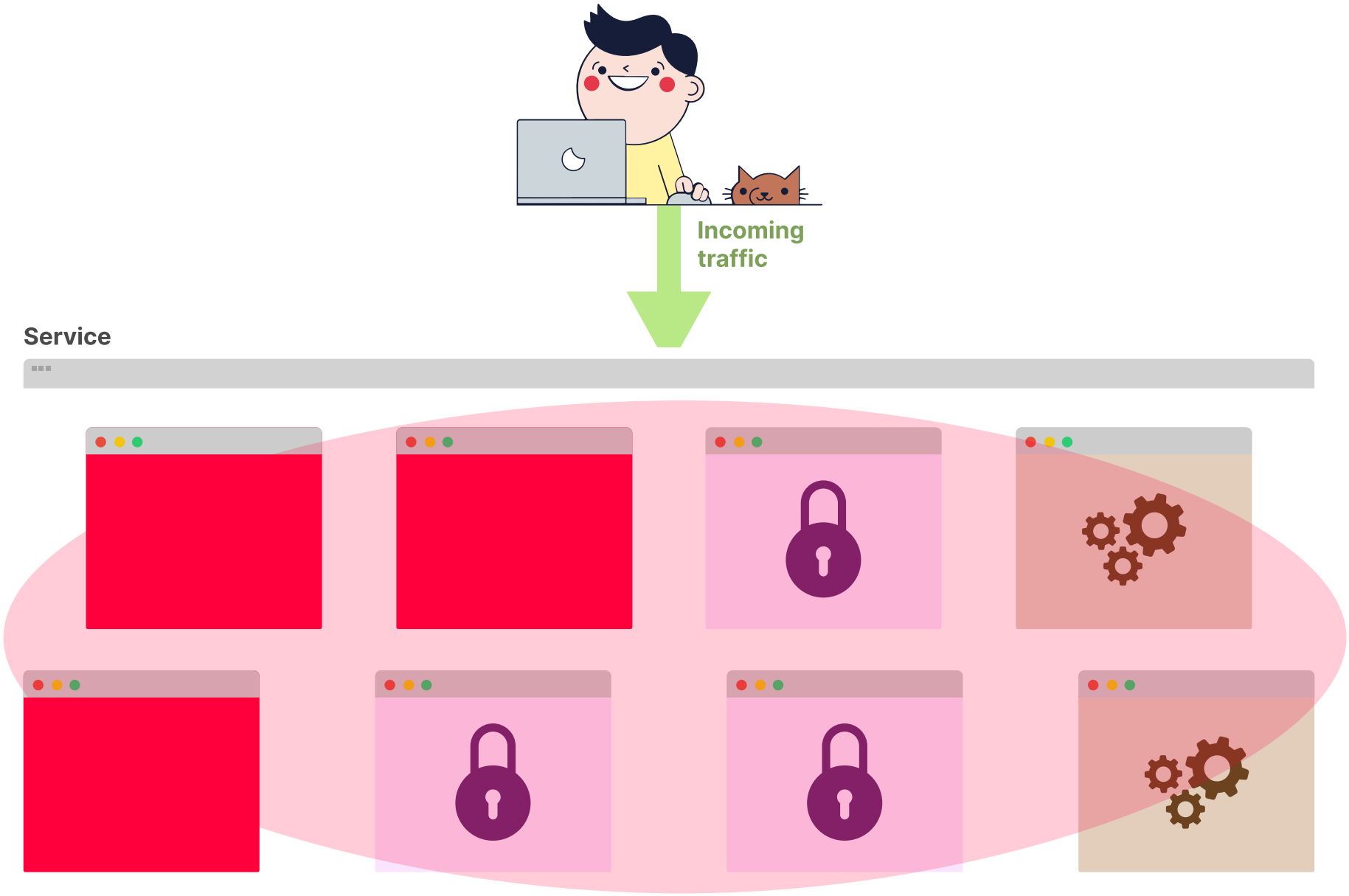
# SELECTOR

## name: app




Service



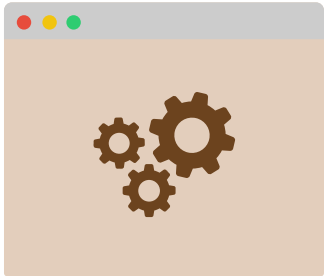

Incoming
traffic
Service

# LABELS



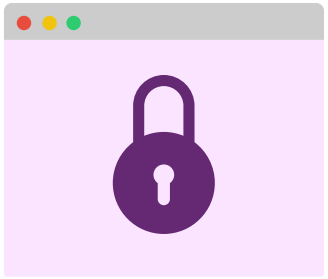
component: frontend

name: app



component: backend

name: app



version: 1.0.0

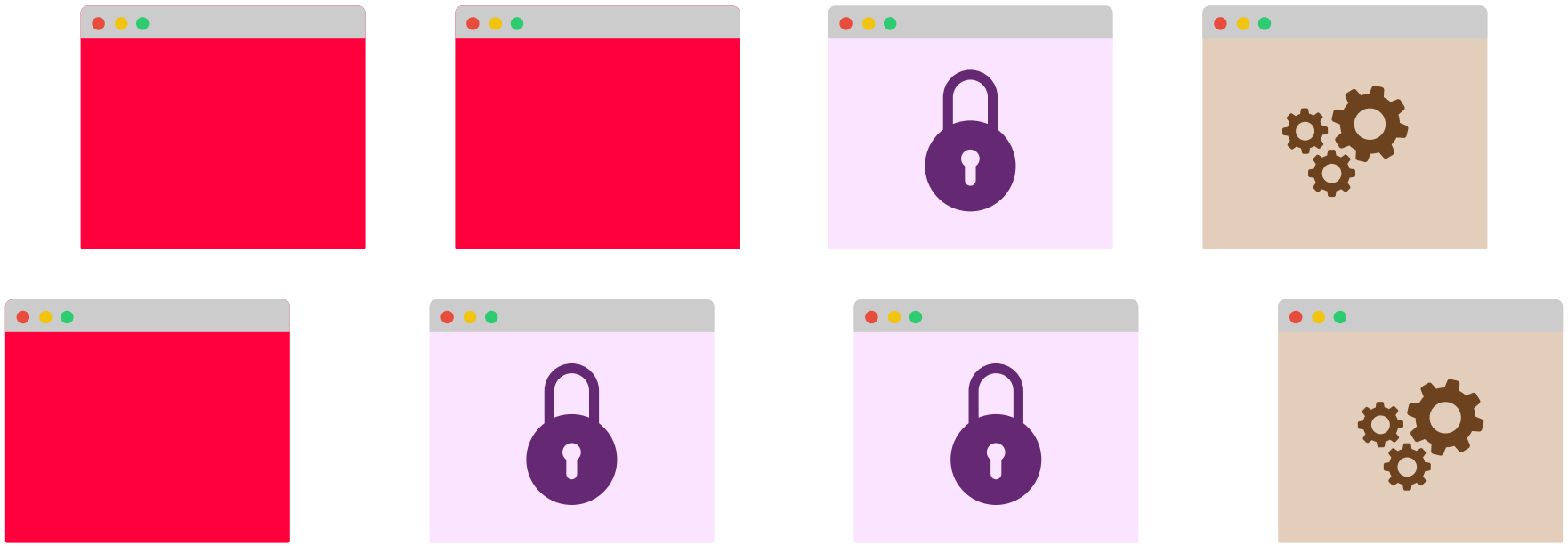
name: app

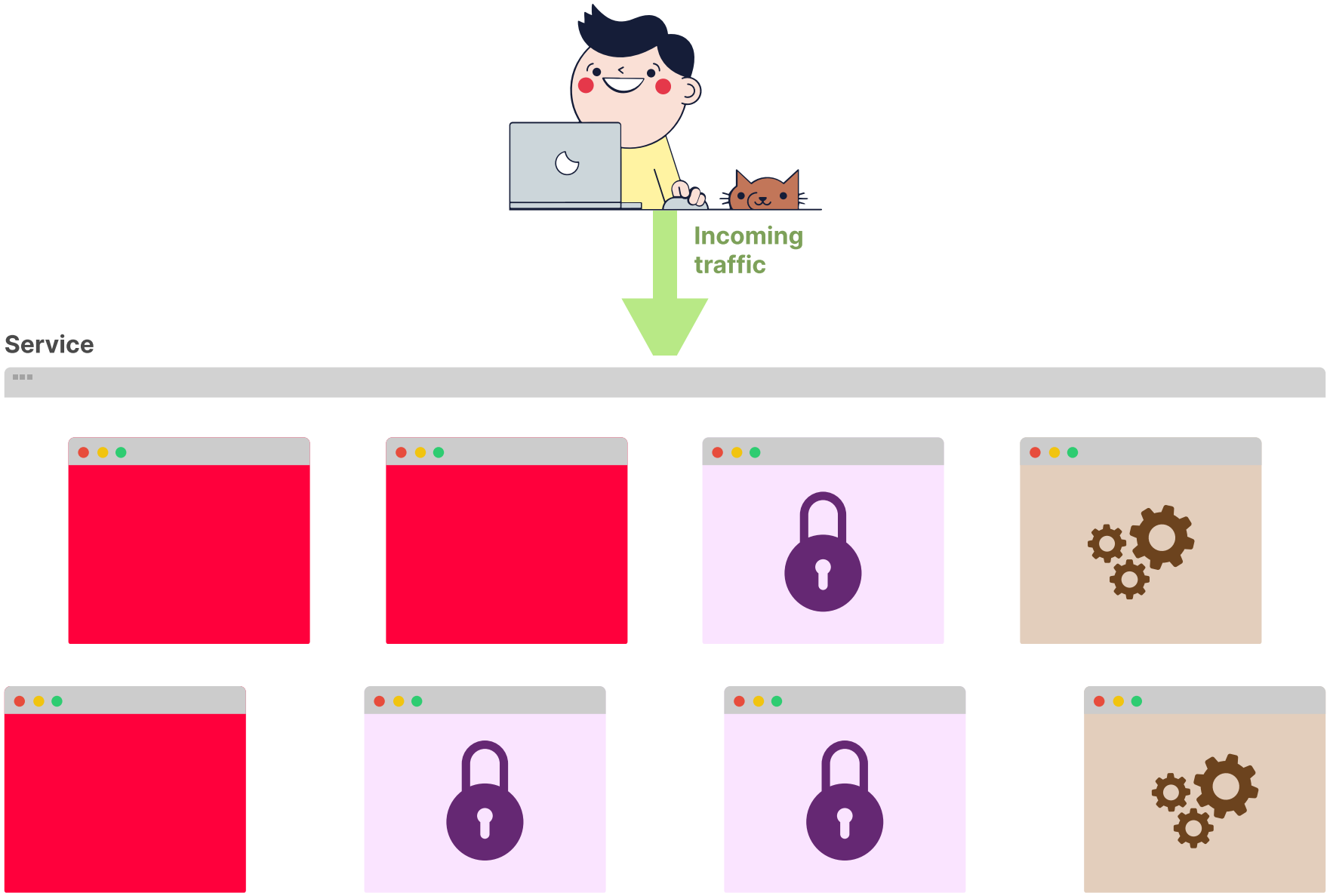
# SELECTOR

## component: frontend
## version: 1.0.0

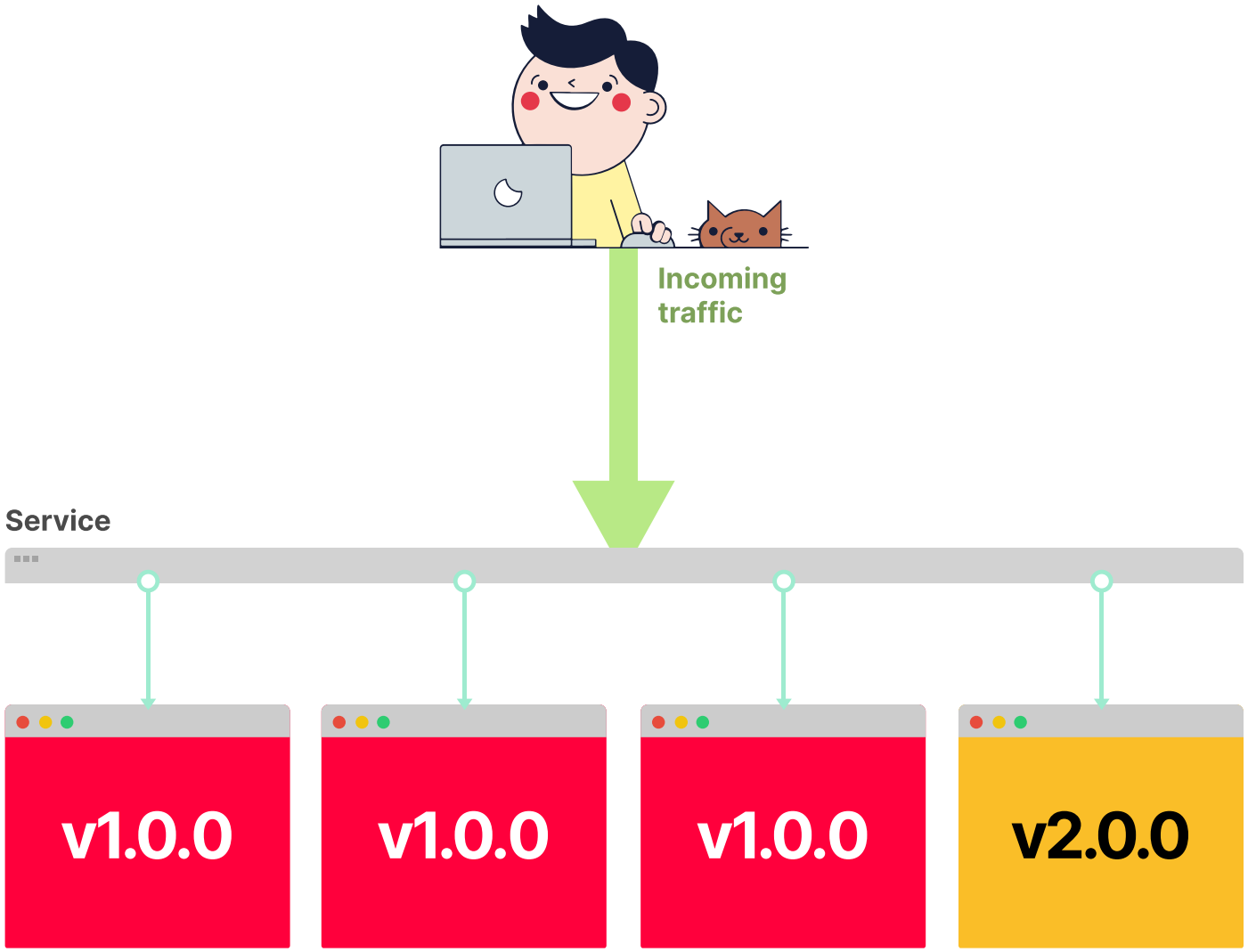



Service




Incoming
traffic
Service

# Canary deployment in practice


Incoming
traffic
Service
v1.0.0
v1.0.0
v1.0.0
v2.0.0

## Canary deployment in practice

# 2 Deployments

# 1 Service

# 1 Common label

## Canary deployment in practice

2 Deployments

**1 Service**

1 Common label

## Canary deployment in practice

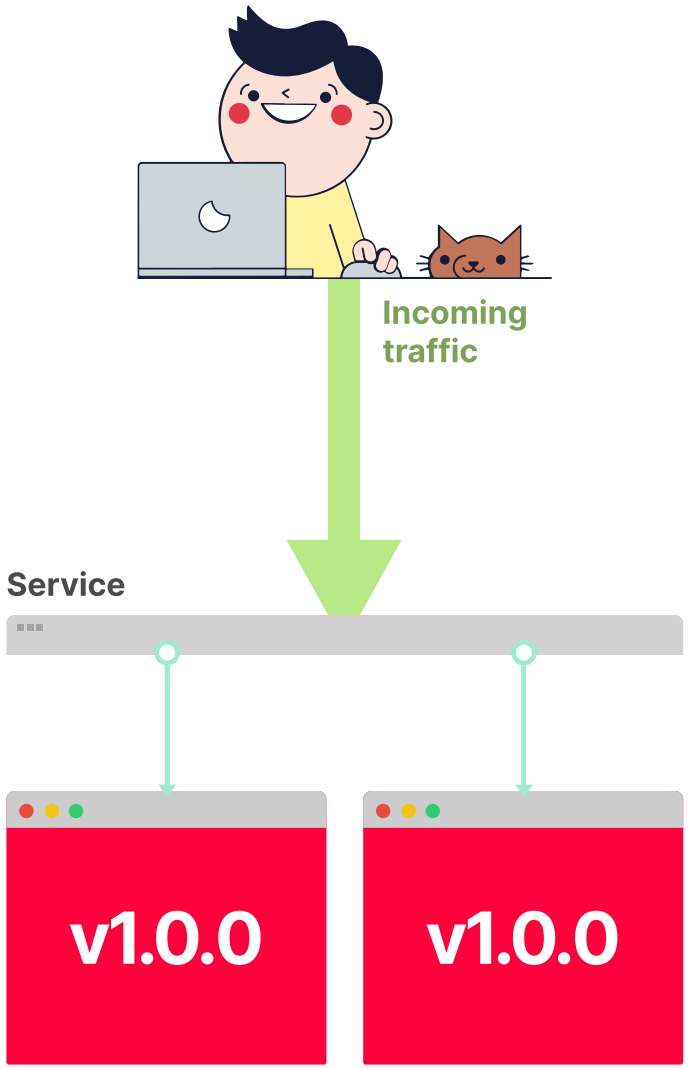
2 Deployments

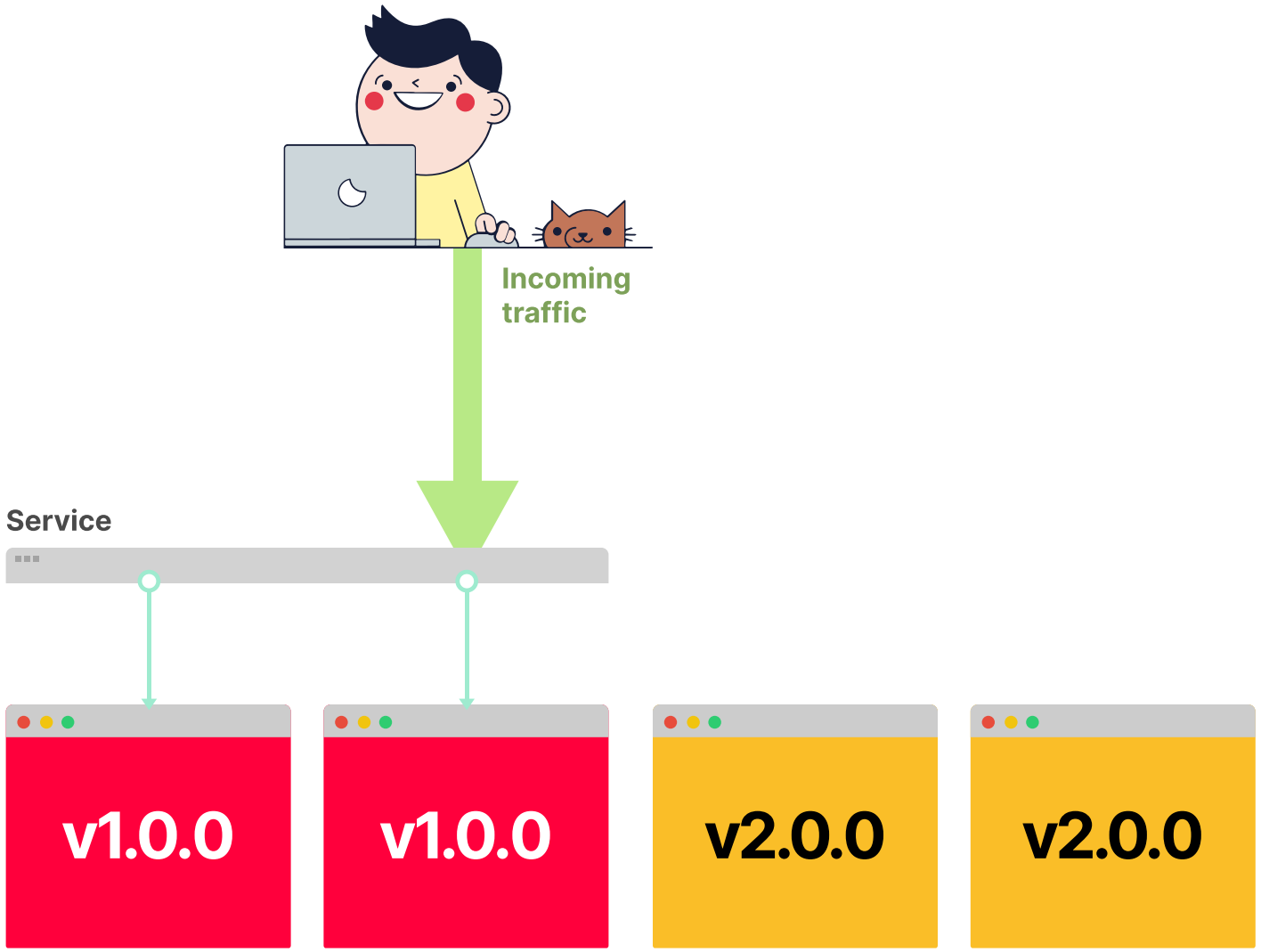
1 Service

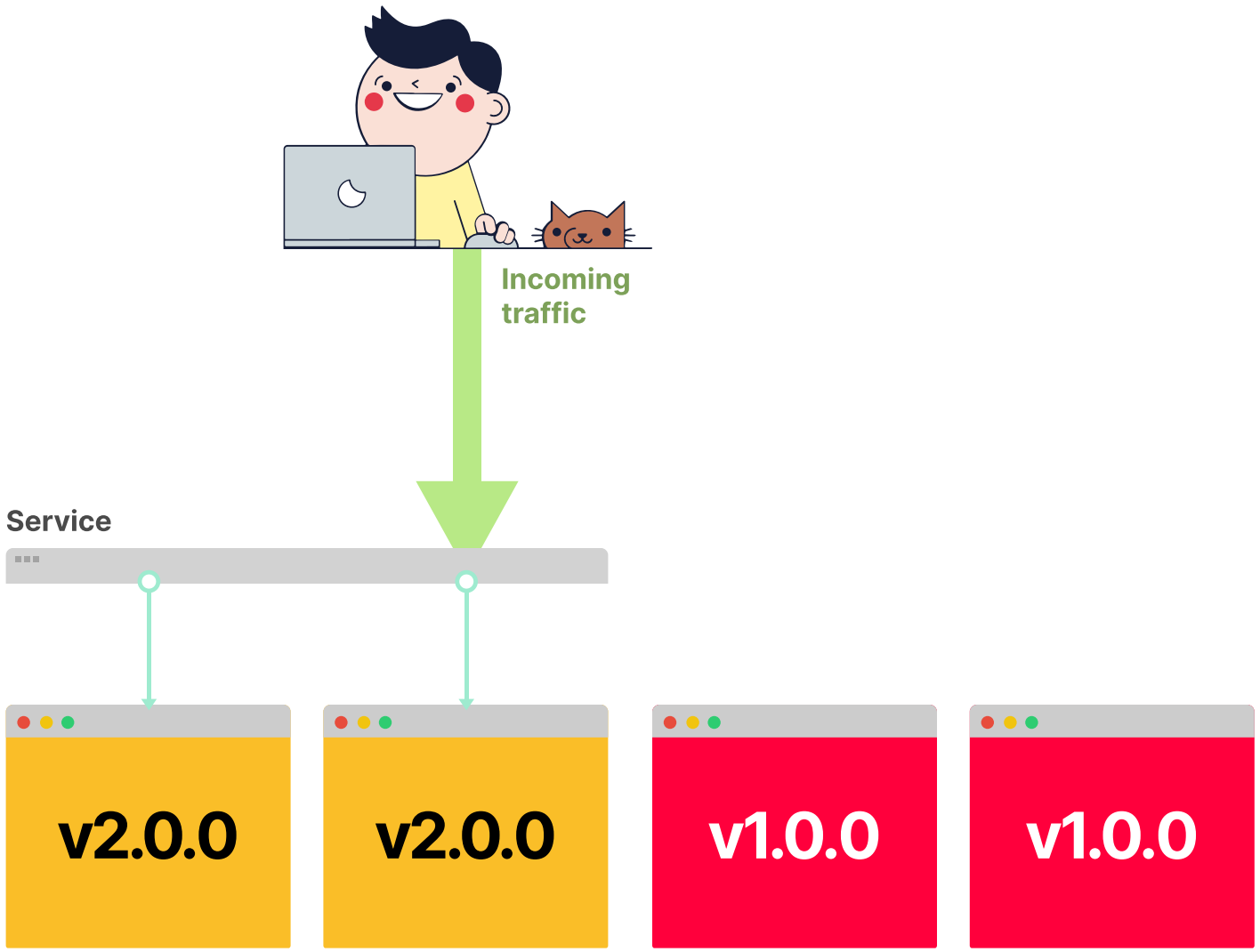
**1 Common label**

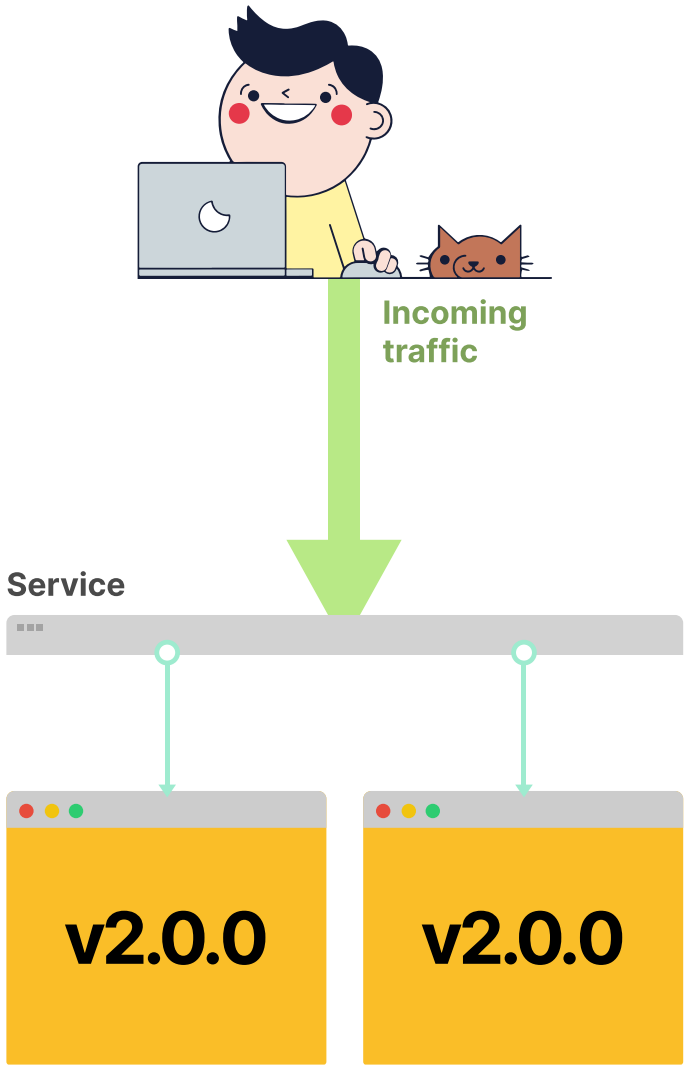
# Blue-green deployment

03


Incoming
traffic
Service
v1.0.0
v1.0.0


Incoming
traffic
Service
v1.0.0
v1.0.0
v2.0.0
v2.0.0


Incoming
traffic
Service
v2.0.0
v2.0.0
v1.0.0
v1.0.0


Incoming
traffic
Service
v2.0.0
v2.0.0

## Blue-green deployment

# 2 Deployments

# 1 Services

# Switch label

## Blue-green deployment

2 Deployments

**1 Services**

Switch label

## Blue-green deployment

2 Deployments

1 Services

**Switch label**

# Recap

Deployment strategies

# Recap — Deployment strategies

1. Rolling updates & readiness probe
2. Canary deployment
3. Service selector & labels
4. Blue-green deployment

# Recap — Deployment strategies

1. Rolling updates & readiness probe

2. Canary deployment

3. Service selector & labels

4. Blue-green deployment

# Recap — Deployment strategies

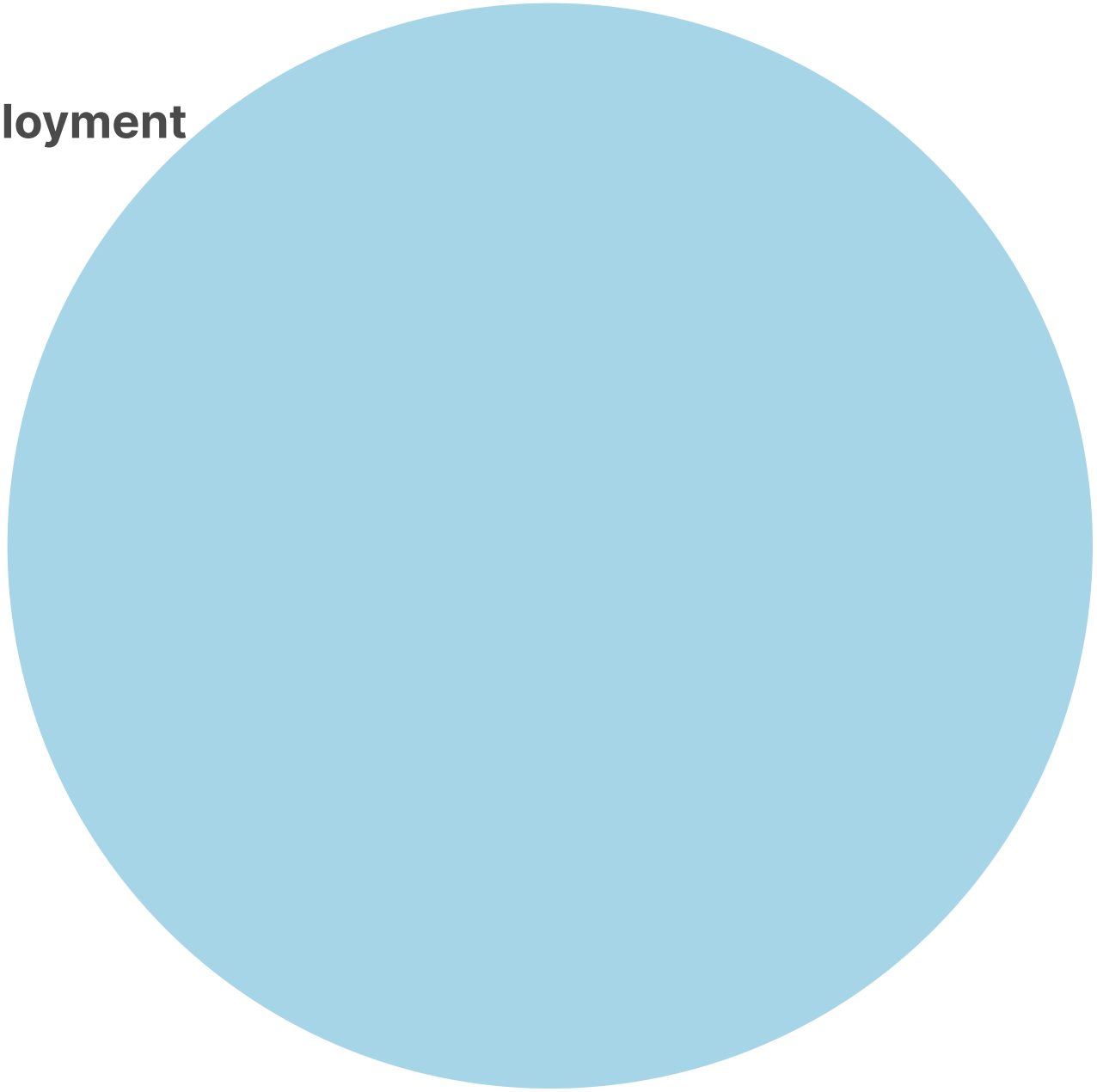
1. Rolling updates & readiness probe
2. Canary deployment
3. **Service selector & labels**
4. Blue-green deployment

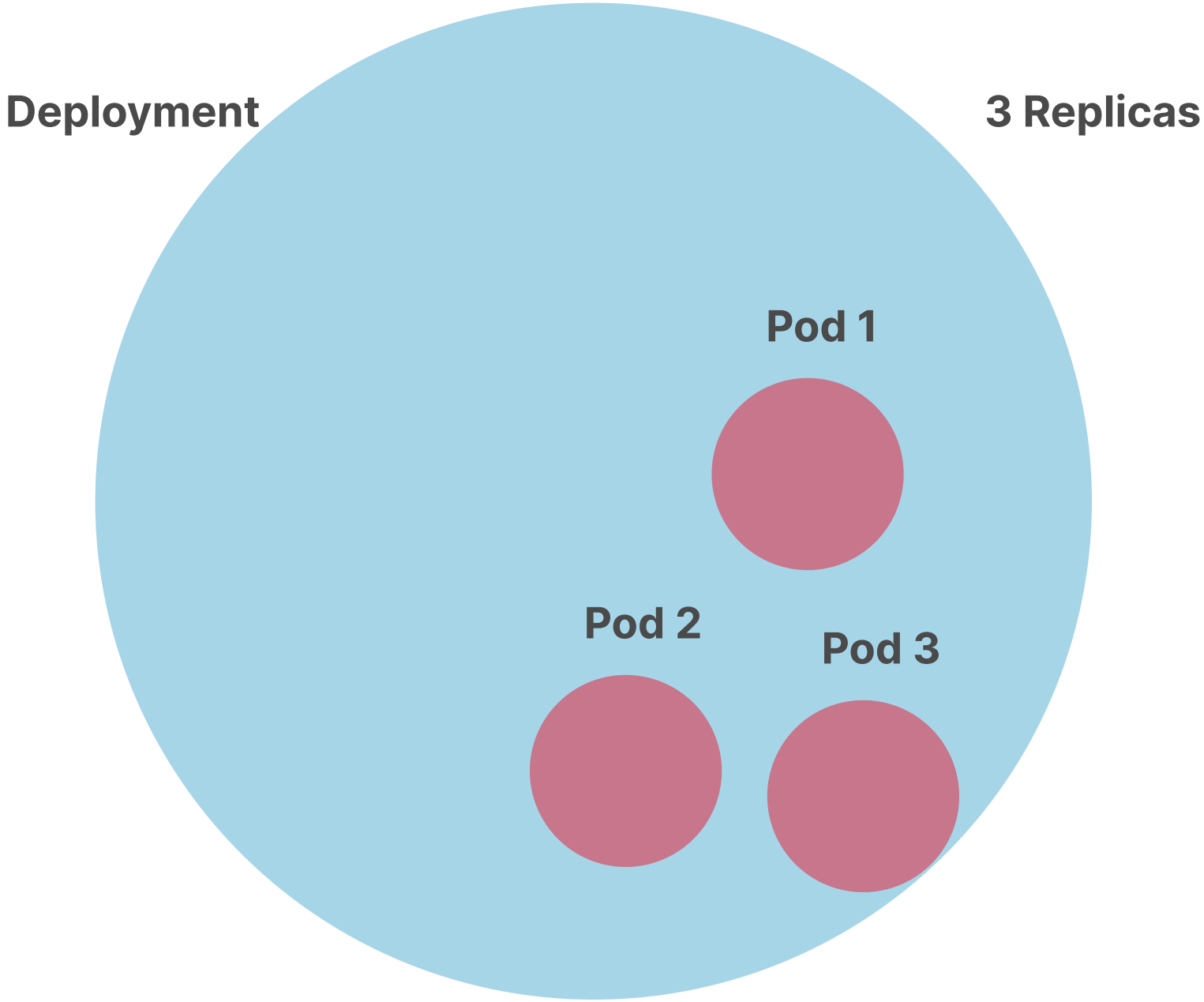
# Recap — Deployment strategies

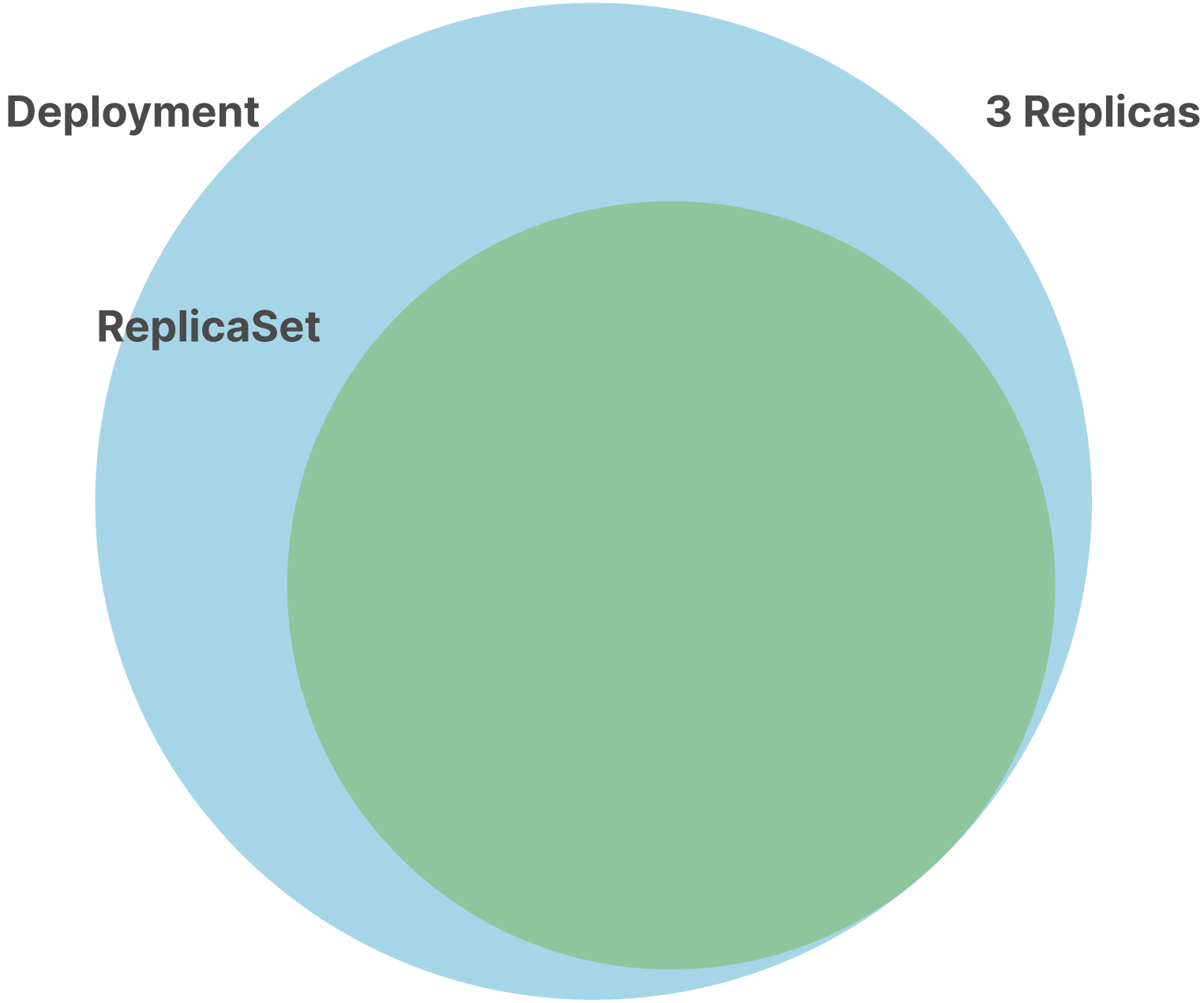
1. Rolling updates & readiness probe
2. Canary deployment
3. Service selector & labels
4. **Blue-green deployment**

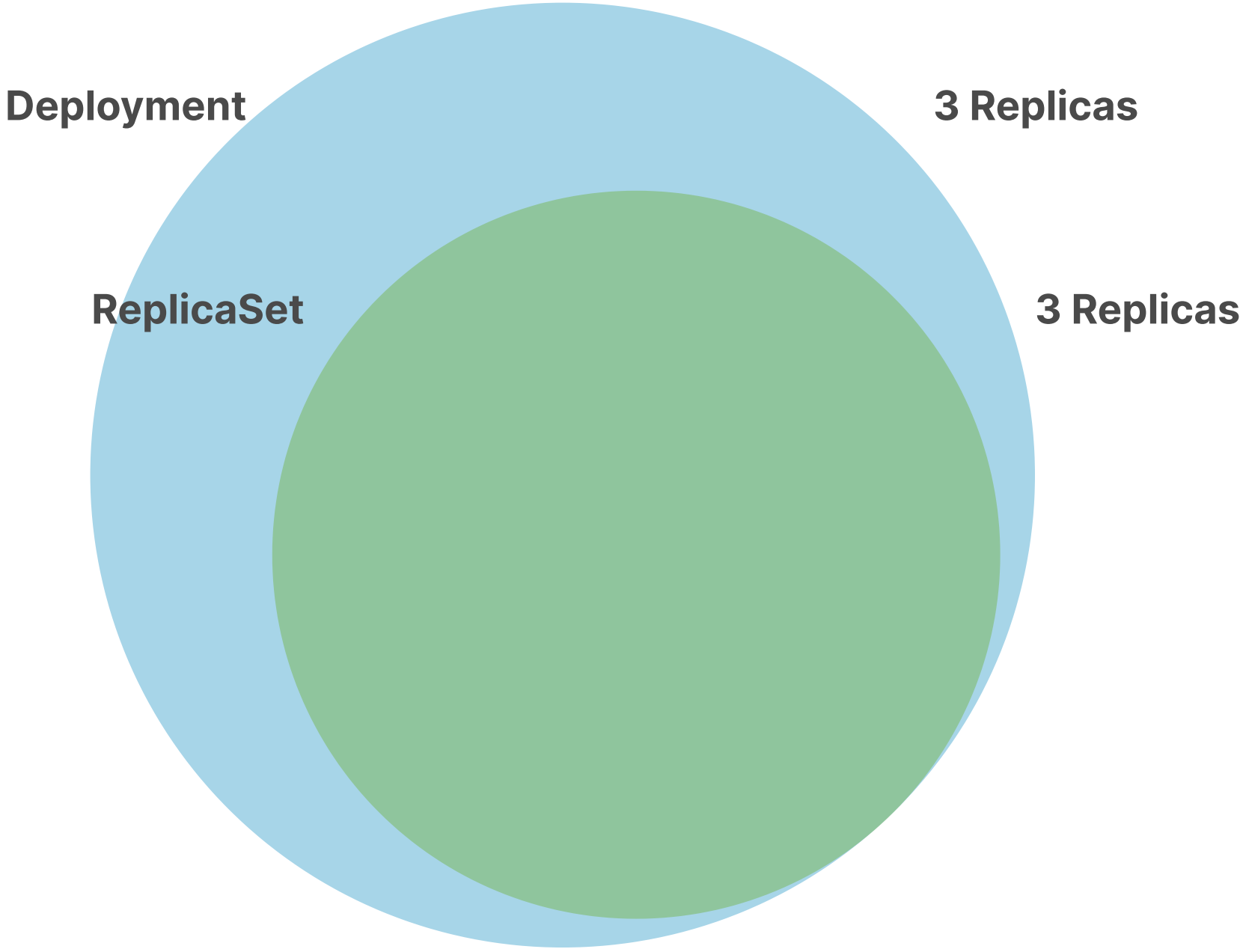
# A deeper look into Deployments

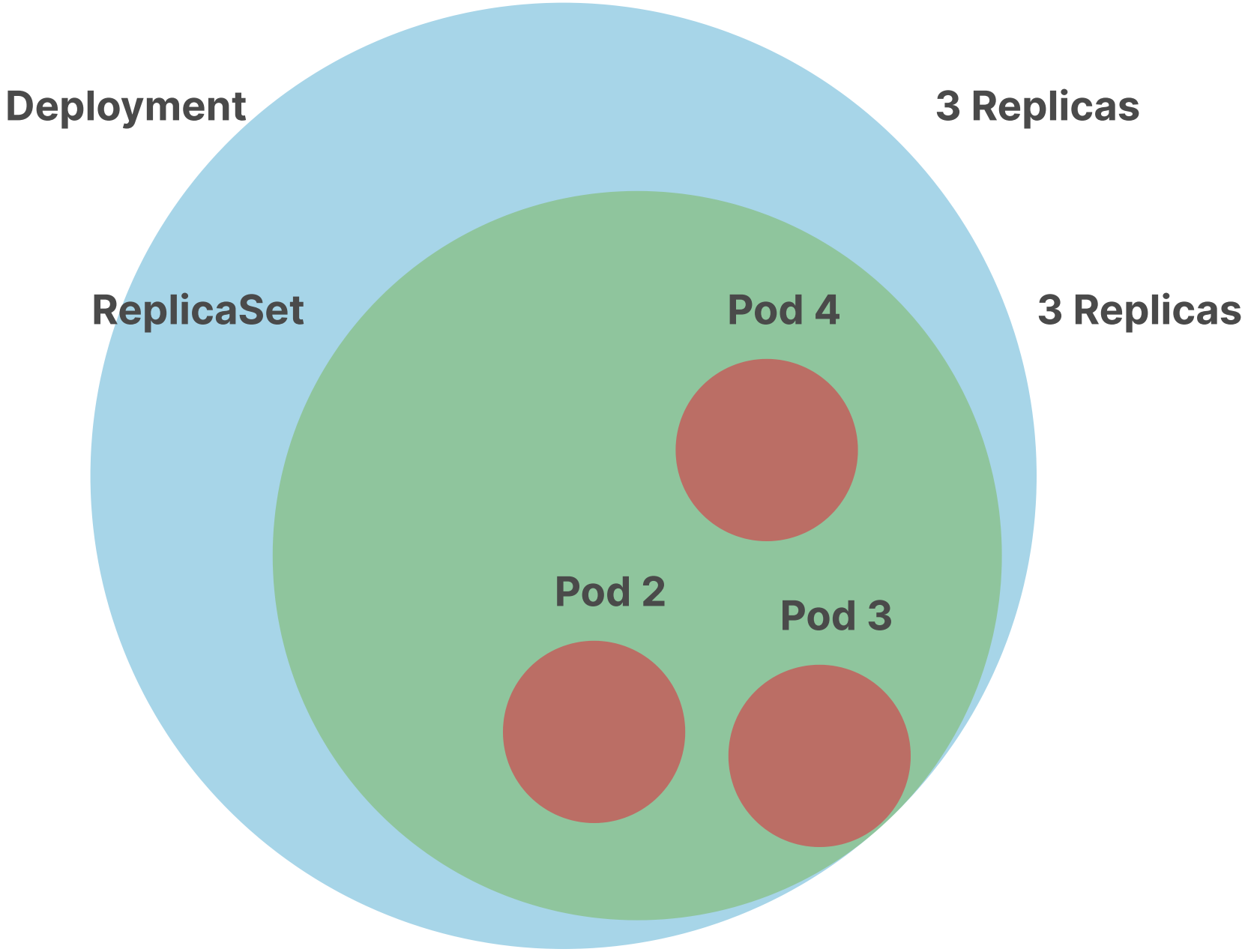
Deployment



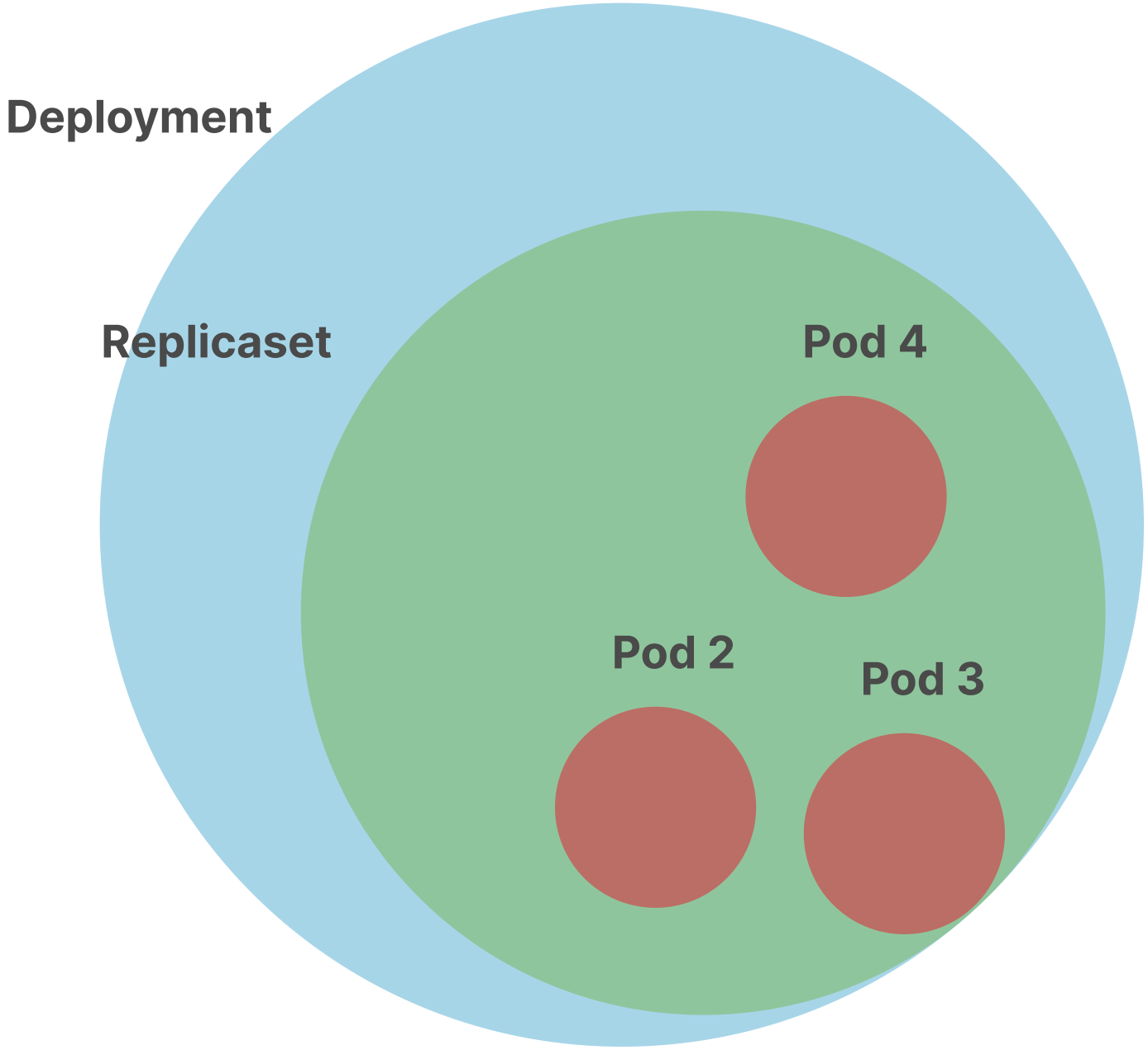

Deployment
3 Replicas
Pod 1
Pod 2
Pod 3

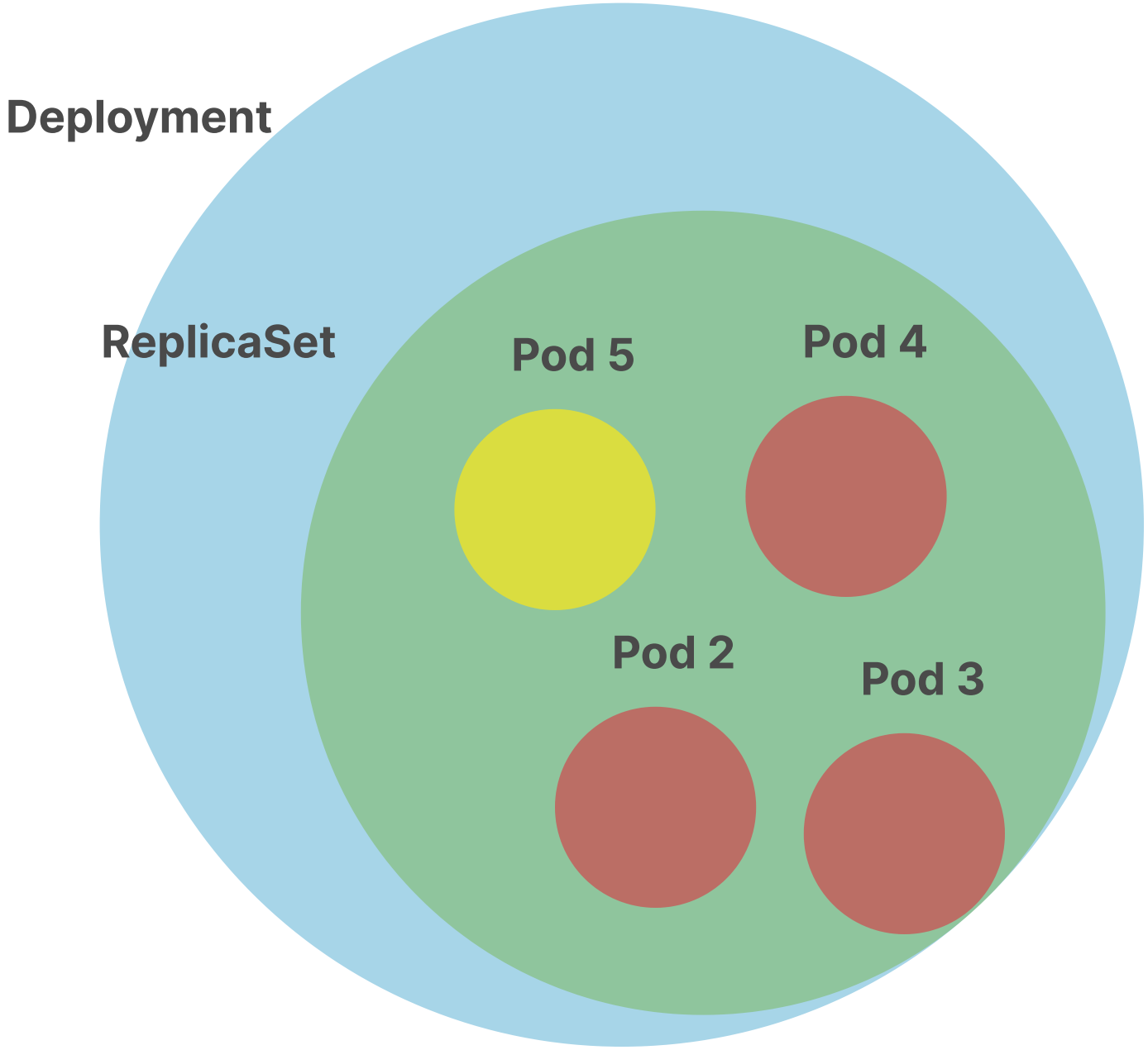

Deployment
3 Replicas
ReplicaSet

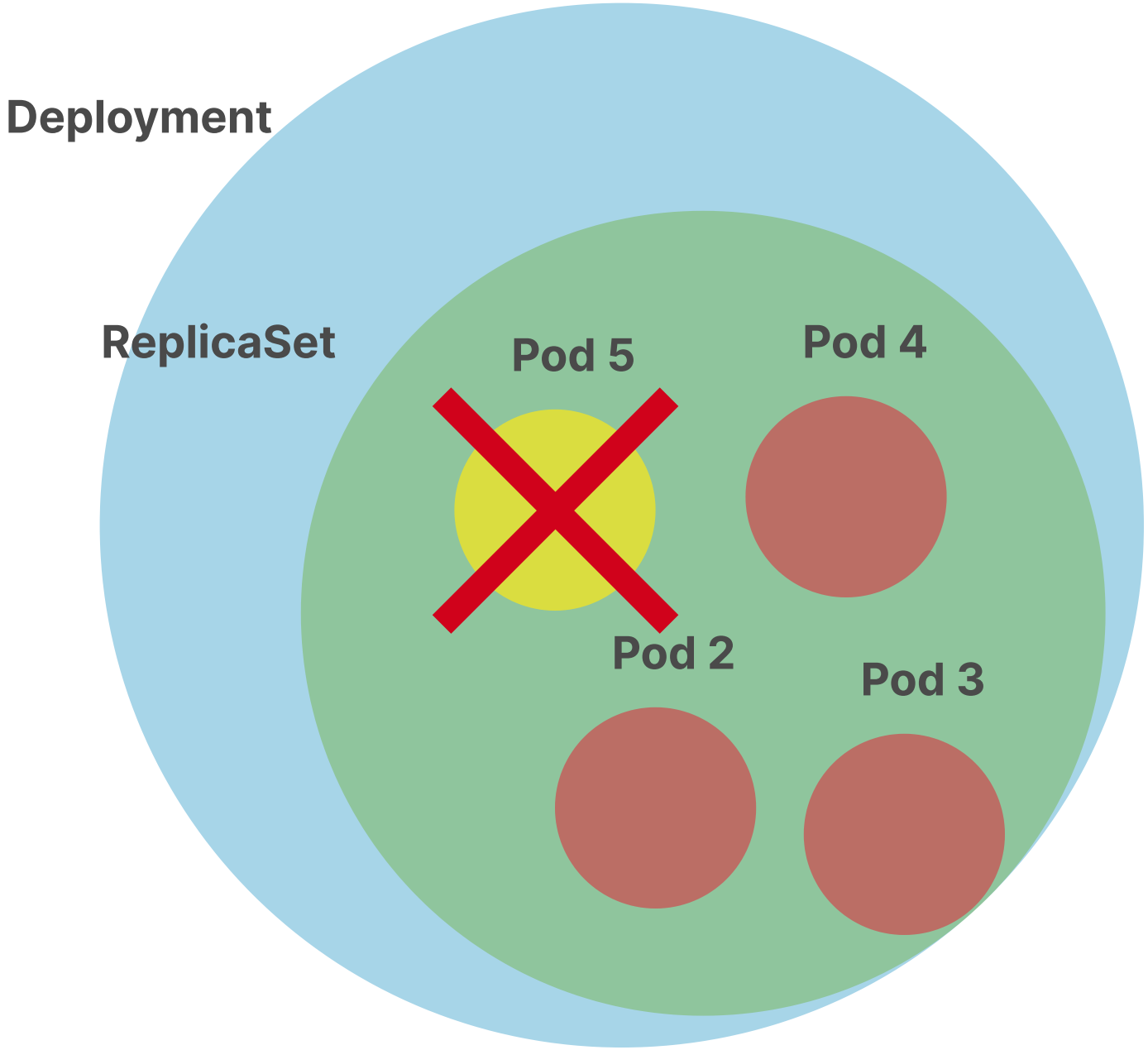

Deployment
3 Replicas
ReplicaSet
3 Replicas

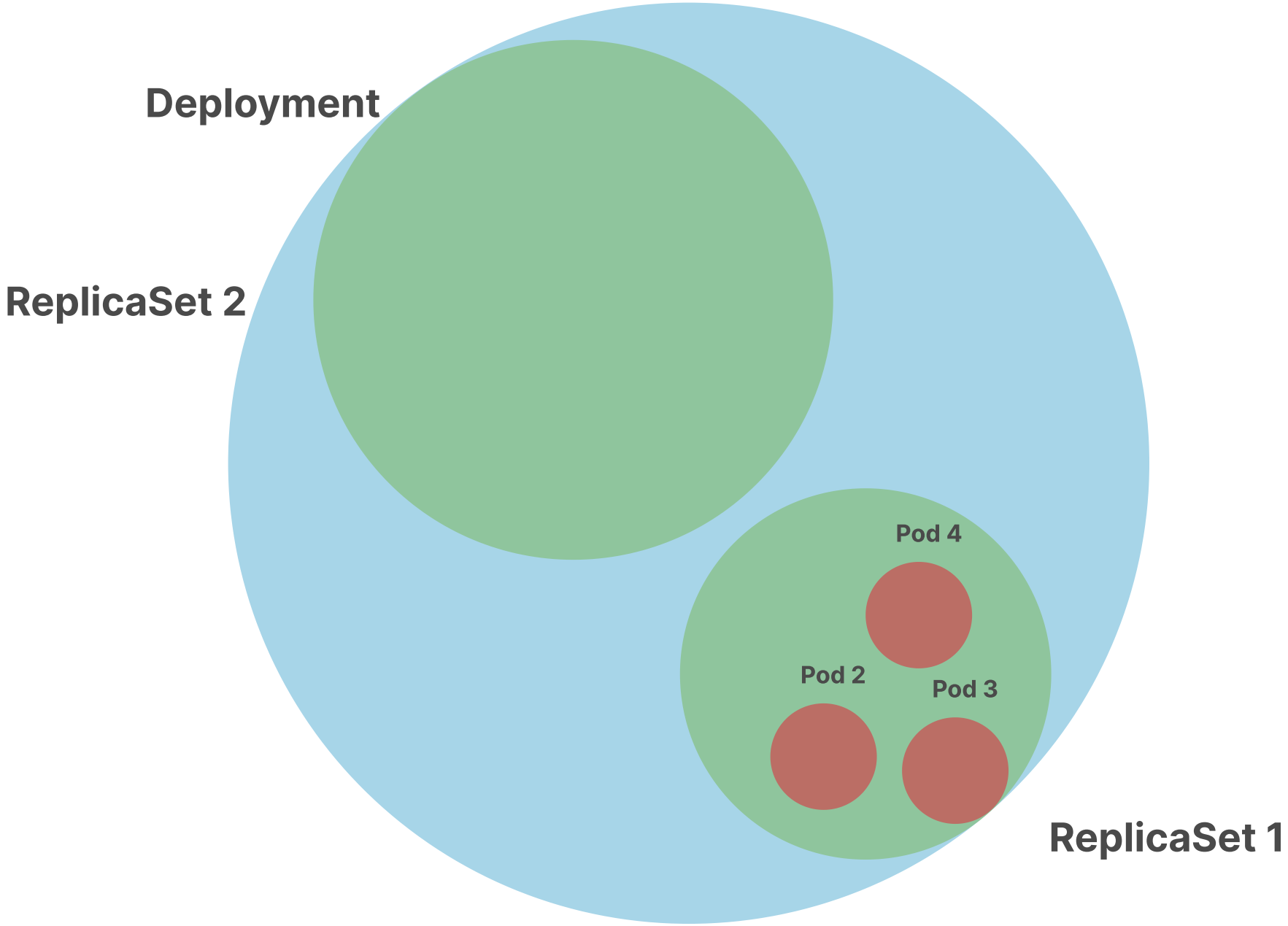

Deployment
3 Replicas
ReplicaSet
3 Replicas
Pod 4
Pod 2
Pod 3

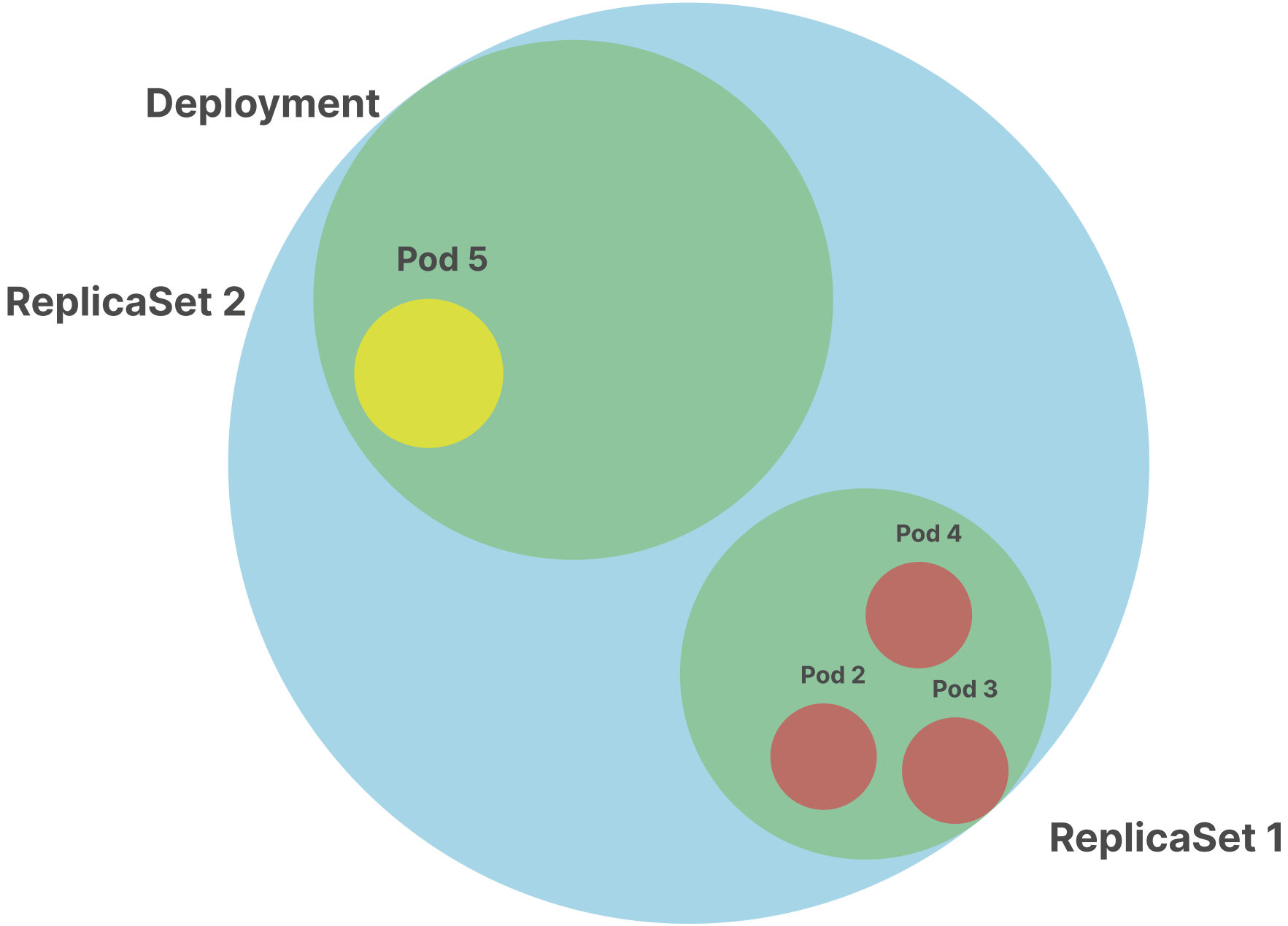
# Rolling updates with ReplicaSets

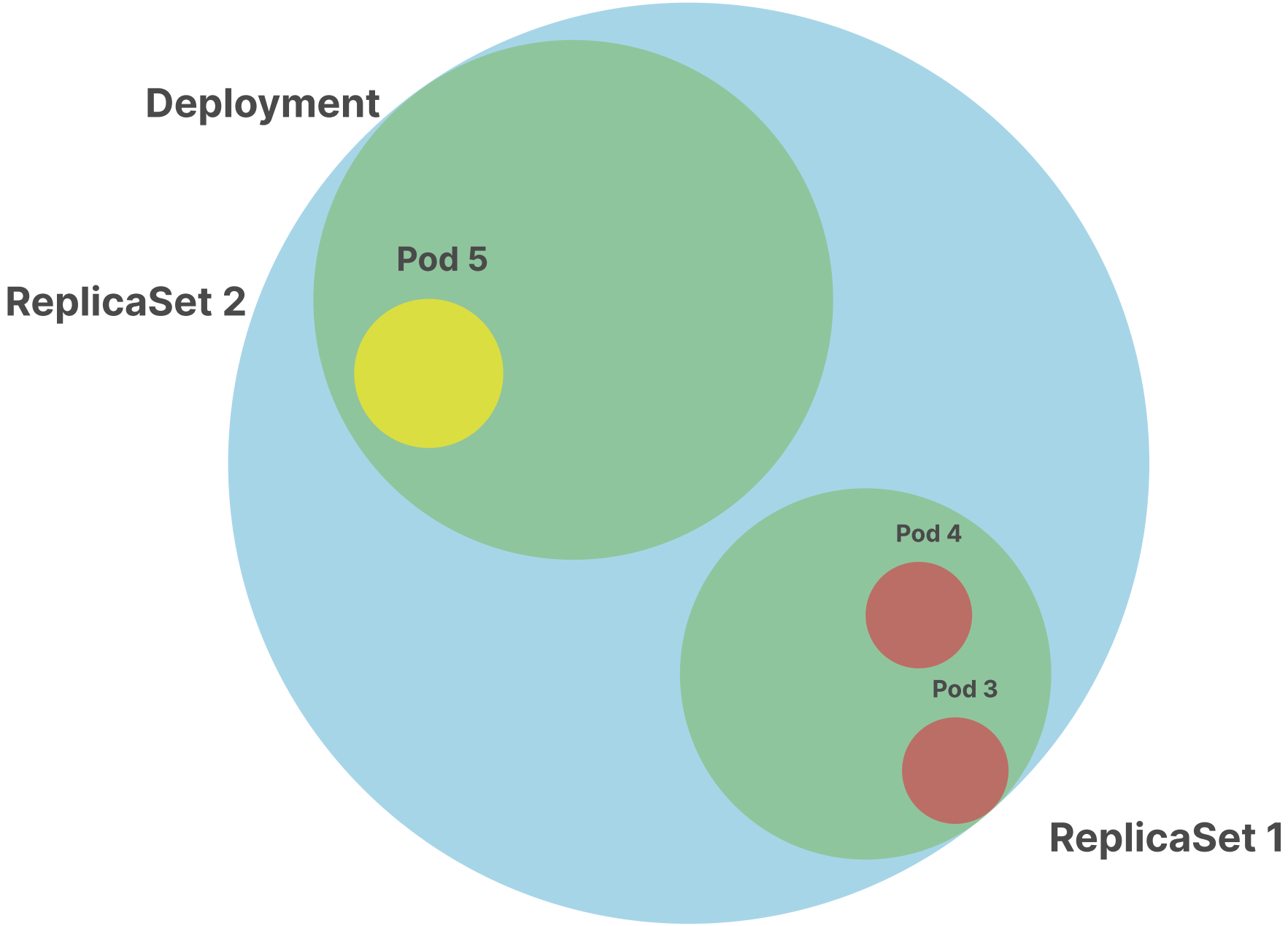

Deployment
Replicaset
Pod 4
Pod 2
Pod 3

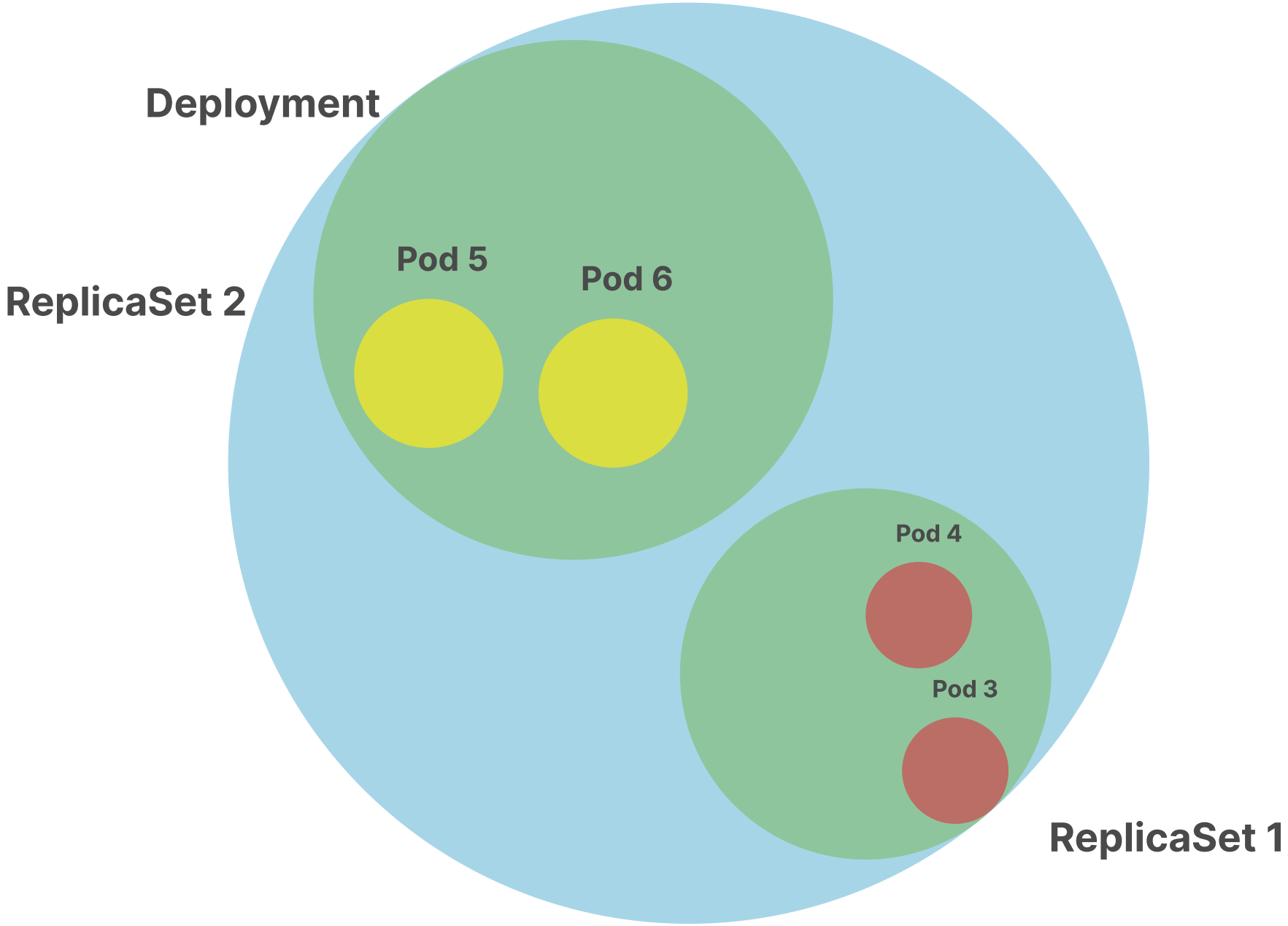

Deployment
ReplicaSet
Pod 5
Pod 4
Pod 2
Pod 3

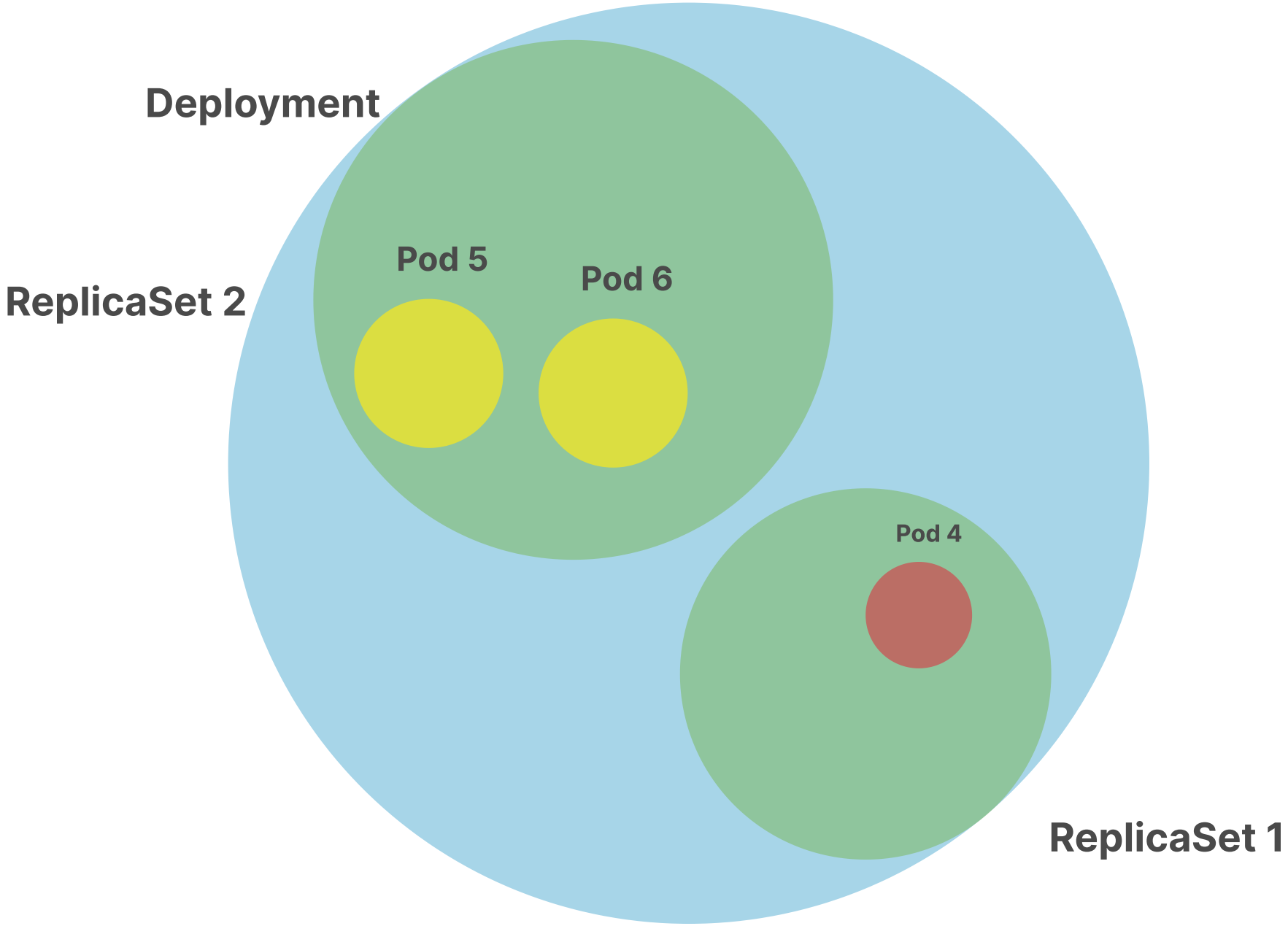

Deployment
ReplicaSet
Pod 5
Pod 4
Pod 2
Pod 3

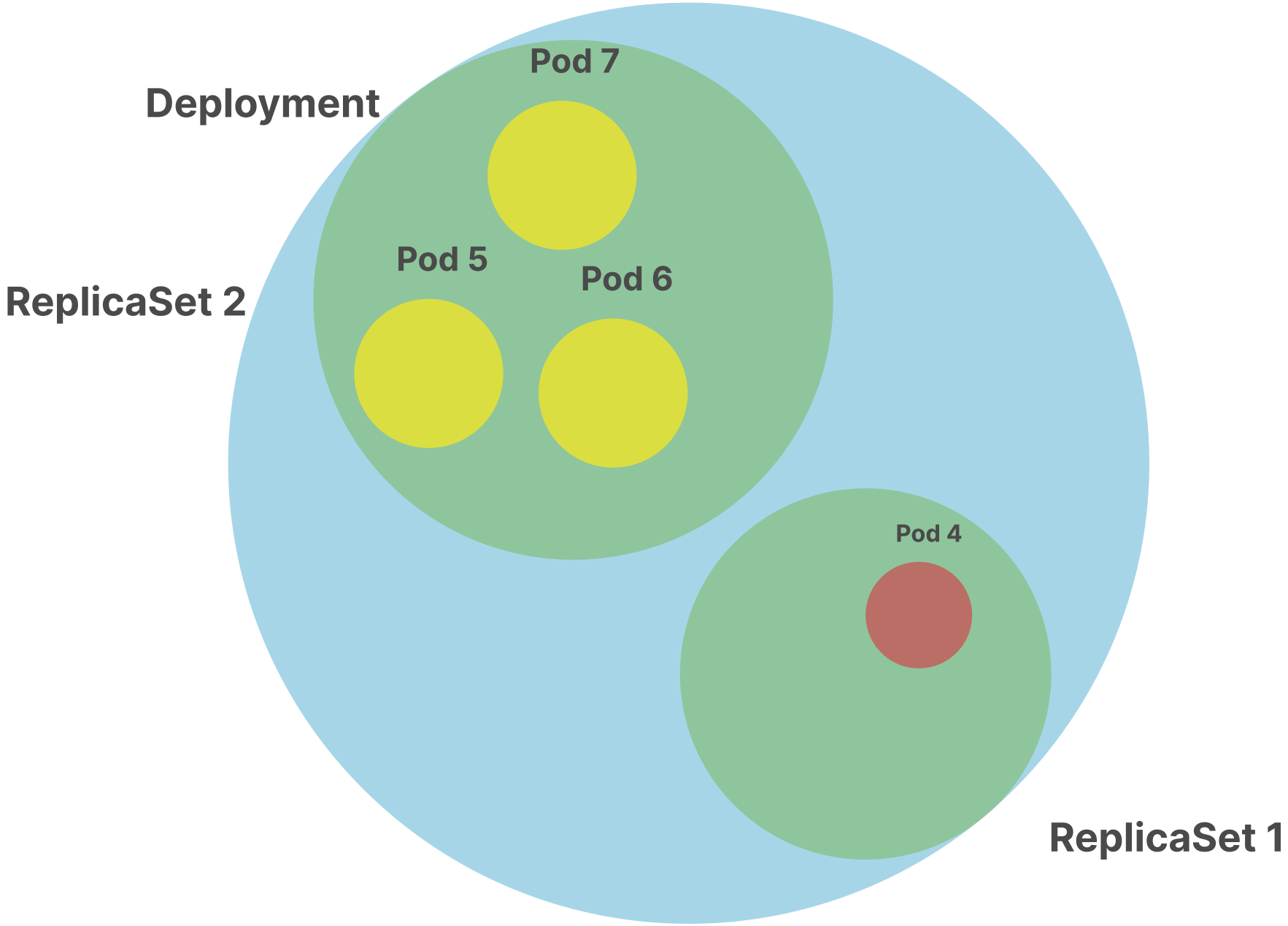

Deployment
ReplicaSet 2
Pod 4
Pod 2
Pod 3
ReplicaSet 1

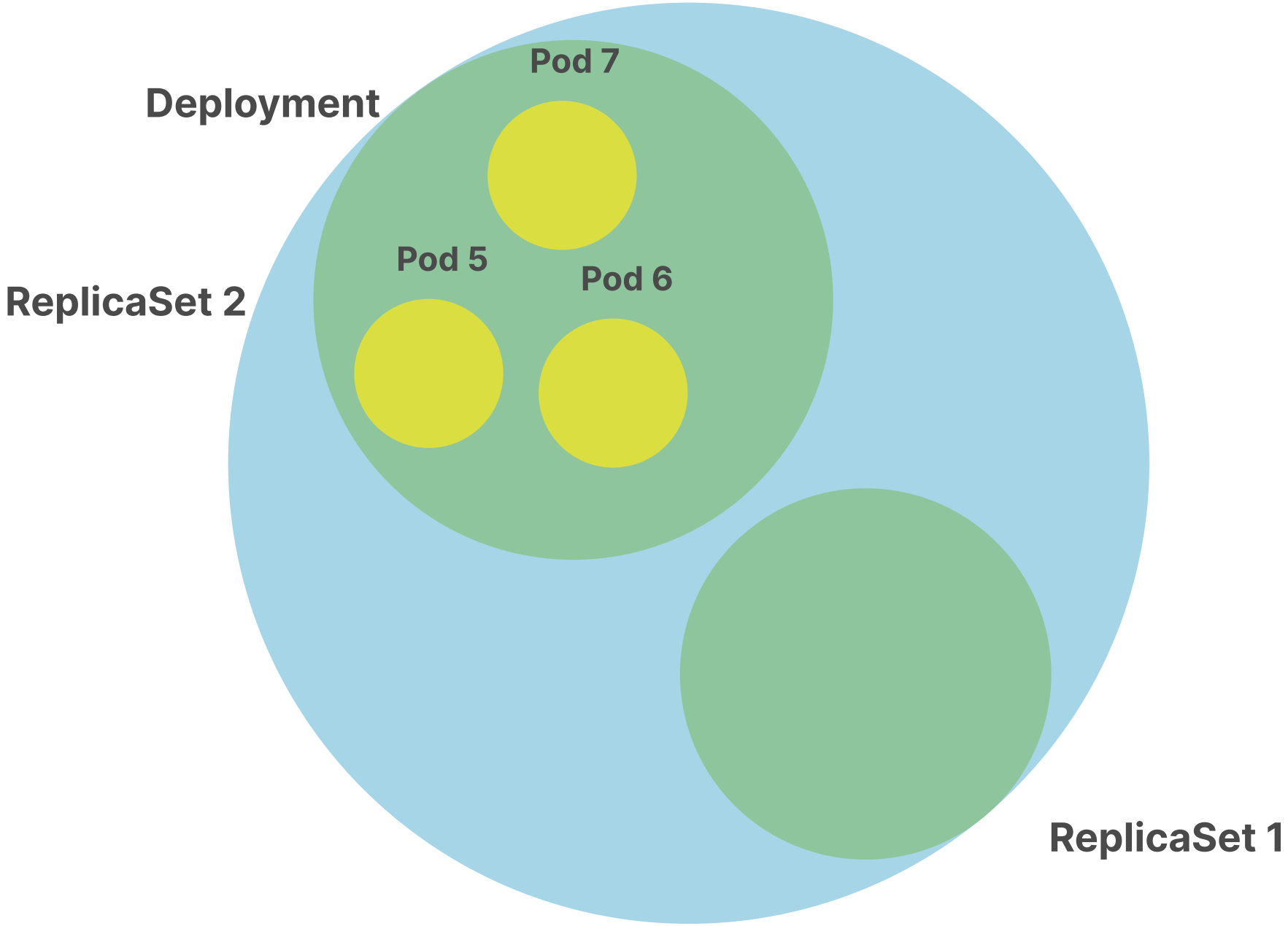

Deployment
ReplicaSet 2
Pod 5
Pod 4
Pod 2
Pod 3
ReplicaSet 1


Deployment
Pod 5
ReplicaSet 2
Pod 4
Pod 3
ReplicaSet 1

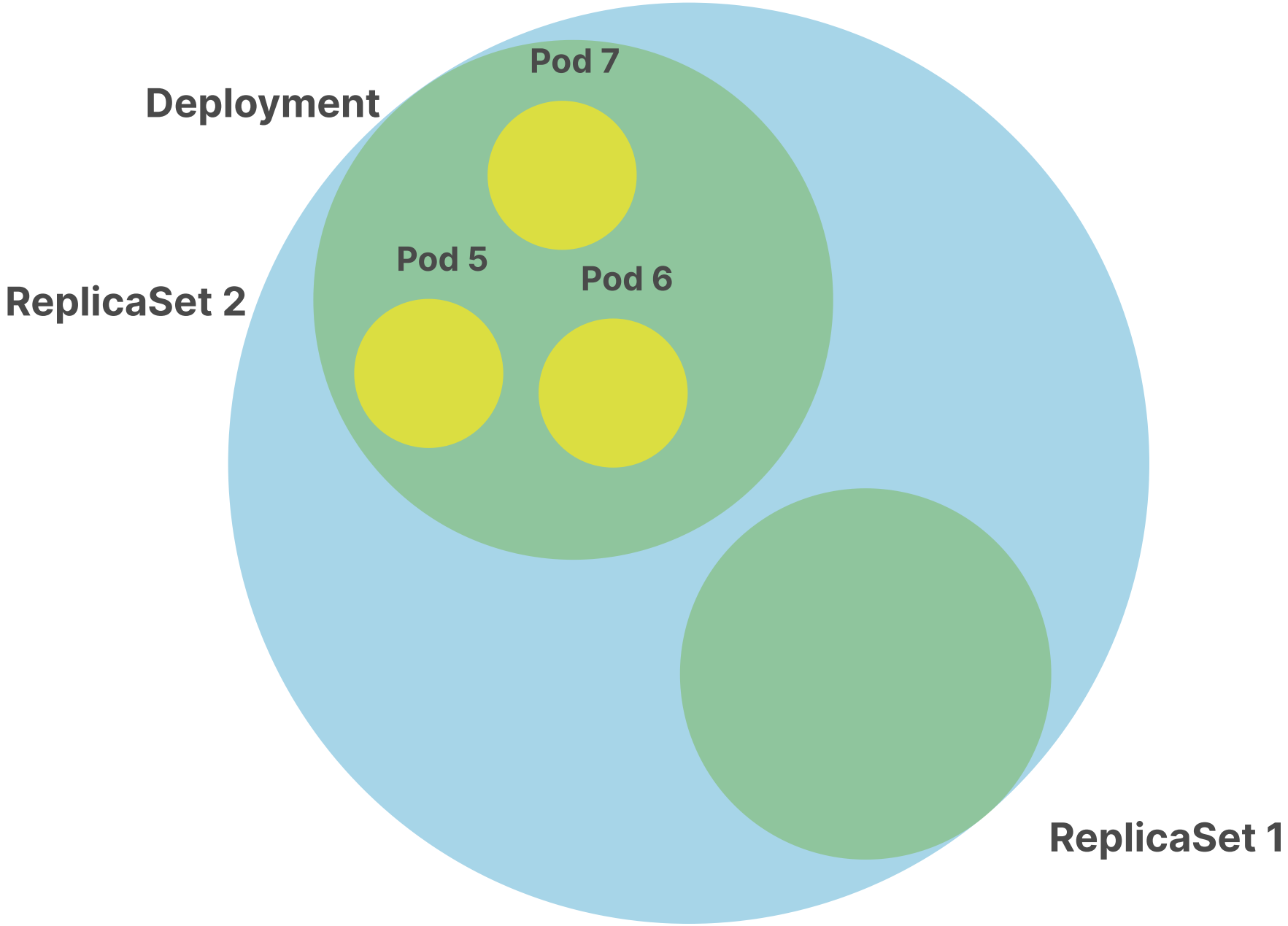

Deployment
ReplicaSet 2
Pod 5
Pod 6
Pod 4
Pod 3
ReplicaSet 1

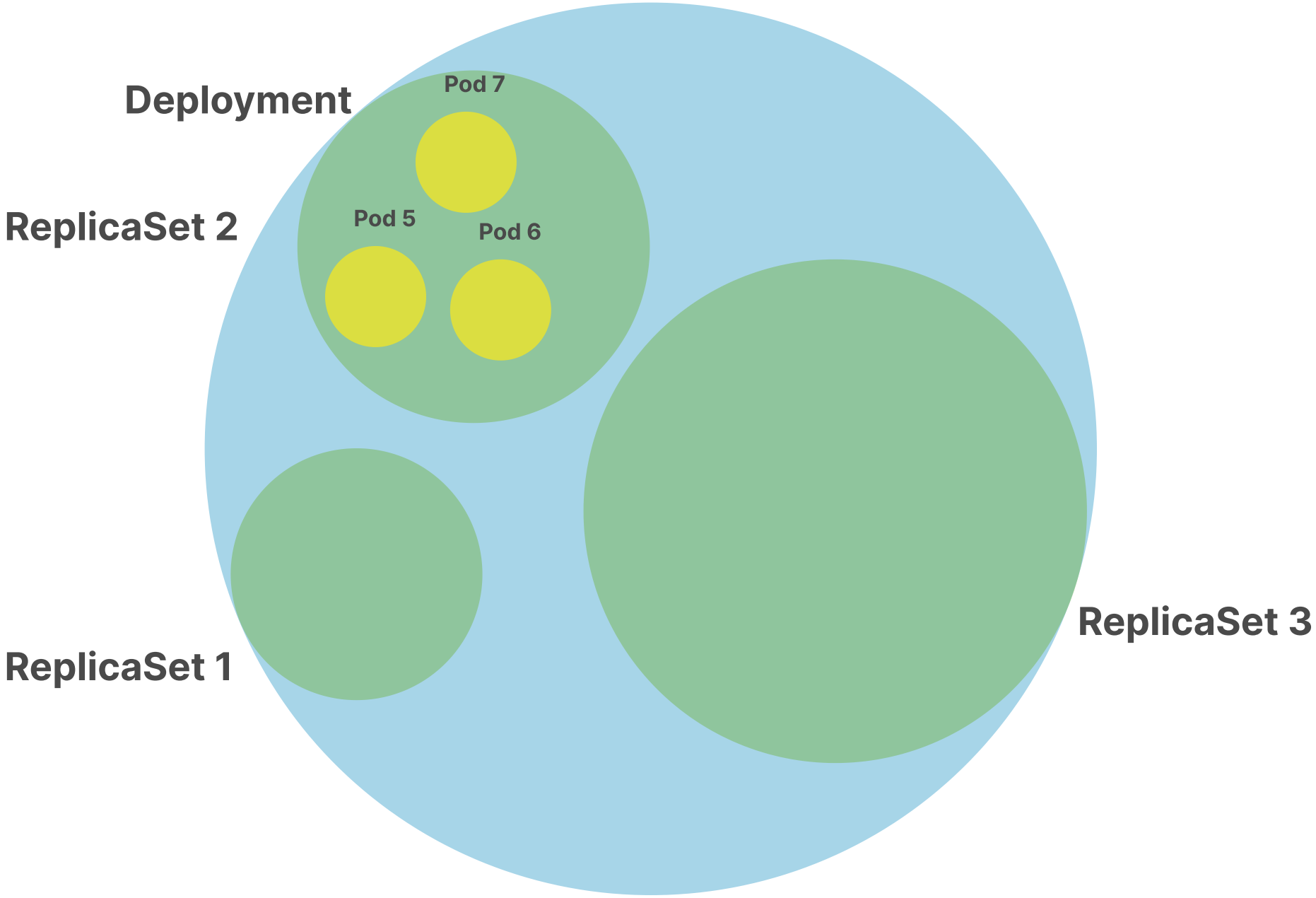

Deployment
ReplicaSet 2
Pod 5
Pod 6
Pod 4
ReplicaSet 1

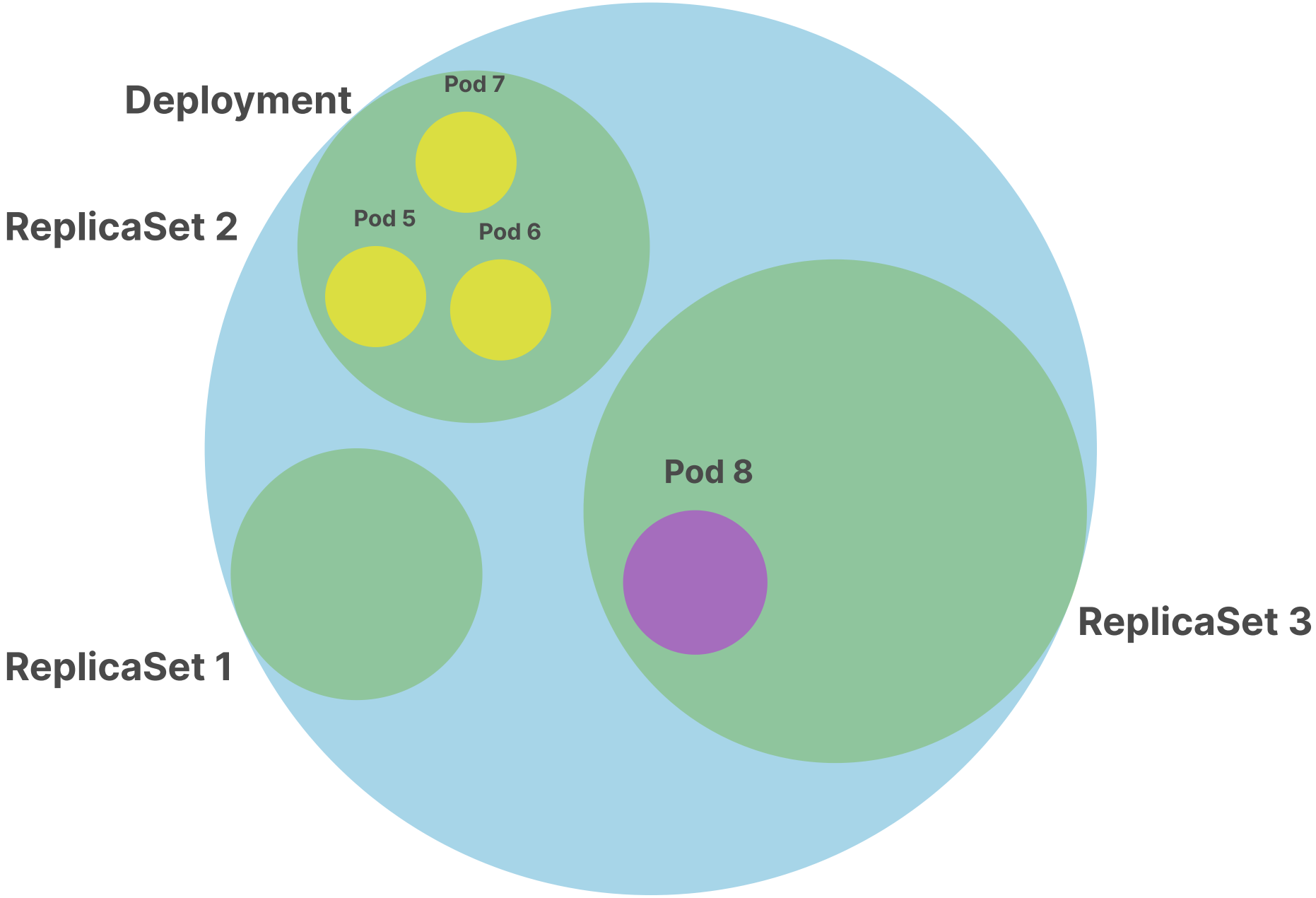

Deployment
Pod 7
Pod 5
Pod 6
ReplicaSet 2
Pod 4
ReplicaSet 1

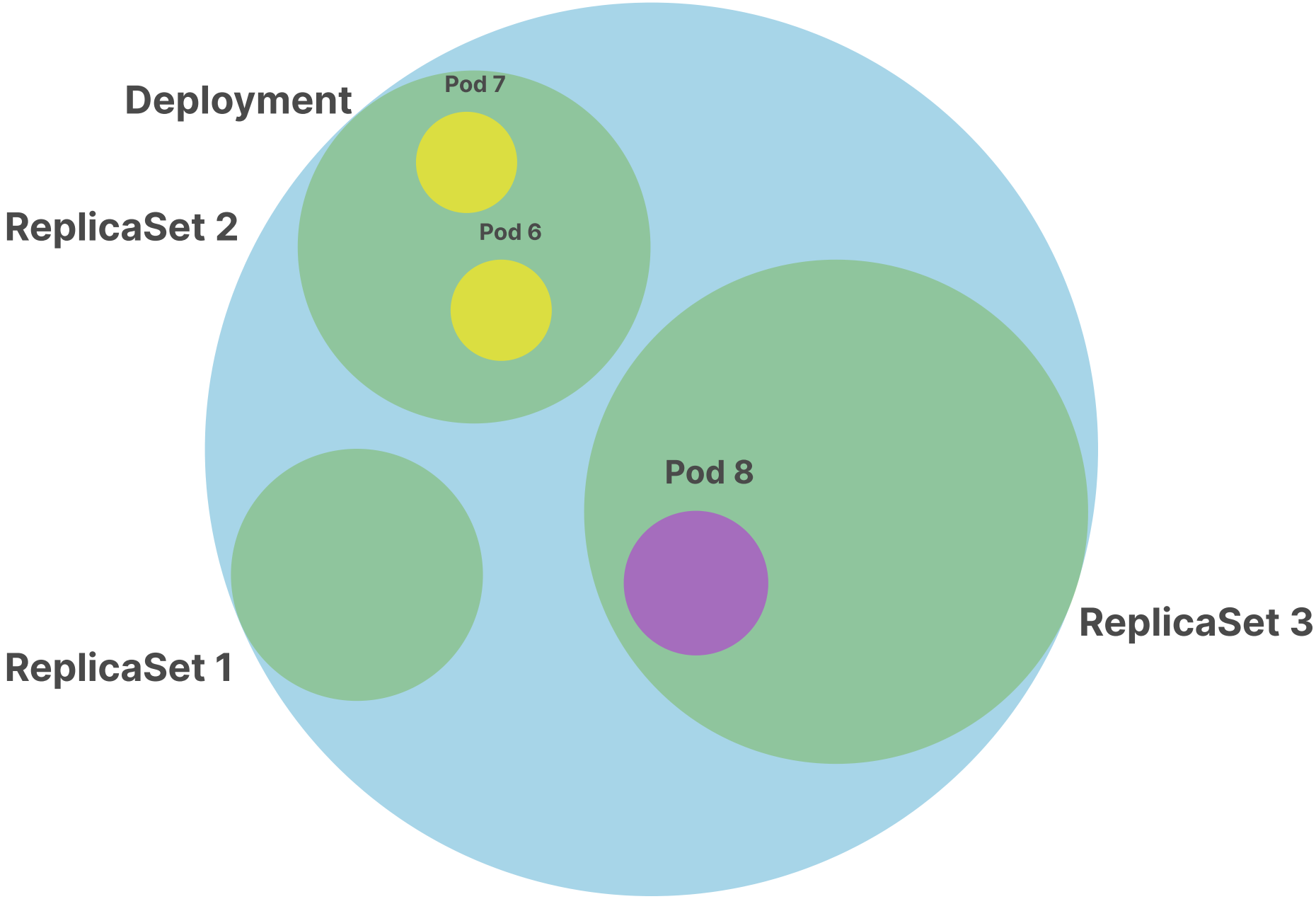

Deployment
Pod 7
Pod 5
Pod 6
ReplicaSet 2
ReplicaSet 1

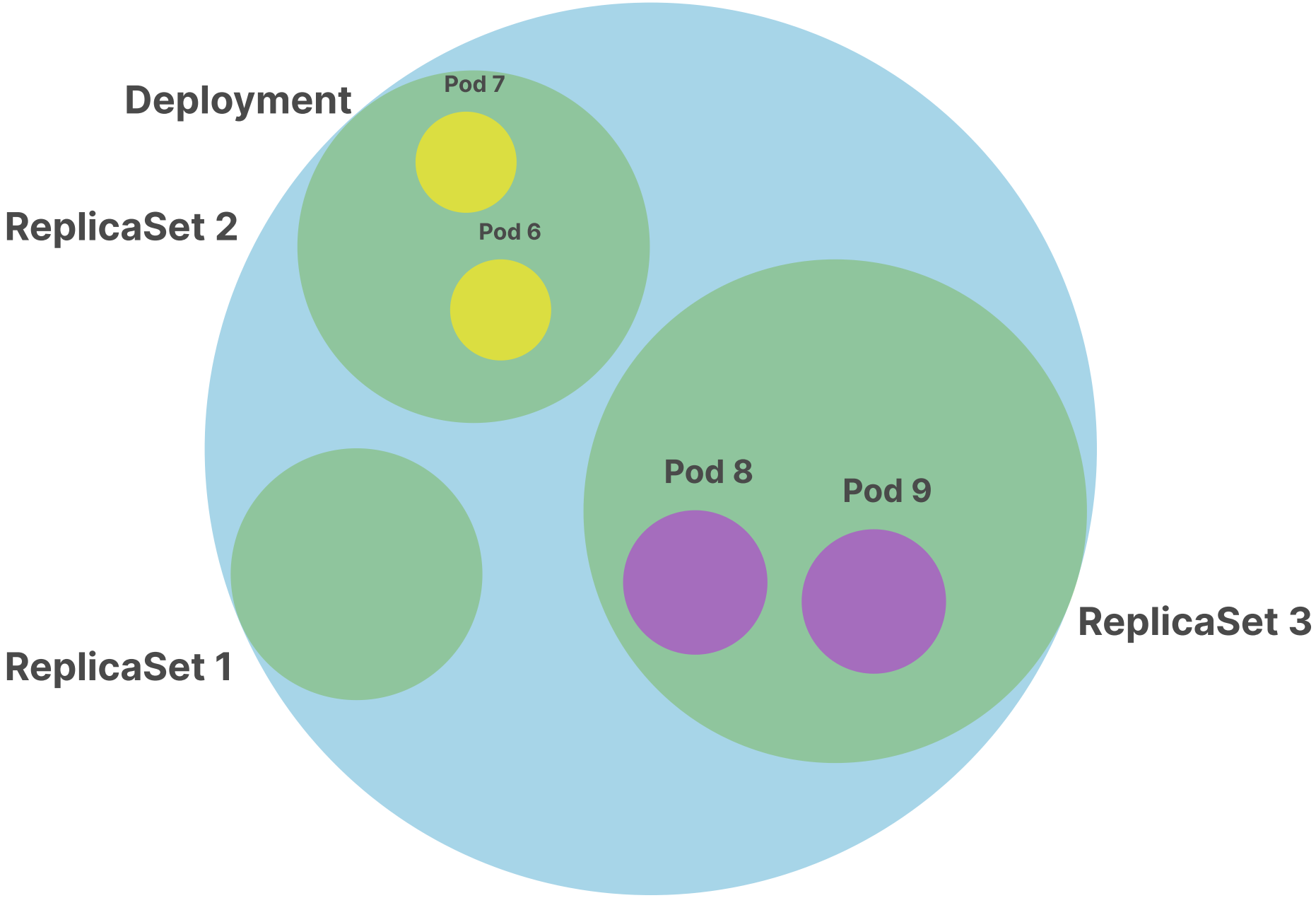
# Rolling updates with ReplicaSets (2)

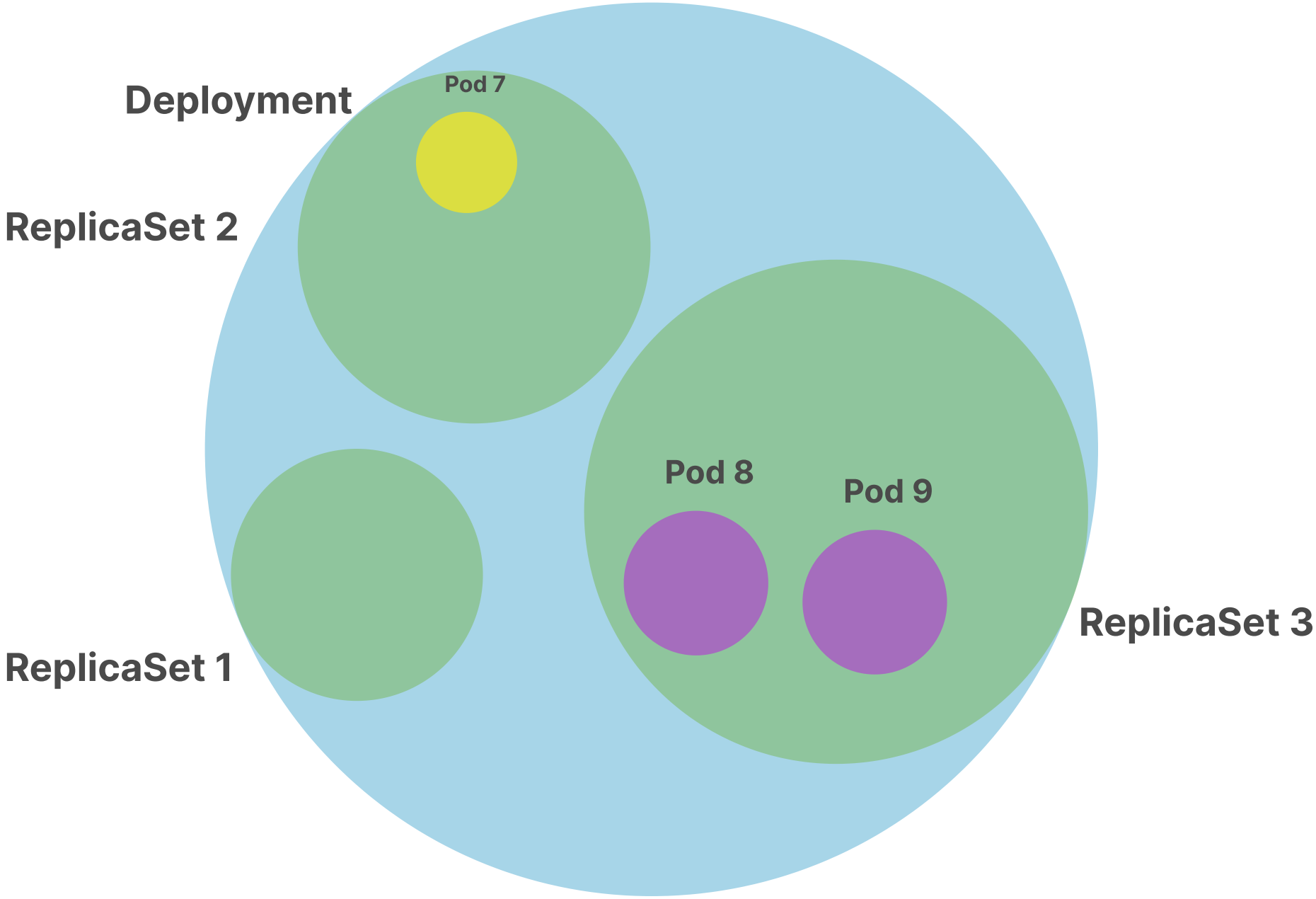

Deployment
Pod 7
Pod 5
Pod 6
ReplicaSet 2
ReplicaSet 1

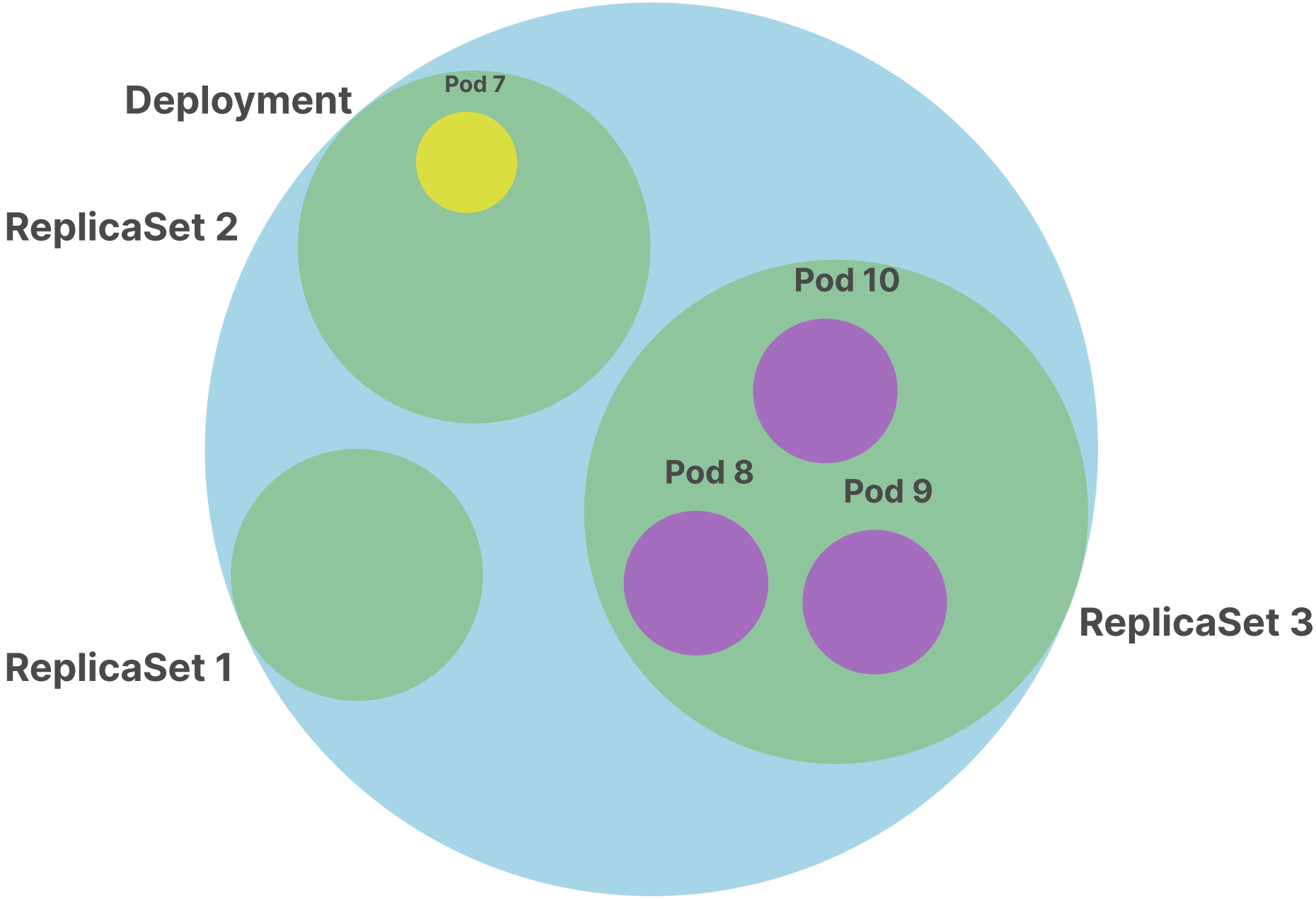

Deployment
Pod 7
ReplicaSet 2
Pod 5
Pod 6
ReplicaSet 1
ReplicaSet 3

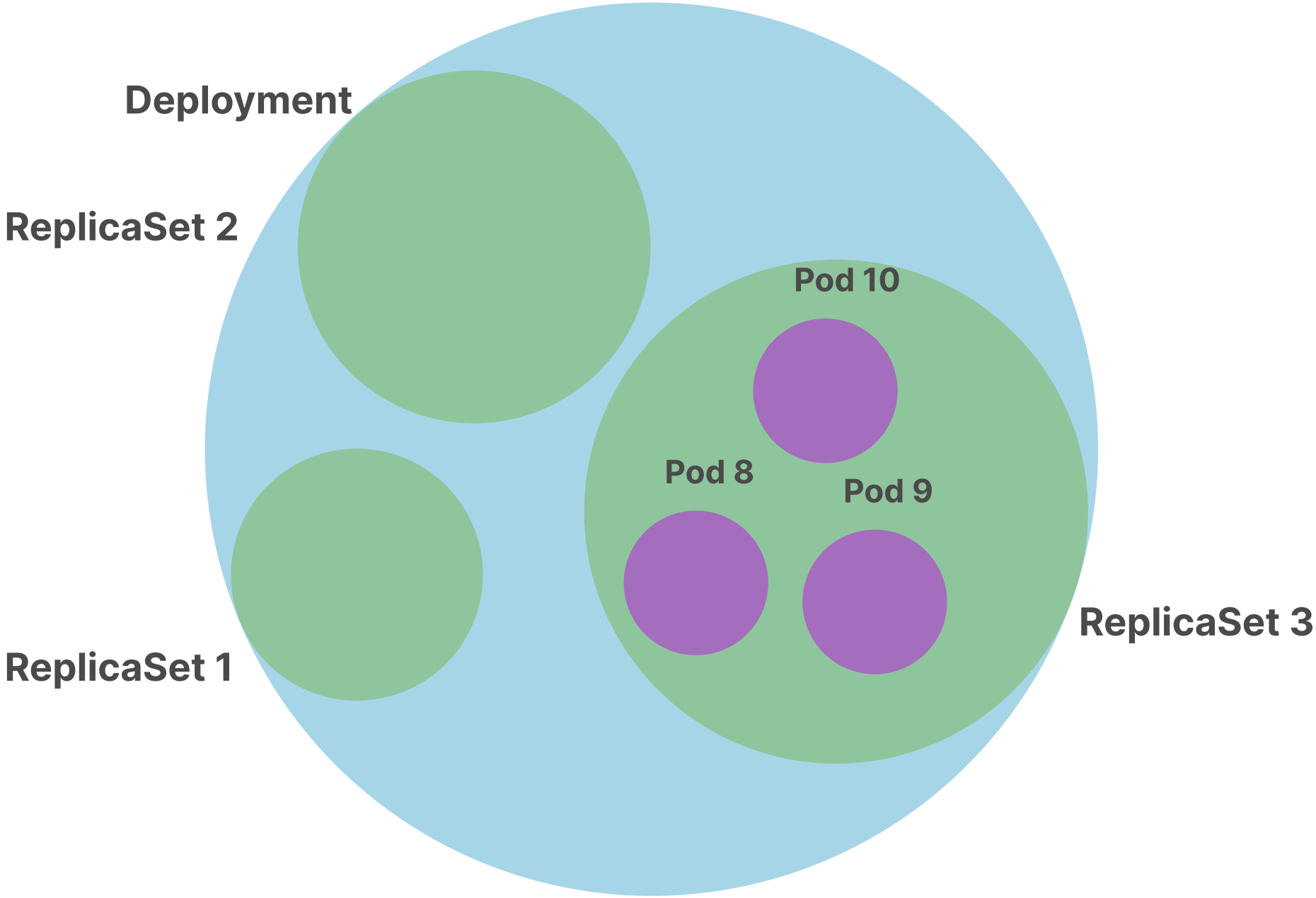

Deployment
Pod 7
ReplicaSet 2
Pod 5
Pod 6
Pod 8
ReplicaSet 3
ReplicaSet 1


Deployment
Pod 7
ReplicaSet 2
Pod 6
Pod 8
ReplicaSet 3
ReplicaSet 1


Deployment
Pod 7
ReplicaSet 2
Pod 6
Pod 8
Pod 9
ReplicaSet 3
ReplicaSet 1


Deployment
Pod 7
ReplicaSet 2
Pod 8
Pod 9
ReplicaSet 3
ReplicaSet 1


Deployment
Pod 7
ReplicaSet 2
Pod 10
Pod 8
Pod 9
ReplicaSet 3
ReplicaSet 1


Deployment
ReplicaSet 2
Pod 10
Pod 8
Pod 9
ReplicaSet 3
ReplicaSet 1

# Deployment orchestration

Revision annotation

revisionHistoryLimit

## Rolling updates with ReplicaSets

Deployment orchestration

**Revision annotation**

revisionHistoryLimit

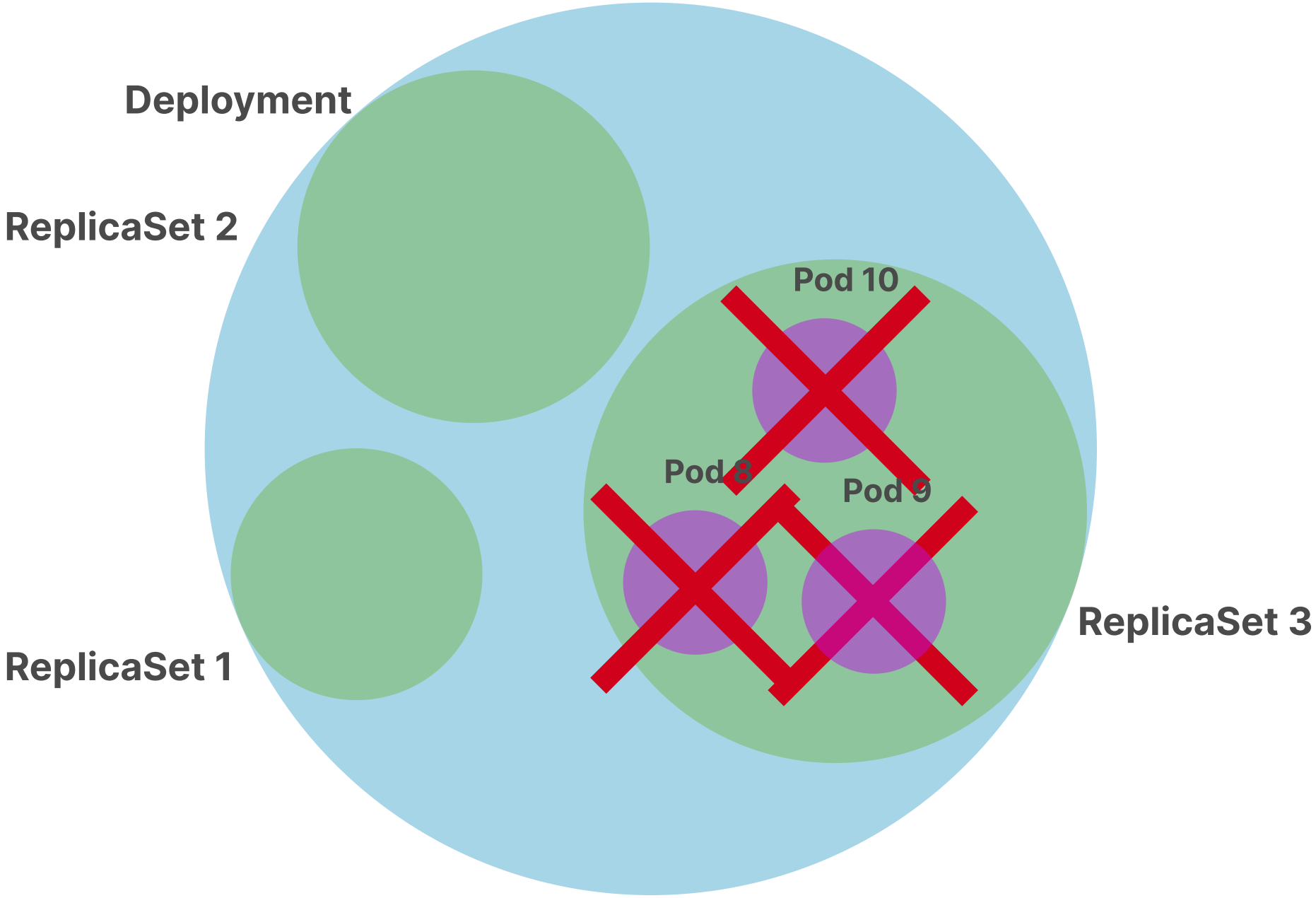
## Rolling updates with ReplicaSets

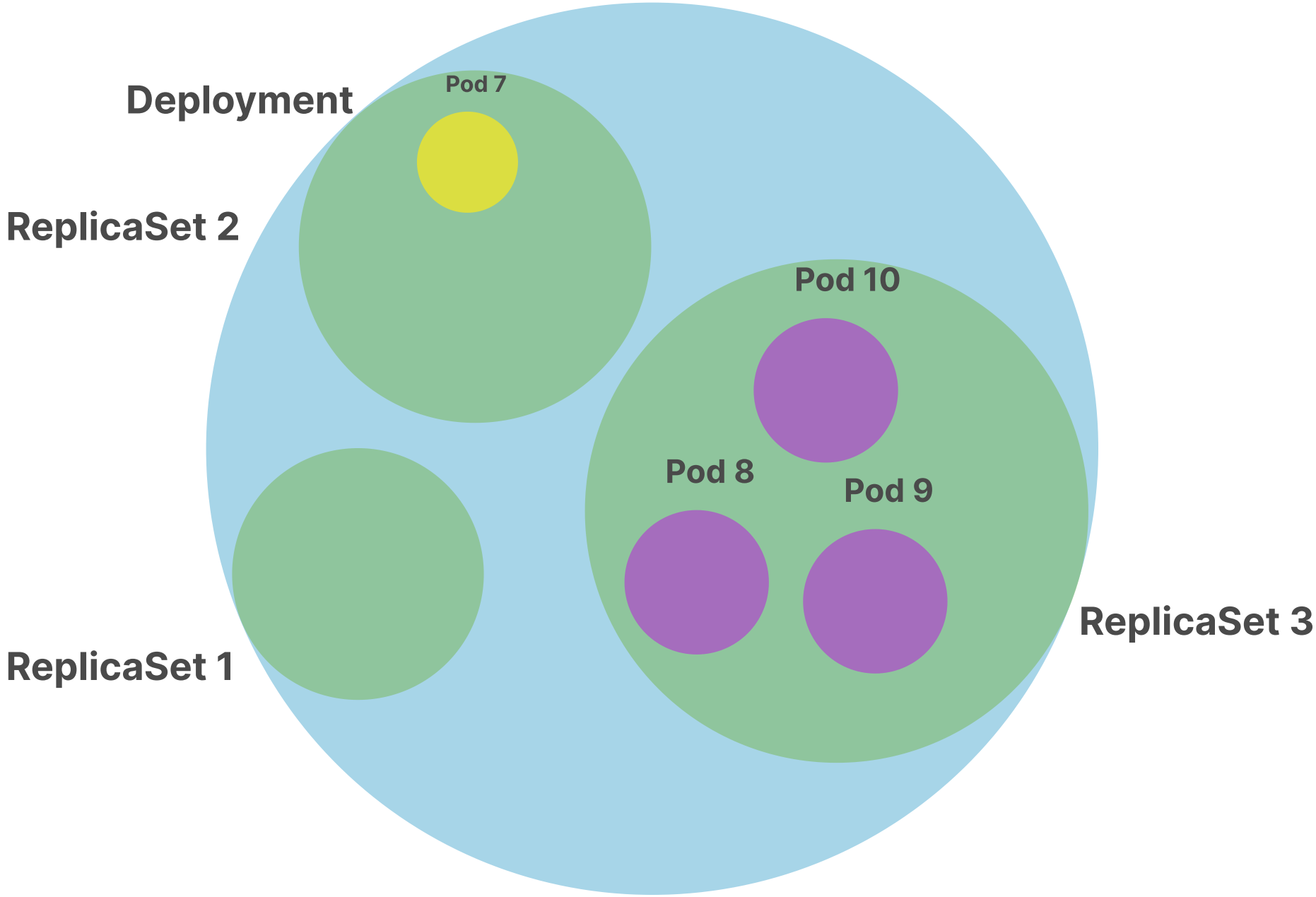
Deployment orchestration

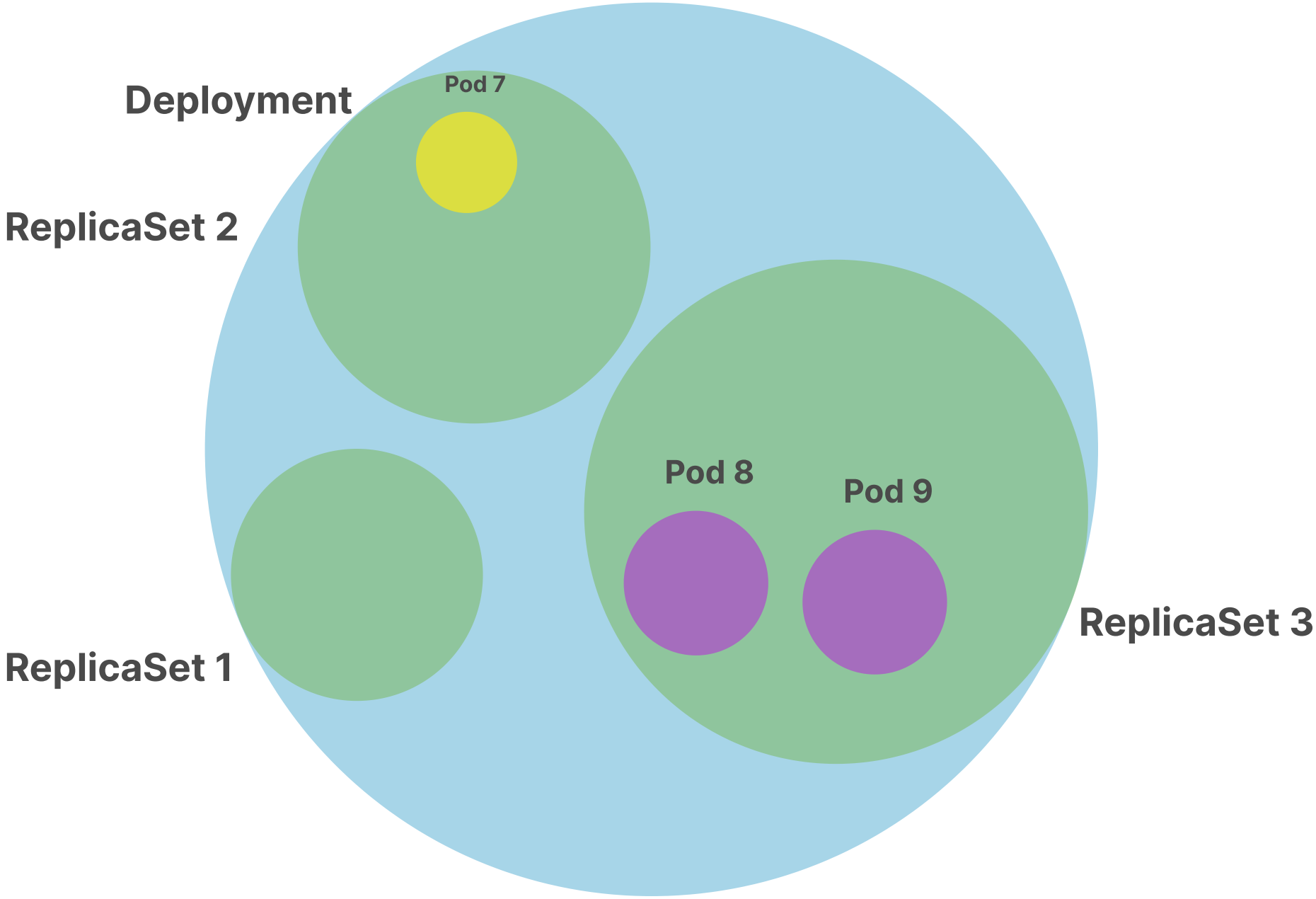
Revision annotation

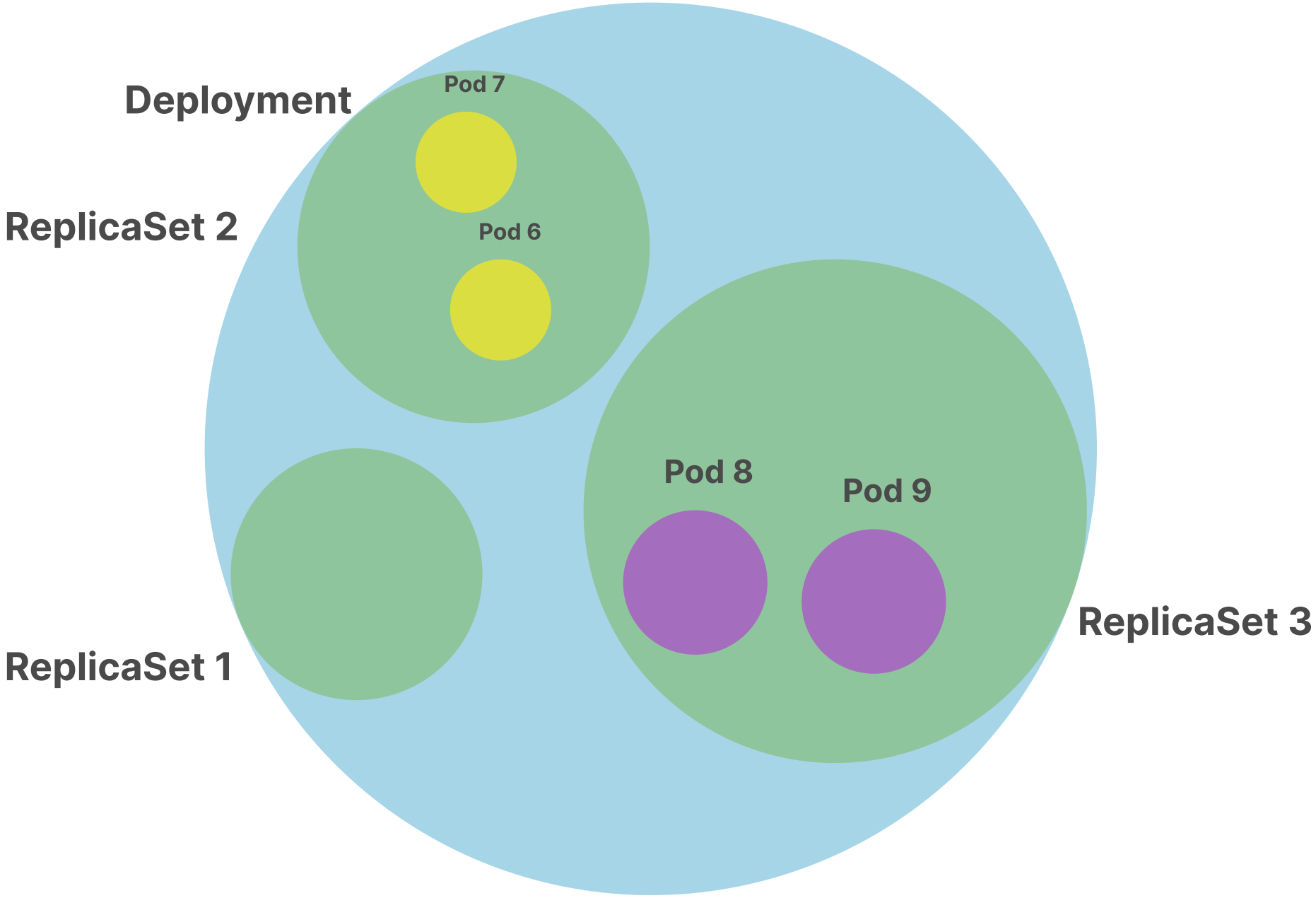
**revisionHistoryLimit**

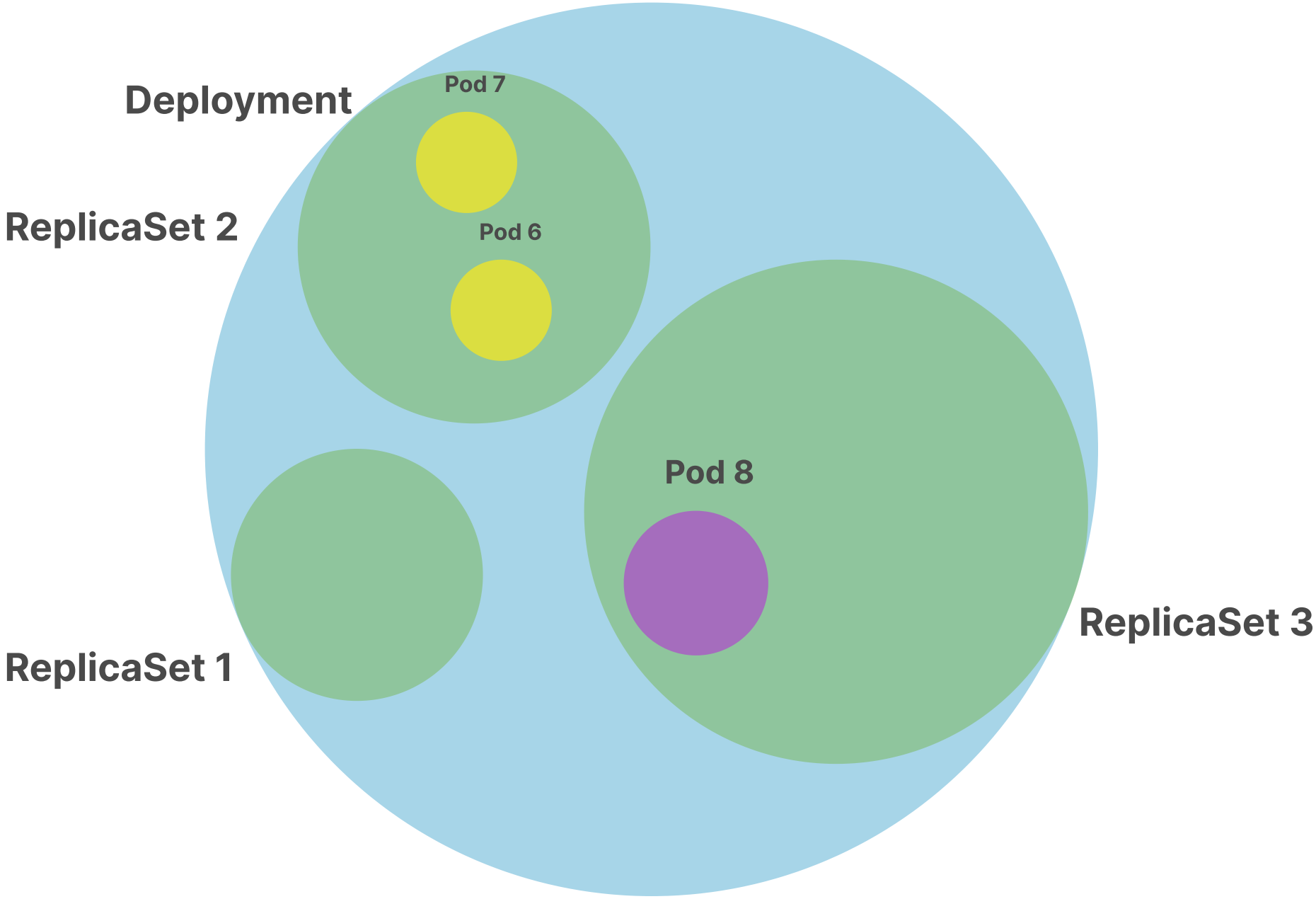
# Rollbacks

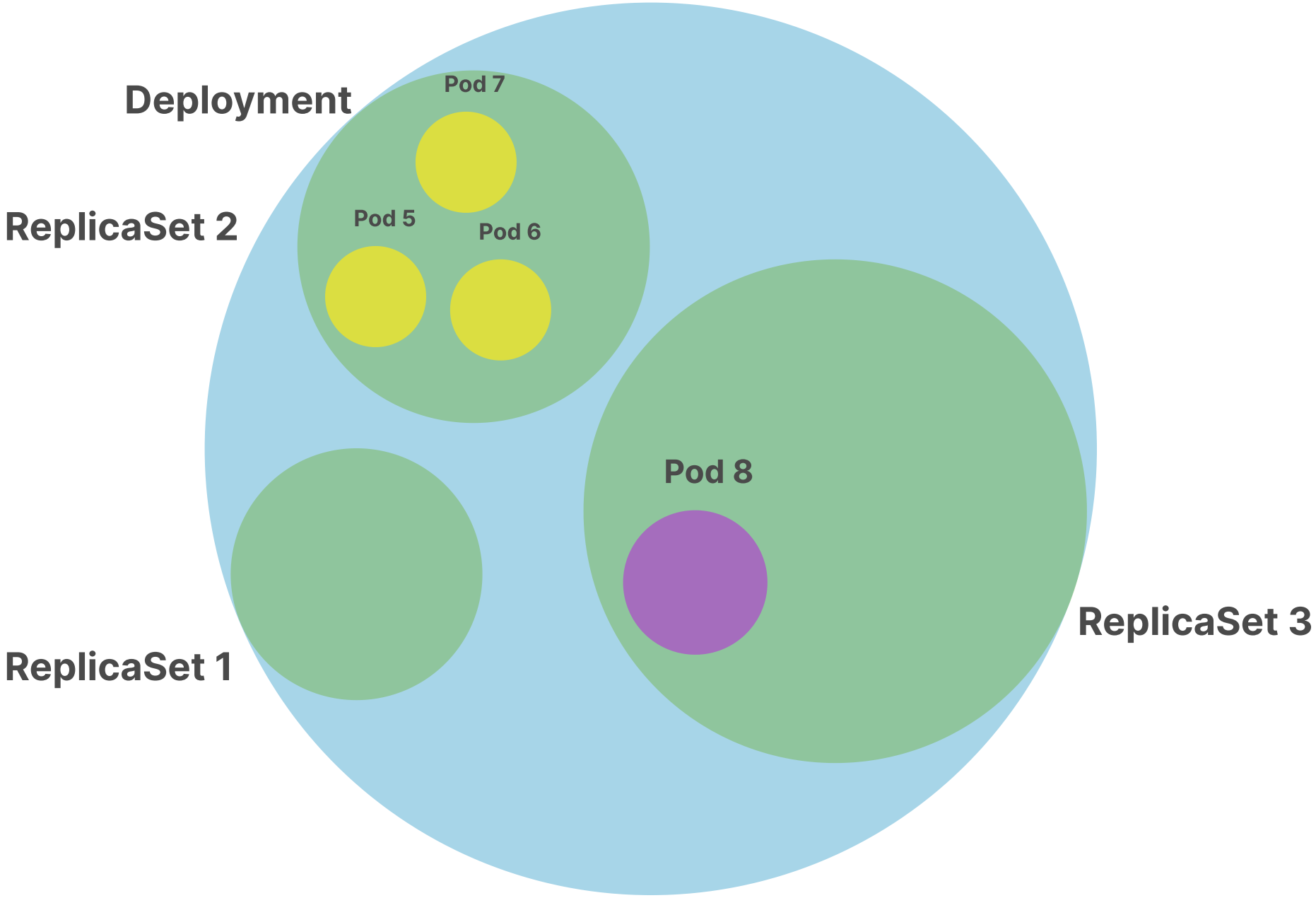

Deployment
ReplicaSet 2
ReplicaSet 1
Pod 10
Pod 8
Pod 9
ReplicaSet 3

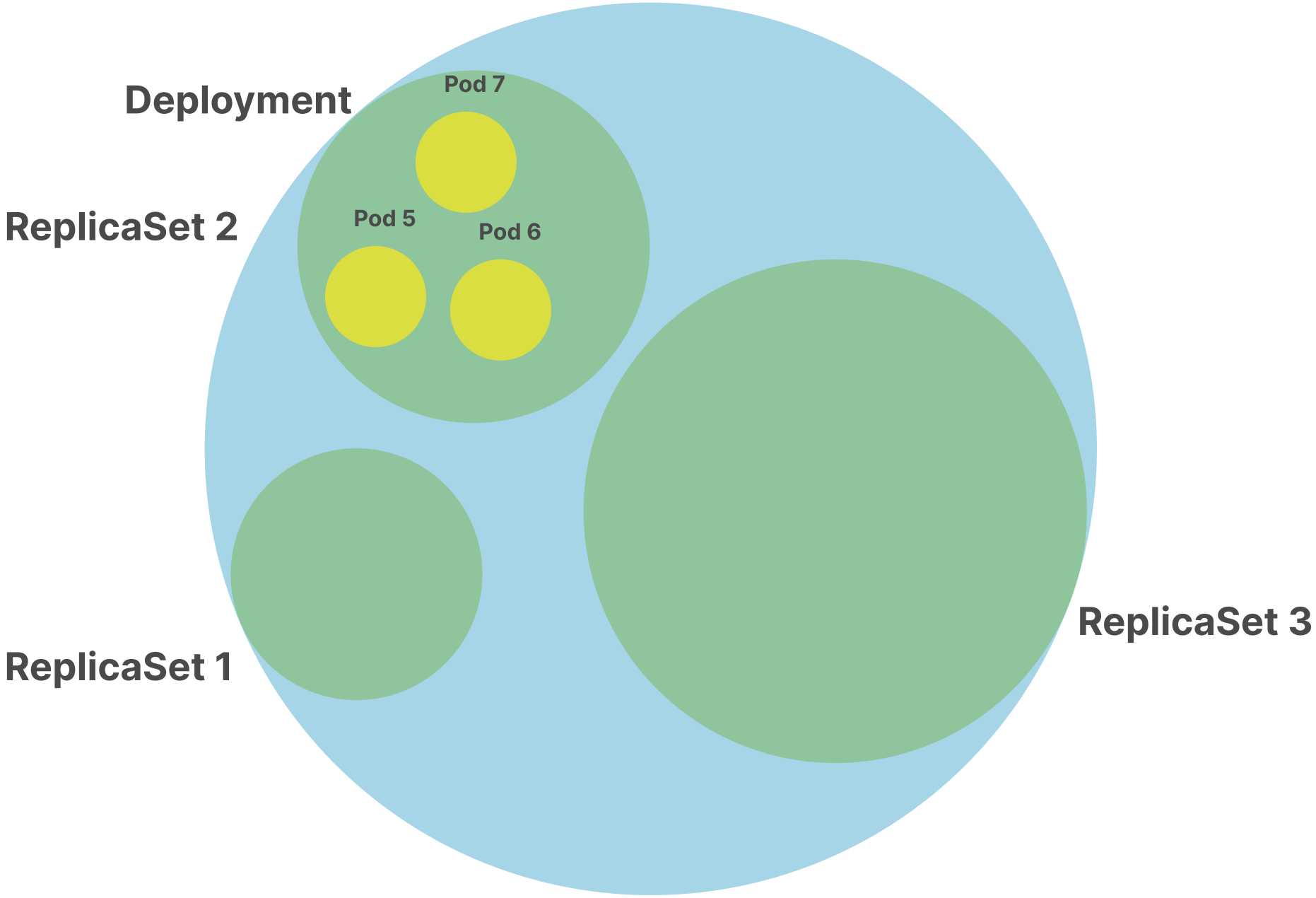

Deployment
Pod 7
ReplicaSet 2
Pod 10
Pod 8
Pod 9
ReplicaSet 3
ReplicaSet 1


Deployment
Pod 7
ReplicaSet 2
Pod 8
Pod 9
ReplicaSet 3
ReplicaSet 1


Deployment
Pod 7
ReplicaSet 2
Pod 6
Pod 8
Pod 9
ReplicaSet 3
ReplicaSet 1


Deployment
Pod 7
ReplicaSet 2
Pod 6
Pod 8
ReplicaSet 3
ReplicaSet 1


Deployment
Pod 7
ReplicaSet 2
Pod 5
Pod 6
Pod 8
ReplicaSet 3
ReplicaSet 1


Deployment
Pod 7
ReplicaSet 2
Pod 5
Pod 6
ReplicaSet 1
ReplicaSet 3

```
$ kubectl rollout \
  undo deployment app \
   --to-revision=2
```

```
$ kubectl rollout \
  history deployment/app
```

# Recap

Deployments & ReplicaSets

# Recap — Deployments & ReplicaSets

1. Deployment, ReplicaSet, Pods

2. Rolling update orchestration

3. ReplicaSet & rollback

# Recap — Deployments & ReplicaSets

1. Deployment, ReplicaSet, Pods

2. Rolling update orchestration

3. ReplicaSet & rollback

# Recap — Deployments & ReplicaSets

1. Deployment, ReplicaSet, Pods

2. Rolling update orchestration

3. ReplicaSet & rollback

# Thanks.